mdstats Dynamical Framework and Density Plotting Architecture Standard

Normative architecture for registered periodic density fields, basin-resolved adaptive widths, resource-aware parallel execution, and hard-budget browser visualization

mdstats architecture manual

2026-08-10

Contents

1 Document status

This document is the **single normative architecture standard and implementation plan** for plotting dynamical periodic frameworks and their associated atomic and framework density fields in `mdstats`.

It consolidates:

- the implemented plotting contracts, diagnostics, planning, and dense/local-sparse framework channels through `mdstats 0.19.65a0`;
- the dynamical-framework scene architecture;
- atomic, framework-vertex, and framework-edge density semantics;
- the corrected dense-to-local-sparse implementation roadmap;
- the revised LD8 exact finite-support execution plan and the hard-budget LD9 browser-mesh optimization plan;
- the implemented LD10 runtime-derived memory, thread, wall-time, and worker policy;
- the implemented LD11 canonical-operator and automatic-backend default policy;
- the implemented LD17 separation of raw density-mesh work, scene visual targets, and standalone terminal limits;
- the PAR-DENS0–PAR-DENS5 long-trajectory density refinement sequence now implemented through optional FP64 GPU execution, with PAR-DENS6 retained as the final end-to-end qualification and auto-tuning gate;
- validation, migration, resource, and failure policies.

The former standalone local-sparse roadmap is superseded. Future planning changes must be made in this manual rather than maintained in a parallel roadmap. The scientific density-value production backend is the completed LD8-S4 hybrid path. LD9-V0 through LD9-V4 are implemented at the functional-acceptance layer; physical-WebGL production-default authorization remains pending external hardware evidence. LD10 replaces scene-fitted host-compute caps with one runtime-derived budget inherited by all density execution layers. LD12 makes Phase-B and Phase-C admission execution-aware: production local-sparse candidates are approved from their exact support-atlas and mixed direct/FFT tile plan rather than from a nominal all-direct contribution count.

The document distinguishes three statuses:

`current baseline` behavior implemented through `mdstats 0.20.118a0`;
`normative target` behavior required by the architecture but not yet fully implemented;
`deferred` behavior requiring a separate approved specification.

The canonical discrete periodized Gaussian and transactional `grid_backend="auto"` are the production defaults for atomic, framework-vertex, and framework-edge fields. Automatic preparation resolves the requested scientific grid and Gaussian bandwidth first, compares exact dense and local-sparse plans at that identical resolution, and approves a whole-scene backend combination before scalar allocation. Localized fields may select sparse storage and broad fields may select dense storage. The dense logical-voxel allowance may not silently broaden an automatic field. The legacy spectral operator remains available only as an explicit dense compatibility mode. No scientific resolution, operator, backend, edge source, or quadrature-policy change may occur without explicit policy and provenance.

2 Source consolidation and authority

This standard incorporates the integration-relevant content from the following package specifications:

Source specification	Architectural material incorporated here
`framework_dynamics_spec.md`	scene ownership, registration, mean framework, trajectories, scene/result records
`atomic_density_spec.md`	atomic measure, CIC deposition, smoothing, adaptive resolution, HDR shells

Source specification	Architectural material incorporated here
`framework_density_spec.md`	vertex and edge measures, edge-source policies, quadrature, dimensional separation
`atomic_mean_graph_spec.md`	periodic mean atomic positions, edge occupancy, mean atomic net
`framework_topology_graph_spec.md`	projected and atom-resolved framework views, periodic path/winding validation
`atomic_connectivity_graph_spec.md`	authoritative atomic connectivity identity and frame consistency
`graph_view_spec.md`	renderer-independent graph identity and immutability
`periodic_graph_spec.md`	display images, canonical/local/expanded periodic preparation
`graph_styles_spec.md`	chemistry-aware and framework-aware style resolution
`graph_3d_spec.md`	Plotly composition, equal Cartesian scale, trace/resource and export policy
superseded local-sparse roadmap	block-sparse storage goals, corrected and merged here

This manual owns:

- cross-module dependency direction;
- coordinate and registration invariants;
- density-operator identity;
- backend-neutral field and provenance contracts;
- global transactional planning;
- the staged implementation sequence and acceptance gates.

Module-specific specifications remain authoritative only for low-level behavior not repeated here. If any older integration statement or standalone roadmap conflicts with this manual, this manual governs.

Requirements use the terms **must**, **should**, and **may** in their usual normative sense. A gate cannot be passed by weakening a tolerance or redefining a requirement without revising this standard and recording the rationale.

3 Architectural objective

The plot is not one monolithic renderer. It is a composition of independently prepared scientific objects in one registered periodic coordinate system:

$$
\begin{aligned}
&\text{collection} + \text{framework topology} + \text{optional atomic connectivity} \\
&\quad \longrightarrow \text{topology-compatible frame geometry} \\
&\quad \longrightarrow \text{periodic gauge normalization and registration} \\
&\quad \longrightarrow \begin{cases} \text{mean framework graph}, \\ \text{atomic trajectories}, \\ \text{mean atomic connectivity graph}, \\ \text{atomic weighted samples}, \\ \text{framework vertex weighted samples}, \\ \text{framework edge weighted samples}, \end{cases} \\
&\quad \longrightarrow \text{renderer-independent scene} \\
&\quad \longrightarrow \text{periodic graph materialization} + \text{density meshes} \\
&\quad \longrightarrow \text{interactive Plotly artifact}.
\end{aligned}
$$

The architecture must support scientific preparation without Plotly. HTML is an output artifact, not the source of scientific identity or geometry.

4 Design principles

4.1 One scientific gauge

Mean framework vertices, trajectory points, mean atomic vertices, atomic density samples, framework vertex samples, and framework edge samples must use the same registration and display-cell gauge. No overlay may independently fold, align, or recenter its source data.

4.2 Authoritative topology

`FrameworkTopology` and atomic-connectivity results are inputs. Plotting modules do not reconstruct connectivity, search for replacement linker paths, change atom roles, or average incompatible graphs.

4.3 Geometry before rendering

Every scientific object is prepared as immutable numerical data. Plotly performs interactive rendering only. It does not define the density estimator, topology, periodic winding, shell threshold, or normalization.

4.4 Physical dimensions remain explicit

Atomic and framework-vertex occupancy are volumetric number measures. Framework-edge density is an arc-length measure per volume. Separate field objects, units, labels, and legend groups are mandatory.

4.5 Resource decisions are transactional

Potentially large requests are estimated and rejected before large partial allocations. The code does not silently coarsen grids, drop paths, remove edges, truncate frames, or decimate meshes.

4.6 Backend equivalence under a declared operator

Every field records a stable smoothing-operator identifier. Dense and sparse backends must produce the same field for the same declared operator, logical grid, samples, and options.

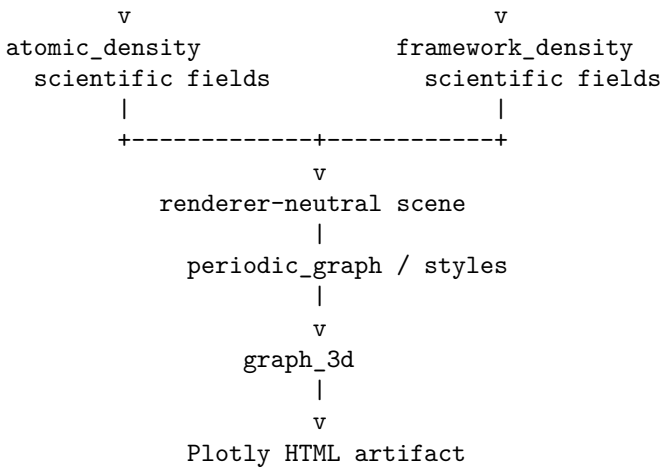
The current `0.19.39a0` spectral operator and the target finite-support discrete operator are not mathematically identical on an under-resolved grid. The transition therefore follows an explicit migration policy. No backend may claim estimator preservation across different operator identifiers.

Direct Gaussian KDE over raw samples remains a different estimator because it omits the CIC assignment kernel.

5 Dependency direction and module ownership

The required dependency direction is

```
analysis / collection objects
        |
        v
scientific plotting adapters
  - framework_topology_graph
  - atomic_connectivity_graph
        |
        v
framework_dynamics scientific preparation
  - registration
  - mean geometry
  - path construction
  - weighted density samples
        |
        +--------------------------+
        |                          |
```



The generic graph renderer must not import framework topology, trajectory, atomic connectivity, or density semantics. The density modules must not import Plotly for scientific preparation.

As of `mdstats 0.20.148a0`, **universal scene composition is owned by GFX3D**, not by this density/framework manual. `FrameworkDynamicsScene` and `FrameworkDynamicsRenderResult` remain qualified scientific/compatibility products, while new user-facing composition goes through `GraphicsScene3DRequest`, registered GFX3D layers, and `mdstats-3d`. This manual remains authoritative for framework registration, connectivity-derived products, atomic/framework density estimators, HDR semantics, resource planning, and density-specific rendering evidence. The canonical generic scene/layer/CLI architecture is defined in `mdstats_3d_graphics_architecture.md`; future ring, cage, site, and kinetics layers must not be added here merely to extend a visualization container.

6 Public scene API

The implemented preparation entry point is:

```python
def prepare_framework_dynamics_scene(
    collection: AtomisticFrameCollection,
    topology: FrameworkTopology,
    *,
    frame_indices: Sequence[int] | None = None,
    display_mode: FrameworkGraphDisplayMode | str = "projected",
    trajectory_selection: TrajectoryAtomSelection | None = None,
    atomic_connectivity: AtomicConnectivityState | AtomicConnectivityResult | None = None,
    atomic_mean_graph_options: AtomicMeanGraphOptions | None = None,
    atomic_density_selections: Sequence[AtomicDensitySelection] | None = None,
    atomic_density_options: AtomicDensityOptions | None = None,
    framework_density_options: FrameworkDensityOptions | None = None,
    options: FrameworkDynamicsOptions | None = None,
    resources: FrameworkDynamicsResources | None = None,
) -> FrameworkDynamicsScene:
    ...
```

The renderer is:

```python
def plot_framework_dynamics_3d(
    scene: FrameworkDynamicsScene,
    *,
    periodic: PeriodicDisplayOptions | None = None,
    style: GraphStyle | None = None,
    focus: GraphFocus | None = None,
    graph_filter: GraphFilter | None = None,
    complexity_policy: GraphComplexityPolicy | None = None,
    graph_options: Graph3DRenderOptions | None = None,
    atomic_mean_graph_options: AtomicMeanGraph3DRenderOptions | None = None,
```

```
    trajectory_options: Trajectory3DRenderOptions | None = None,
    density_options: AtomicDensity3DRenderOptions | None = None,
    framework_density_options: FrameworkDensity3DRenderOptions | None = None,
) -> FrameworkDynamicsRenderResult:
    ...
```

Preparation and rendering are intentionally separate so one scene can be inspected, serialized in future formats, or rendered with different visual options without recomputing the scientific geometry.

7 Input contracts

7.1 Frame collection

`collection` must be an `AtomisticFrameCollection` with fixed atom ordering and atomic numbers. The selected frame positions are a nonempty, unique, strictly increasing sequence. If omitted, all frames are selected.

Mean geometry and density preparation accept trajectory and independent-ensemble semantics. Continuous or folded trajectory overlays require

```
collection.require_trajectory("atomic trajectory visualization")
```

and reject an independent ensemble.

7.2 Framework topology

One authoritative `FrameworkTopology` must describe every selected frame. For every frame, the adapter verifies:

- vertex and retained-path atom indices exist;
- atomic numbers and role mappings agree;
- PBC masks agree;
- cells are finite and nonsingular;
- projected node and edge identity is stable;
- normalized periodic winding is stable;
- stored atomic paths remain compatible with frame-local minimum-image geometry.

A mismatch raises `GraphAdapterError`. Reactive trajectories must be partitioned into topology-compatible segments before plotting.

7.3 Atomic connectivity

An atomic mean graph requires an authoritative `AtomicConnectivityState` or `AtomicConnectivityResult`. Selected frames must be covered by the connectivity object, with stable active atom scope, atomic numbers, and PBC mask.

7.4 Full periodicity and registration restrictions for densities

The current dense convolution backend requires

```
np.all(collection.pbc)
```

for atomic and framework density fields. Mixed or nonperiodic axes are rejected. Framework and trajectory graph rendering may support broader periodic masks.

Periodic density also requires a valid periodic identification in the display cell. For material and framework-registered material coordinates, the selected samples are naturally periodic in the display cell.

Laboratory trajectories and mean geometry may use instantaneous cells,

$$\mathbf{x}_t = \mathbf{f}_t H_t,$$

but a periodic laboratory density in one display cell is scientifically valid only when every selected cell is equivalent to the display cell within the configured cell-comparison tolerance. Otherwise, translations by H_t are not translations by H_d, and folding laboratory positions modulo H_d imposes a different periodic lattice.

Therefore:

- variable-cell laboratory graph and trajectory overlays remain supported;
- periodic laboratory density must reject materially different selected cells;
- metadata records the maximum cell mismatch and the tolerance used;
- a future nonperiodic Cartesian-box laboratory-density backend is deferred.

8 Mathematical conventions

8.1 Row-vector cell convention

The cell matrix has lattice vectors as rows:

$$H = \begin{bmatrix} \mathbf{a}_1 \\ \mathbf{a}_2 \\ \mathbf{a}_3 \end{bmatrix}.$$

A fractional row vector $\mathbf{f}$ maps to Cartesian coordinates as

$$\mathbf{x} = \mathbf{f}H.$$

8.2 Periodic Euclidean metric

For two fractional coordinates, the minimum-image distance is

$$d_\mathrm{MIC}(\mathbf{f}_1, \mathbf{f}_2) = \min_{\mathbf{n}\in\mathbb{Z}^3} \left\|(\mathbf{f}_1 - \mathbf{f}_2 + \mathbf{n})H\right\|_2.$$

The architecture never substitutes independent fractional-component rounding for a proper triclinic Euclidean minimum-image search.

8.3 Frame weights

The current scene uses uniform weights

$$w_t = \frac{1}{N_f}, \qquad \sum_t w_t = 1.$$

Time quadrature and externally supplied ensemble weights are deferred. Every mean and density in one scene uses the same normalized frame weights.

9 Framework topology views

`graph_view_from_framework_topology(...)` produces one immutable `DecoratedGraphView` in either of two modes.

9.1 Projected framework mode

Projected framework atoms are graph nodes. Contracted linker paths are multigraph edges. Parallel paths and periodic self-image edges remain distinct. Every edge retains:

- authoritative projected-edge identity;
- endpoint atom identities;

- canonical image shift;
- retained atomic path and orientation metadata;
- linker composition and diagnostic attributes.

The graph is scientifically undirected. Stored path orientation is a canonical representation needed to reverse atom order and lattice translations together.

9.2 Atomic-path diagnostic mode

Every retained projected edge is expanded into its exact stored atomic segments. The adapter does not run a new path search. Diagnostic edge identity includes the parent projected edge and segment position, so two projected edges may retain separate identical atom-pair segments.

9.3 Frame-local winding validation

For a stored path

$$v_0 \to v_1 \to \cdots \to v_k,$$

frame-local segment shifts are summed:

$$\mathbf{M}^{\mathrm{path}}_{v_0 v_k} = \sum_{\ell=0}^{k-1} \mu_{v_\ell v_{\ell+1}}.$$

A deterministic spanning forest reconstructs the frame gauge and verifies that the canonical graph shift is compatible with the frame-local path sum. This guards against drawing an authoritative topology on an unrelated frame.

10 Periodic gauge normalization

For frame t, let the framework adapter provide wrapped fractional node coordinate $\mathbf{f}^{\mathrm{w}}_v(t)$ and oriented edge shift $\mathbf{n}_e(t) \in \mathbb{Z}^3$ for $e = (u, v)$.

A deterministic spanning forest assigns integer node gauges:

$$\mathbf{g}_v(t) = \mathbf{g}_u(t) + \mathbf{n}_e(t)$$

on tree edges. The lifted coordinate is

$$\mathbf{f}^{\mathrm{L}}_v(t) = \mathbf{f}^{\mathrm{w}}_v(t) + \mathbf{g}_v(t) + \mathbf{q}_{C(v)}(t),$$

where $\mathbf{q}_C(t)$ places each connected component. In a trajectory, the component anchor agrees with the reader-supplied unwrapped coordinate of the lowest-index component node. In an ensemble, components are independently placed in a canonical reference gauge.

Residual edge voltage is

$$\tilde{\mathbf{n}}_e = \mathbf{n}_e(t) + \mathbf{g}_u(t) - \mathbf{g}_v(t).$$

It must be identical in every selected frame. The gauge changes displayed images but not the scientific periodic graph.

11 Display cell

`FrameworkDynamicsOptions.display_cell` selects:

`"reference"` the cell of `reference_frame`;
`"mean"` the arithmetic mean of selected finite cell matrices, $H_{\mathrm{d}} = N_f^{-1} \sum_t H_t$.

The display cell must be finite and nonsingular. The mean cell is a visualization reference, not a claimed thermodynamic mean strain.

12 Registration modes

12.1 Material coordinates

Lifted fractional coordinates are mapped into one display cell:

$$\mathbf{x}_v^{\mathrm{mat}}(t) = \mathbf{f}_v^{\mathrm{L}}(t) H_{\mathrm{d}}.$$

Homogeneous expansion, contraction, and shear are removed from internal motion. This is the default and is generally preferred for site localization and hopping.

12.2 Laboratory coordinates

Instantaneous cells are preserved:

$$\mathbf{x}_v^{\mathrm{lab}}(t) = \mathbf{f}_v^{\mathrm{L}}(t) H_t.$$

These coordinates are valid for trajectories, mean geometry, and other nonperiodic Cartesian overlays.

A periodic density may convert laboratory positions to display fractional coordinates,

$$\mathbf{u}_v^{\mathrm{lab}}(t) = \mathbf{x}_v^{\mathrm{lab}}(t) H_{\mathrm{d}}^{-1},$$

only after verifying that each selected H_t is equivalent to H_{d} within numerical tolerance. If not, periodic laboratory density preparation raises an input-compatibility error rather than silently folding onto a different lattice.

12.3 Framework-registered coordinates

The lifted framework centroid is

$$\mathbf{c}(t) = \frac{1}{N_V} \sum_v \mathbf{f}_v^{\mathrm{L}}(t).$$

Relative translational drift is

$$\Delta\mathbf{c}(t) = \mathbf{c}(t) - \mathbf{c}(t_0).$$

Framework-registered coordinates subtract this same drift from every framework and atomic coordinate. Only translation is removed. Rotational and affine best-fit alignment are not implemented.

13 Geometrical mean framework

After registration, mean framework positions are

$$\overline{\mathbf{x}}_v = \sum_t w_t \mathbf{x}_v(t).$$

The result is one immutable `DecoratedGraphView` with:

- stable node and edge keys;
- reference scientific attributes;
- mean registered node positions;
- normalized residual edge shifts;
- the chosen display cell;
- source frame, topology, registration, and weighting metadata.

The mean view is a visualization object. It does not replace the authoritative `FrameworkTopology` or become input to scientific topology analysis.

14 Atomic trajectories

14.1 Selection

`TrajectoryAtomSelection` resolves the sorted union of explicit atom indices and species selectors. The result must be nonempty and within the collection.

14.2 Continuous paths

Reader-owned trajectory fractional coordinates are assumed already time-unwrapped. Material or framework-registered paths use the display cell. Laboratory paths use the instantaneous cell. Consecutive saved frames are connected by straight visual segments; the line is not a reconstruction of unresolved motion.

14.3 Folded paths

For periodic axis α,

$$m_{i\alpha}(t) = \lfloor f_{i\alpha}(t) \rfloor, \qquad f^{\mathrm{fold}}_{i\alpha}(t) = f_{i\alpha}(t) - m_{i\alpha}(t).$$

A displayed line segment is broken if the lattice image changes on any periodic axis. The renderer inserts `None` separators and never draws a false cell-spanning diagonal.

14.4 Prepared path record

`TrajectoryPathSet` stores:

```
atom indices and atomic numbers
selected collection frame positions and frame IDs
physical times when available
continuous positions
displayed positions
lattice image vectors
segment-break flags
display mode and selection label
```

Arrays are defensive, C-contiguous, finite, and read-only.

15 Atomic mean connectivity overlay

The mean atomic graph is optional and is prepared only when atomic connectivity is provided.

15.1 Periodic mean positions

Mean atomic nodes use the same registered atomic coordinates as density samples, not connectivity-tree lifts. For atom i, the displayed mean is the periodic Fréchet mean

$$\bar{\mathbf{x}}_i = \arg\min_{\mathbf{x}\in\mathbb{R}^3/\Lambda} \sum_t w_t d_{\mathrm{MIC}} \left(\mathbf{x}, \tilde{\mathbf{f}}_i(t) H_{\mathrm{d}}\right)^2 .$$

The implemented iteration averages Cartesian minimum-image displacement vectors. Integer shifts of any input frame do not move the mean.

15.2 Edge occupancy

For unordered atomic pair e,

$$p_e = \sum_t w_t \mathbf{1}_{e\in E_t}.$$

Modes are:

- `persistent`: retain only $p_e = 1$;
- `occupancy`: retain $p_e \geq p_{\min}$.

Displayed image shifts are recomputed from corrected mean positions using the Euclidean minimum-image convention. Connectivity gauge changes therefore cannot create runaway bonds.

15.3 Rendering

One node trace is emitted per species using ASE Jmol-style colors. Retained bonds are one line trace with separators. Species, bonds, trajectories, and every density field remain independently legend-toggleable.

16 Common density source model

Every density field begins with registered weighted samples in one display cell. All channels reduce to one transient sample-batch record:

```
@dataclass(frozen=True)
class PeriodicWeightedSamples3D:
    fractional_positions: NDArray[np.float64]       # (n_samples, 3), folded to [0, 1)
    weights: NDArray[np.float64]                    # (n_samples,), finite and nonnegative
    sample_group_ids: NDArray[np.int64] | None      # transient parent mapping
    source_provenance: DensitySourceProvenance
    total_measure: float
    measure_kind: str                               # occupancy | arc_length
    measure_units: str                              # count | angstrom
    metadata: Mapping[str, Any]
```

The sample arrays are C-contiguous, read-only, and shape-validated. `weights.sum()` must equal `total_measure` within the normalization tolerance before deposition. `sample_group_ids`, when present, maps transient quadrature or replicated samples to a parent source group. It is not copied into every final field. Persistent scientific identity is stored once in `source_provenance`.

The current implementation directly constructs dense deposition arrays, but its scientific sample semantics already follow this model.

17 Atomic occupancy density

For selected atom set S,

$$\rho_A(\mathbf{x}) = \sum_t w_t \sum_{i \in S} \delta[\mathbf{x} - \tilde{\mathbf{u}}_i(t) H_{\mathrm{d}}] \,.$$

The field integrates to

$$\int_\Omega \rho_A(\mathbf{x})\, d^3x = |S|.$$

An individual-atom field integrates to one; a species/group field integrates to the number of selected atoms. Units are Å^{-3}. It is an occupancy density, not mass or charge density.

18 Framework vertex occupancy density

For projected framework vertices V,

$$\rho_V(\mathbf{x}) = \sum_t w_t \sum_{v \in V} \delta[\mathbf{x} - \tilde{\mathbf{u}}_v(t) H_{\mathrm{d}}] \,.$$

It integrates to $|V|$ and has units Å^{-3}. It describes the spatial distribution of projected framework vertices, not all retained linker atoms unless the topology itself identifies them as projected vertices.

19 Framework edge-length density

For edge curve $\gamma_{e,t}(s)$ parametrized by arc length,

$$\rho_E(\mathbf{x}) = \sum_t w_t \sum_e \int_0^{L_e(t)} \delta\big[\mathbf{x} - \gamma_{e,t}(s)\big]\, ds.$$

Its total measure is

$$\int_\Omega \rho_E(\mathbf{x})\, d^3x = \sum_t w_t \sum_e L_e(t),$$

and its units are Å^{-2}.

19.1 Edge source policies

`edge_source="projected"` use straight frame-local projected framework segments;
`edge_source="atomic_paths"` use the exact retained atom-resolved path segments.

Both preserve authoritative edge/path identity. The first represents the coarse net; the second represents the dynamic support of retained linker atoms.

19.2 Edge quadrature

A physical segment of length L is divided into

$$n = \max\left(1, \left\lceil \frac{L}{h_E} \right\rceil\right)$$

midpoint samples. Each receives weight

$$\Delta\ell = \frac{w_t L}{n}.$$

Quadrature weights therefore sum exactly to the weighted segment length before deposition.

Let the real-space sampling-basis vectors be the rows of

$$D = \mathrm{diag}(N_1^{-1}, N_2^{-1}, N_3^{-1}) H_{\mathrm{d}},$$

and define the conservative policy scale

$$h_{\mathrm{axis,min}} = \min_a \|\mathbf{b}_a\|_2.$$

This is an edge-sampling policy scale, not a real-space covering-radius certificate. Define the unrefined policy interval

$$h_E^{(0)} = \begin{cases} \min(h_{\mathrm{axis,min}}, \sigma/2), & \sigma > 0, \\ h_{\mathrm{axis,min}}, & \sigma = 0. \end{cases}$$

The implemented architecture distinguishes two modes:

`auto` choose one deterministic, transactionally predictable interval after density resolution is known,

$$h_E^{\mathrm{realized}} = \min\left(h_E^{\mathrm{nominal}}, \frac{h_E^{(0)}}{2^r} \right),$$

where r is `edge_quadrature_refinement_levels` in $[0, 8]$. The default is $r = 2$. The fixed refinement depth permits exact sample, CIC, kernel-pair, block, and memory planning before scalar-field allocation. Setting $r = 0$ exposes the unrefined policy.

`explicit` preserve the user-supplied interval exactly and report an under-resolution warning when it exceeds $h_E^{(0)}$.

The default framework-edge policy is `auto`. The nominal interval, base policy, refinement depth, realized interval, segment/sample counts, and any under-resolution diagnostic are recorded. The default $r = 2$ policy is certified by focused comparisons against half the realized interval:

```
relative L1 <= 2e-3
relative L-infinity <= 1e-2
HDR threshold relative difference <= 1e-3
```

on projected and atom-resolved paths, orthogonal and skewed cells, and periodic boundary crossings. This is a validated project policy rather than a universal a posteriori error estimator. Runtime trial fields are not constructed before global scene approval. Resource failure never silently increases h_E or omits edges.

If framework-edge resolution is inherited from framework-vertex spread, metadata records `resolution_reference_source="fram` it is not presented as an edge-derived spread statistic.

20 Density operator architecture

20.1 Logical node convention

A logical grid shape (N_1, N_2, N_3) denotes exactly $N_1 N_2 N_3$ periodic **nodes**. Node (i, j, k) is located at

$$\mathbf{f}_{ijk} = \left(\frac{i}{N_1}, \frac{j}{N_2}, \frac{k}{N_3} \right), \qquad \mathbf{x}_{ijk} = \mathbf{f}_{ijk} H_{\mathrm{d}}.$$

The same convention governs CIC deposition, convolution, HDR thresholding, voxel clouds, and marching-cubes cells. A node field must never be rendered at $(i + 1/2)/N_i$ as though it were cell-centered.

When `grid_shape=None`, the legacy user-facing interval rule is

$$N_i = \max\left(4, \left\lceil \frac{\|\mathbf{a}_i\|_2}{h} \right\rceil\right), \qquad h = 0.20 \text{ Å}.$$

An explicit `grid_shape` overrides automatic sizing.

20.2 Periodic cloud-in-cell deposition

For folded fractional sample $\mathbf{u}$, scaled coordinate is

$$\mathbf{y} = (N_1 u_1, N_2 u_2, N_3 u_3).$$

With $y_a = b_a + d_a$, the source weight is distributed to the eight periodic nodes. Corner weight is

$$W_\epsilon = \prod_{a=1}^{3} \begin{cases} 1 - d_a, & \epsilon_a = 0, \\ d_a, & \epsilon_a = 1. \end{cases}$$

The weights sum to one. Atomic occupancy, framework vertices, and edge quadrature all use this assignment with different source weights.

20.3 Current baseline operator: `legacy_spectral_v1`

`mdstats 0.19.39a0` convolves deposited node masses with the finite-mode spectral multiplier

$$\widehat{G}_{\text{legacy}}(\mathbf{m}) = \exp\left[-\frac{1}{2}\sigma^2 \left\|2\pi \mathbf{m} H_{\text{d}}^{-T}\right\|_2^2\right].$$

The inverse FFT produces a periodic node-mass array. The implementation then sets **all** values below

$$10^{-13} \max(1, \max|m_g|)$$

to zero and renormalizes. This clips small positive values as well as negative roundoff; it is not merely a negative-value correction. Metadata identifies this behavior as `legacy_spectral_v1`.

An explicit `gaussian_bandwidth=0` returns the CIC node masses unchanged.

20.4 Normative target operator: `discrete_periodized_v1`

Dense and sparse backends share one normalized, finite-support, discrete periodized Gaussian stencil.

Let

$$D = \text{diag}(N_1^{-1}, N_2^{-1}, N_3^{-1}) H_{\text{d}}, \qquad \Delta V = \frac{|\det H_{\text{d}}|}{N_1 N_2 N_3}.$$

For a logical offset δ and periodic image $\mathbf{n}$, define

$$\Delta\mathbf{x}_{\delta,\mathbf{n}} = \left(\frac{\delta}{\mathbf{N}} + \mathbf{n}\right) H_{\text{d}}.$$

Before normalization, the dimensionless stencil contribution is

$$a_{\delta,\mathbf{n}} = \Delta V K_\sigma(\Delta\mathbf{x}_{\delta,\mathbf{n}}) \mathbf{1}\left[\|\Delta\mathbf{x}_{\delta,\mathbf{n}}\|_2 \le r_{\text{cut}}\right],$$

where

$$K_\sigma(\mathbf{r}) = \frac{1}{(2\pi\sigma^2)^{3/2}} \exp\left(-\frac{\|\mathbf{r}\|_2^2}{2\sigma^2}\right).$$

Canonical logical offsets receive the sum of all retained periodic-image contributions,

$$a_\delta = \sum_{\mathbf{n}} a_{\delta,\mathbf{n}}, \qquad S_a = \sum_\delta a_\delta,$$

and the stored stencil is

$$g_\delta = \frac{a_\delta}{S_a}, \qquad \sum_\delta g_\delta = 1.$$

For deposited node masses m_j, the density is

$$\rho_g = \frac{1}{\Delta V} \sum_j m_j g_{g-j}.$$

The dense backend performs circular convolution by FFT of this same discrete stencil array. The sparse backend scatters these same weights from occupied CIC nodes. Their outputs therefore use one declared operator and are compared under the normative floating-point policy.

For $\sigma = 0$, the stencil is the identity:

$$g_{\mathbf{0}} = 1, \qquad g_{\delta \neq \mathbf{0}} = 0.$$

For $\sigma > 0$, `kernel_tail_tolerance` is restricted to

$$10^{-15} \leq \varepsilon \leq 10^{-3},$$

and selects r_{cut} through the three-dimensional Gaussian tail bound. The following metadata are recorded:

```
continuous_tail_mass_bound
stencil_pre_normalization_sum = S_a
stencil_normalization_factor = 1 / S_a
stencil_offset_count
periodic_image_contribution_count
```

`1-S_a` is not called omitted discrete mass: it mixes continuous-tail truncation with grid quadrature error and is not a certified difference from an infinite discrete reference sum.

The unaggregated contributions also define the stencil covariance

$$C_g = \frac{1}{S_a} \sum_{\delta,\mathbf{n}} a_{\delta,\mathbf{n}} \Delta\mathbf{x}_{\delta,\mathbf{n}} \Delta\mathbf{x}_{\delta,\mathbf{n}}^T.$$

This covariance is used by the target artificial-broadening diagnostic.

20.5 Operator migration policy

`legacy_spectral_v1` remains available for reproducibility. The canonical operator may become the default only through a versioned migration that:

1. implements both operators in the dense backend;
2. records the operator identifier in every field;
3. quantifies differences on default and under-resolved inputs;
4. preserves an explicit legacy compatibility option;

5. updates serialized-schema migration notes and release documentation.

Sparse fields are not described as equivalent to legacy spectral fields. They are exactly equivalent to dense fields using `discrete_periodized_v1`.

Direct Gaussian KDE over raw samples remains outside this architecture.

21 Gaussian and spread-aware resolution

21.1 Legacy lattice-axis interval

For automatic shape selection,

$$h_i = \frac{\|\mathbf{a}_i\|_2}{N_i}, \qquad h_{\max} = \max_i h_i.$$

When bandwidth is not explicit, the current coupling is

$$\sigma = r_g h_{\max}, \qquad r_g = 2.$$

This remains a compatibility-facing definition.

21.2 Metric-aware resolution diagnostic

The axis-edge measure is not a complete resolution certificate for a skewed lattice. Define the real-space sample-lattice basis

$$D = \operatorname{diag}(N_1^{-1}, N_2^{-1}, N_3^{-1}) H_{\mathrm{d}}.$$

Let the shortest nonzero reciprocal sampling-lattice vector be

$$k_{\min} = \min_{\mathbf{m} \in \mathbb{Z}^3 \setminus \{0\}} \left\| 2\pi \mathbf{m} D^{-T} \right\|_2 .$$

Define the reciprocal-resolution diagnostic

$$h_{\text{reciprocal}} = \frac{2\pi}{k_{\min}}.$$

It reduces to the largest Cartesian grid spacing for an orthogonal anisotropic grid and exposes poor reciprocal-plane/Nyquist resolution caused by a skew basis. It is not a real-space nearest-node distance or covering radius and must not be used as one. Every field records $h_{\max}$, $h_{\text{reciprocal}}$, and $\sigma / h_{\text{reciprocal}}$.

The implementation uses a deterministic certified bounded enumeration for the shortest reciprocal vector. This diagnostic does not change the legacy automatic shape rule. A true real-space covering-radius diagnostic is deferred until a use case requires it.

21.3 Periodic mean and spread diagnostics

For item i, let $\bar{\mathbf{x}}_i$ be a periodic Fréchet/Karcher mean. Minimum-image Cartesian displacements define

$$s_i = \sqrt{\frac{1}{3} \sum_t w_t \|\Delta \mathbf{x}_i(t)\|_2^2} = \sqrt{\frac{\operatorname{tr} C_i}{3}}.$$

The field reference spread is

$$s_q = Q_q(\{s_i\}),$$

with default $q = 0.10$ and `numpy.quantile(method="linear")`.

The deterministic mean solver uses the following initial candidates in this order:

1. component-wise circular mean mapped to the cell;
2. weighted sample medoid;
3. first sample in stable source/frame order;
4. sample farthest from the medoid, with stable tie breaking.

Duplicate starts are removed in stable order. Default numerical policies are:

```
max_iterations = 128
update_tolerance = 1e-12 * L_ref
objective_relative_tolerance = 1e-12
mean_separation_tolerance = 1e-8 * L_ref
minimum_valid_reference_fraction = 0.50
minimum_valid_reference_count = 1
```

Each periodic mean returns:

```
mean_converged
iteration_count
final_update_norm
objective_value
mean_ambiguity_detected
candidate_solution_count
```

Starts that converge to means separated beyond `mean_separation_tolerance` but whose objectives agree within `objective_relative_tolerance` mark the mean ambiguous. A nonconverged or ambiguous item is excluded from the automatic-resolution reference. Automatic refinement requires at least the larger of the declared minimum count and minimum fraction of selected items. Otherwise nominal or explicit resolution is retained with a warning and complete diagnostics.

21.4 Adaptive target

The current baseline policy is versioned as `broadening_metric="gaussian_sigma_v1"` and triggers when

$$\sigma_{\text{nominal}} > \alpha s_q, \qquad \alpha = 0.5.$$

It is retained unchanged through LD0-R1, LD0-R2, and LD0-R3 for numerical compatibility.

The normative target policy is `broadening_metric="effective_cic_stencil_rms_v1"`. For sample s, with CIC phase d_{sa} and real-space sampling-basis rows $\mathbf{b}_a$, the CIC assignment covariance is

$$C_{\text{CIC},s} = \sum_{a=1}^{3} d_{sa}(1 - d_{sa})\mathbf{b}_a\mathbf{b}_a^T.$$

With normalized positive sample weights ω_s, define

$$\overline{C}_{\text{CIC}} = \sum_s \omega_s C_{\text{CIC},s},$$

and combine it with the canonical-stencil covariance:

$$s_{\text{art}} = \sqrt{\frac{\text{tr}(\overline{C}_{\text{CIC}} + C_g)}{3}}.$$

The target condition is

$$s_{\mathrm{art}} \leq \alpha s_q.$$

Because C_g belongs to the canonical operator, this target becomes operational only at gate LD0-B after LD0-K. It is a versioned scientific migration; it is not silently substituted for the legacy sigma-only criterion.

Scientific resolution and storage planning are separate. A fine logical grid may be valid even when a dense array does not fit.

For $s_q = 0$, no positive finite artificial-width target exists in general. The standard requires:

```
adaptive_target_defined = False
adaptive_refinement_applied = False
resolution falls back to nominal or explicit input
an explanatory warning and metadata record are emitted
```

The implementation must not attempt unbounded refinement. For $\sigma = 0$, CIC broadening remains present and is included in s_{art}.

22 Field identity, storage, and provenance

22.1 Current dense scalar field

`PeriodicScalarField3D` currently stores a dense `values` array plus scientific metadata. Its `integral` member is a property and remains so.

22.2 Normative source provenance

Integer atom indices alone cannot identify every density source. Persistent provenance is:

```python
@dataclass(frozen=True)
class DensitySourceProvenance:
    schema_version: str
    source_kind: str
    atom_indices: tuple[int, ...] = ()
    vertex_keys: tuple[CanonicalSourceKey, ...] = ()
    edge_keys: tuple[CanonicalSourceKey, ...] = ()
    metadata: FrozenJSONMapping = field(default_factory=FrozenJSONMapping)
```

`CanonicalSourceKey` is a tagged, recursively JSON-compatible tuple representation; arbitrary Python `Hashable` objects are not serialized directly. Framework multiedges and atom-resolved paths retain their full canonical keys. Transient sample parent mappings remain in `PeriodicWeightedSamples3D.sample_group_ids` and are not stored redundantly in every field.

All arrays held by frozen records are defensive C-contiguous copies with `writeable=False`. Metadata are recursively frozen and JSON-compatible. Provenance, field, plan, and storage records each carry explicit schema versions.

22.3 Normative storage summary

Every field provides a backend-neutral summary:

```python
@dataclass(frozen=True)
class DensityStorageSummary:
    schema_version: str
    storage_backend: str
    logical_grid_shape: tuple[int, int, int]
    logical_node_count: int
    nonzero_node_count: int
    stored_value_count: int
    stored_block_count: int
```

```
    estimated_bytes: int
    realized_bytes: int | None
    metadata: FrozenJSONMapping
```

`stored_value_count` means allocated scalar slots, including padding in partial blocks. It is distinct from nonzero nodes. Counts and byte estimates are nonnegative, checked for integer overflow, and serialized canonically.

22.4 Normative scalar-field protocol

The scientific identity protocol is intentionally small:

```
class ScalarField3D(Protocol):
    schema_version: str
    field_key: str
    label: str
    physical_units: str
    display_cell: NDArray[np.float64]
    total_measure: float
    gaussian_bandwidth: float
    smoothing_operator: str
    broadening_metric: str
    storage_backend: str
    source_provenance: DensitySourceProvenance
    metadata: FrozenJSONMapping

    @property
    def grid_shape(self) -> tuple[int, int, int]: ...

    @property
    def voxel_volume(self) -> float: ...

    @property
    def integral(self) -> float: ...

    def threshold_for_mass_fraction(self, q: float) -> float: ...
    def storage_summary(self) -> DensityStorageSummary: ...
```

Backend-neutral numerical access is a separate public capability:

```
class PeriodicNodeFieldAccess(Protocol):
    def iter_stored_nodes(
        self,
        *,
        batch_size: int | None = None,
    ) -> Iterator[tuple[NDArray[np.int64], NDArray[np.float64]]]: ...

    def gather_node_values(
        self,
        logical_indices: NDArray[np.int64],
    ) -> NDArray[np.float64]: ...
```

`iter_stored_nodes()` yields lexicographically ordered, read-only batches of shape `(n, 3)` and `(n,)`. Dense fields yield every logical node; sparse fields yield stored nodes, including stored zeros only when required by their storage contract. `gather_node_values()` accepts any integer indices, applies periodic modulo indexing, and returns zero for absent sparse nodes. Inputs and outputs use `int64` and `float64` respectively.

Rendering and resource operations use only these public capabilities:

```
prepare_density_voxel_cloud(field, node_access, ...)
prepare_density_mesh(field, node_access, ...)
estimate_density_render_resources(field, node_access, ...)
```

`field_cartesian_bounds(field, node_access, ...)`

They must not inspect private dense arrays, block dictionaries, or implementation classes. Concrete-type dispatch is permitted only through registered public adapters, never through private attributes.

23 Normative numerical policy

All scientific fields use `float64`; logical indices use `int64`. Define the scene reference length

$$L_{\mathrm{ref}} = \max\left(1\ \text{Å}, \max_i \|\mathbf{a}_i\|_2\right).$$

Let

$$\|x\|_{1,r} = \max(1, \|x\|_1), \qquad \|x\|_{\infty,r} = \max(1, \|x\|_\infty).$$

Unless a gate states a tighter criterion, array equivalence requires

```
relative L1 error <= 2e-11
relative L-infinity error <= 5e-11
absolute integral error <= 5e-13 * max(1, total_measure)
```

where relative errors divide by the corresponding reference norm above. Stable index orders, provenance, option records, and metadata must be byte-identical for identical inputs on one supported platform. Cross-platform floating arrays and meshes use the stated numerical/geometric tolerances; exact triangle ordering is required only for identical library versions and platform.

Cell equivalence for periodic laboratory density is

$$\|H_t - H_{\mathrm{d}}\|_F \le 10^{-10}\ \text{Å} + 10^{-10}\|H_{\mathrm{d}}\|_F.$$

A field-specific override must be explicit and serialized.

24 Highest-density probability shells

For target fraction q, the scientific discrete HDR threshold is the largest superlevel boundary reached by descending node values such that

$$\Delta V \sum_{\rho_g \ge c_q} \rho_g \ge qM.$$

All nodes equal to the selected threshold are included. Therefore the achieved fraction may exceed q when ties occur. The future detailed API returns

```
HDRThresholdResult(
    requested_mass_fraction=...,
    threshold=...,
    achieved_mass_fraction=...,
    tied_node_count=...,
)
```

The compatibility method `threshold_for_mass_fraction(q)` continues to return the threshold alone.

Inactive sparse nodes are exactly zero and do not affect positive HDR thresholds. The field must retain the scientific threshold independently of any later rendering fallback.

25 Density rendering preparation

25.1 Periodic dense mesh convention

For a node field of shape (N_1, N_2, N_3), the first node plane is copied to a terminal plane along every periodic axis, producing

$$(N_1 + 1) \times (N_2 + 1) \times (N_3 + 1).$$

Coordinates are

$$\mathbf{x}_{abc} = \left(\frac{a}{N_1}, \frac{b}{N_2}, \frac{c}{N_3} \right) H_{\mathrm{d}}.$$

Lewiner marching cubes extracts explicit vertices and faces before Plotly serialization.

25.2 Scientific threshold versus render level

The renderer may convert a temporary volume to `float32`, and a degenerate level may require a numerically interior fallback. Therefore each shell records:

```
scientific_hdr_threshold
scientific_achieved_mass_fraction
actual_render_level
render_level_adjusted
render_adjustment_reason
```

A rendered mesh must not be claimed to enclose exactly the requested discrete mass when its actual render level differs from the scientific threshold.

25.3 Node-cloud convention and baseline correction

A diagnostic voxel cloud represents node values at i/N_i, not voxel centers at $(i + 1/2)/N_i$. The `0.19.39a0` dense fallback uses the latter and is displaced by half a grid step. LD0-R2 corrects this rendering bug. The correction is explicitly exempt from byte-for-byte preservation because it restores consistency with CIC and mesh coordinates.

Raw folded samples may be rendered separately when stored.

26 Renderer composition

The composite renderer performs operations in this order:

1. render the mean framework through the generic 3-D graph renderer;
2. append the optional mean atomic graph;
3. append trajectory traces and endpoint markers; when trajectory legends are enabled, the start-circle and end-diamond marker classes are separately labeled legend entries by default and use independent toggle groups;
4. append atomic density meshes/clouds;
5. append framework vertex and edge density meshes/clouds;
6. combine graph, path, density, and cell bounds;
7. enforce equal Cartesian display scale;
8. set grouped legend behavior;
9. return trace provenance and export methods.

The generic renderer remains unaware of trajectories and density semantics.

27 Euclidean metric in the 3-D scene

The final Cartesian range lengths are R_x, R_y, R_z. Manual scene aspect lengths A_x, A_y, A_z satisfy

$$\frac{A_x}{R_x} = \frac{A_y}{R_y} = \frac{A_z}{R_z}.$$

Thus one angstrom is displayed at the same scale along every Cartesian axis. This is distinct from density-grid resolution: an under-resolved scalar field may still produce a distorted extracted shell even when scene aspect is correct.

28 Periodic graph display and cell wireframes

The mean graph is passed through `PeriodicDisplayOptions`, supporting canonical, local-unwrapped, or expanded-cell materialization according to the generic periodic graph contract. Scientific source keys remain distinct from display-image keys.

Density fields remain scientifically canonical. Rendering uses an explicit `density_display_mode`:

`canonical` render the canonical-cell field once; this is the default for every graph mode;
`match_graph` replicate already prepared canonical meshes or node clouds by the same integer image shifts used for graph expansion. No density is recomputed and no replicated trace becomes a new scientific field.

`match_graph` is valid only for expanded-cell graph materialization. Replicated points, vertices, faces, traces, and estimated HTML bytes count against render budgets before any replication is performed.

The 3-D renderer may draw a reference cell, all primary cells in an expanded display, and an outer boundary. Cell wireframes and ghost images are rendering geometry. They do not change source graph or density identity.

29 Styling and legend ownership

The default framework style is chemistry-aware and distinguishes projected and atomic-path diagnostic views. Framework nodes may be displayed as markers, compact dots, or hidden while scientific node identity remains present.

Composite legend groups are owned by scientific channels:

```
mean framework node/edge groups
atomic species groups
atomic bonds
one trajectory group per selected atom
trajectory start and end markers as independent default legend entries
one group per atomic density field
framework vertex density
framework edge-length density
```

The layout uses

```
legend={"groupclick": "togglegroup"}
```

so nested shells from one field toggle together without coupling unrelated channels.

30 Prepared scene model

The current `FrameworkDynamicsScene` stores:

```
mean_framework: DecoratedGraphView
trajectory_paths: TrajectoryPathSet | None
atomic_mean_graph: AtomicMeanGraph | None
frame_indices: ndarray[int64]
weights: ndarray[float64]
```

```
display_cell: ndarray[float64]
options: FrameworkDynamicsOptions
resources: FrameworkDynamicsResources
atomic_density_fields: tuple[PeriodicScalarField3D, ...]
framework_density_fields: FrameworkDensityFields | None
metadata: Mapping[str, Any]
```

Future backend neutrality changes the density annotations to a common field protocol, not the scene's scientific ownership. The normative target annotations are:

```
atomic_density_fields: tuple[ScalarField3D, ...]
framework_density_fields: FrameworkDensityFields[ScalarField3D] | None
planning_record: DensityScenePlan | None
```

31 Render result model

`FrameworkDynamicsRenderResult` owns:

```
Plotly figure
source FrameworkDynamicsScene
base graph-render result
trajectory trace indices by atom
atomic mean graph trace indices
endpoint trace indices
atomic density trace indices by field key
framework density trace indices by field key
trace-indexed density cloud provenance
render metadata
HTML serialization and file-writing methods
```

Trace indices are provenance for programmatic visibility control and testing. They are not scientific identifiers.

32 Resource and transactional planning architecture

32.1 Shared option records

Numerical and storage concerns are separated:

```
DensityResolutionOptions(...)
DensityKernelOptions(...)
DensityStorageOptions(...)
DensityRenderOptions(...)
```

`DensityResolutionOptions` owns grid shape/interval, Gaussian ratio or explicit bandwidth, adaptive policy, quantile, and broadening metric. `DensityKernelOptions` owns smoothing-operator selection, `kernel_tail_tolerance`, and operator-specific controls. `DensityStorageOptions` owns dense/sparse/auto storage and block choices. `DensityRenderOptions` owns shell fractions, mesh/cloud choices, display replication, and rendering budgets.

Atomic and framework option records add source selection and edge/quadrature policies while reusing these common records.

32.2 Current baseline limits

`max_density_voxels` remains the dense compatibility limit until the common planner is operational. Existing trace and vertex limits remain active.

32.3 Three-phase global transaction

Scene preparation is transactional at three explicitly bounded levels.

32.3.1 Phase A - metadata-only conservative bounds

Without constructing large sample or index arrays, compute conservative bounds for:

- source sample count and bytes;
- logical shape and node count;
- stencil bounding-box values and image candidates;
- CIC target-node insertions;
- block slots and kernel pairs;
- render replication and worst-case mesh cells/faces;
- total scene peak bytes.

A Phase-A limit failure stops immediately.

32.3.2 Phase B - bounded exact index planning

After Phase-A approval, construct only budgeted integer/index structures needed to resolve exact counts:

- canonical source keys and sample-group counts;
- occupied CIC node indices;
- canonical stencil offsets and image-contribution indices;
- active block indices and valid masks;
- candidate logical cells and component index sets.

Phase-B allocations count against `max_density_planning_bytes`. No full floating field, component volume, or mesh is allocated. All channel plans are then reviewed together.

32.3.3 Phase C - global approval and scientific allocation

Only after every requested atomic and framework channel passes Phases A and B may floating-point fields and meshes be allocated. Allocation follows the approved plan. A later framework failure cannot occur after an unplanned atomic field has already consumed the scene budget.

Planner estimates and realized values are compared after construction. Exceeding an approved hard limit is an error; material underestimation is a test failure requiring planner revision.

32.3.4 LD12 hybrid-aware Phase-C accounting

For the production local-sparse path, Phase B must construct the same packed CIC source, exact finite-support atlas, and deterministic direct/FFT tile plan used by LD8-S3 realization. Let

$$C_i = N_{\mathrm{source},i} N_{\mathrm{stencil},i}$$

be the exact mathematical contribution count for field i, and let

$$D_i = \sum_{t \in \mathcal{D}_i} N_{\mathrm{source},t} N_{\mathrm{stencil},i}$$

be the actual direct-tile pair count. `max_density_kernel_pairs` applies to the scene sum of D_i for hybrid fields plus the all-direct counts of explicit LD7 fields. The nominal C_i remains an identity and diagnostic quantity; it must not be treated as direct execution work when FFT tiles are selected.

FFT tiles contribute their calibrated padded-transform cost directly to the scene wall-time estimate. Phase C therefore admits the selected mixed execution plan, not an imaginary all-direct implementation. An all-direct hybrid plan still has $D_i = C_i$ and receives no relaxation. An all-FFT plan has $D_i = 0$ but remains bounded by padded-node, peak-memory, and wall-time limits.

The Phase-B metadata must distinguish `exact_contribution_count`, `direct_pair_count`, `fft_padded_node_count`, and `hybrid_estimated_wall_seconds`. A plan/realization mismatch is a planner defect; it must never be repaired by coarsening the grid or increasing Gaussian smearing.

32.4 Backend-neutral accounting

Dense and sparse plans report logical nodes, nonzero nodes, valid block nodes, allocated block slots, stencil values, kernel pairs, component values, marching-cubes cells, vertices, faces, replicated render geometry, and total peak bytes separately.

The planner must never silently increase grid interval, Gaussian bandwidth, tail tolerance, edge quadrature spacing, or omit sources/shells to satisfy resources.

33 Failure semantics

`GraphAdapterError` invalid frames, cells, registration, selections, topology identity, graph gauge, periodic winding, field normalization, or incompatible scene inputs.

`TrajectoryRequiredError` trajectory overlay requested from an independent ensemble.

`GraphComplexityError` declared preparation or rendering resource limit exceeded.

`GraphStyleError` invalid numerical/render options such as grid shape, bandwidth, fractions, opacity, or marker size.

`GraphVisualizationError` optional Plotly/scikit-image import, serialization, browser-validation, or output writing failure.

`BrowserMeshBudgetFailure` structured `GraphComplexityError` payload identifying an unsatisfied raw-tile, transient-memory, final face, final vertex, trace-count, HTML-byte, fidelity, topology, seam, or browser-usability constraint. Interactive HTML is not written.

Failures do not reconstruct topology, remap atoms, interpolate missing paths, omit frames, change resolution, drop scientific channels, or emit geometry above a hard browser limit.

34 Metadata and provenance

Scene metadata records at least:

```
scene schema version
topology and projected-graph digests
collection semantics
reference frame
display mode
registration mode
display-cell policy
cell-equivalence tolerance and maximum mismatch
selected frame positions
trajectory atom indices
atomic mean graph presence
atomic density field keys
framework density field keys
global planning summary and approval result
```

Every density field records:

```
field schema and source provenance
physical units and normalization definition
registration mode and frame count
weighting policy
logical node convention
grid definition, shape, target interval, realized axis intervals
reciprocal-resolution diagnostic and reciprocal-vector diagnostic
Gaussian bandwidth and both Gaussian/grid ratios
smoothing operator identifier and operator parameters
continuous tail tolerance and actual stencil normalization diagnostics
broadening-metric identifier, artificial covariance, and adaptive-smearing status
periodic-mean convergence and ambiguity diagnostics
```

```
sample spread definition, quantile, reference, min/median/max
backend and backend-specific storage summary
Phase-A bounds, Phase-B exact counts, and Phase-C approval identifiers
raw and final normalization information
scientific HDR thresholds and achieved fractions
actual render levels and any adjustments
edge quadrature mode, nominal/realized spacing, and source reference
```

Dense and sparse schemas expose comparable scientific metadata while retaining backend-specific storage details.

35 Implementation plan authority

The remainder of this manual is the normative implementation plan. It replaces the standalone local-sparse roadmap. Gate names, acceptance criteria, and stop conditions are maintained here only.

36 Local sparse density architecture

36.1 Objective

The local sparse backend retains the global logical node lattice and CIC source assignment while allocating only local support:

registered weighted samples → sparse CIC node masses → canonical discrete periodized stencil → block-sparse node field.

The sparse backend supports only `discrete_periodized_v1`. Dense comparison uses the same operator. The default and normative Gaussian tail tolerance remains `kernel_tail_tolerance=1.0e-8`; no LD8 optimization may loosen it silently.

36.2 Weighted sample record

The sparse backend consumes the one common `PeriodicWeightedSamples3D` record defined under **Common density source model**. No second or backend-specific definition is permitted.

36.3 Sparse field model

```python
@dataclass(frozen=True)
class PeriodicBlockScalarField3D:
    schema_version: str
    field_key: str
    label: str
    physical_units: str
    logical_grid_shape: tuple[int, int, int]
    block_shape: tuple[int, int, int]
    active_block_indices: NDArray[np.int64]       # (n_blocks, 3), lexicographic
    block_values: NDArray[np.float64]             # (n_blocks, bx, by, bz)
    block_valid_masks: NDArray[np.bool_] | None   # same block axes when partial
    display_cell: NDArray[np.float64]              # (3, 3)
    total_measure: float
    gaussian_bandwidth: float
    smoothing_operator: str
    broadening_metric: str
    voxel_volume: float
    storage_backend: Literal["local_sparse"]
    source_provenance: DensitySourceProvenance
    metadata: FrozenJSONMapping
```

All arrays are C-contiguous, defensive, read-only, and dtype/shape validated. `active_block_indices` are unique modulo the block lattice. Partial terminal blocks use explicit valid masks. The class implements `ScalarField3D` and `PeriodicNodeFieldAccess`; absent logical nodes gather as zero.

36.4 Sparse CIC aggregation

The eight periodic contributions of every sample are aggregated into a sorted list of occupied logical nodes. Duplicate node contributions are combined deterministically. The deposited measure is checked before smoothing.

36.5 Canonical stencil construction

The stencil follows `discrete_periodized_v1`. Its support is chosen from `DensityKernelOptions.kernel_tail_tolerance`. Conservative reciprocal-plane bounds enumerate candidate integer offsets and periodic images; exact Cartesian distances filter them. Canonical offsets are sorted lexicographically.

The constructor records the continuous tail bound, pre-normalization sum, normalization factor, offset count, image-contribution count, and covariance. It does does not expose an omitted-mass field without a separately certified infinite discrete reference sum.

36.6 Active nodes and blocks

Each occupied CIC node scatters the canonical stencil to periodic target nodes. Targets are accumulated before block packing. Only blocks containing nonzero stored values are allocated. A complete neighboring block layer is not stored for meshing.

36.7 Normalization and HDR

The sparse field is normalized to its exact target measure after accumulation. The pre-normalization field integral, normalization factor, canonical-stencil diagnostics, and final integral are recorded. HDR thresholds use active values plus implicit zeros and retain tie diagnostics.

36.8 Sparse voxel rendering

Node clouds use exact logical node coordinates i/N_i. Selection is deterministic and resource-bounded. Sparse fields are never densified merely to render a diagnostic cloud.

36.9 Sparse mesh ownership

Sparse meshing is defined on logical **cells**, not merely connected blocks. A logical cell is identified by its lower periodic node index, and each candidate cell appears in exactly one threshold-local face-adjacency component.

For each shell level:

1. identify candidate logical cells whose eight-node range crosses the level;
2. construct periodic face-adjacent cell components;
3. lift each component with integer image coordinates and detect cycle winding;
4. assemble one component-local node array plus ghost values through public periodic lookup;
5. construct an exact component-cell mask;
6. contour the component once with a mask-aware Lewiner implementation, or use a cell-aware contouring wrapper that returns source-cell identity;
7. reject any implementation that relies on post hoc lower-cell triangle ownership from a black-box whole-volume call that does not expose source cells;
8. clip triangles against the canonical fractional cell when canonical display is requested;
9. canonicalize vertices within $10^{-10}L_{\mathrm{ref}}$ and remove duplicate faces by sorted vertex triplets.

A nonwinding lifted component is meshed once; ghost halos never create a second owned component. A component with nonzero torus winding cannot be represented as one compact chart. Until a tiled periodic mesher exists, it falls back to dense canonical meshing when feasible or to node-cloud rendering.

Canonical clipping can produce Euclidean cut boundaries. Validation therefore distinguishes:

- ordinary interior edges, which must have incidence two;
- canonical-boundary cut edges, which may have incidence one but must pair across opposite periodic faces within $10^{-10}L_{\mathrm{ref}}$;
- periodic closure on the torus, which is required;
- ordinary Euclidean watertightness, which is not required after canonical clipping.

Independent vertex wrapping is forbidden. Every mesh edge produced from one logical cell must satisfy

$$\ell_{\mathrm{edge}} \le (\|\mathbf{b}_1\| + \|\mathbf{b}_2\| + \|\mathbf{b}_3\|)(1 + 10^{-10}).$$

36.10 Determinism

For fixed inputs, the following are independent of hash order and scheduling:

- occupied-node ordering;
- stencil ordering;
- active-block ordering;
- accumulation order within the defined floating-point policy;
- component labels and lifted charts;
- component labels and canonical mesh index ordering are exact on one supported platform/library version; cross-platform mesh geometry follows the normative tolerance;
- metadata and serialized scientific records.

37 Normative implementation gates

37.1 LD0-R1 - Contracts, options, provenance, and dense adapters

Deliver the four shared option records, canonical schemas, unified weighted samples, structured provenance, `ScalarField3D`, `PeriodicNodeFieldAccess`, storage summaries, and zero-copy dense adapters. Reserve backend/operator/broadening identifiers and reject unimplemented combinations.

Acceptance:

1. `legacy_spectral_v1` dense values and integrals satisfy the normative numerical policy against `0.19.39a0`;
2. dense adapters do not copy the scientific values array;
3. all public arrays are read-only and all metadata are recursively frozen;
4. canonical JSON round trips preserve every schema field and source key;
5. no scientific preparation module imports Plotly.

37.2 LD0-R2 - Registration, means, resolution diagnostics, and rendering correction

Deliver cell-equivalence validation, deterministic periodic-mean diagnostics, quantile policy, zero-spread handling, $h_{\mathrm{reciprocal}}$, and the node-cloud coordinate correction.

Acceptance:

1. variable-cell laboratory periodic density fails the exact cell-equivalence rule;
2. laboratory trajectories remain supported;
3. mean convergence/ambiguity fixtures reproduce stable diagnostics;
4. invalid means are excluded under the declared minimum-count/fraction policy;
5. node-cloud positions equal i/N_i within 10^{-14} in fractional coordinates;
6. legacy scientific density values and meshes remain within the normative numerical policy.

37.3 LD0-R3 - Bounded global scene planning

Deliver Phase-A upper-bound plans, Phase-B exact index plans, planning-memory limits, scene-wide peak-memory accounting, and Phase-C global approval.

Acceptance:

1. every hard limit is exercised by a focused pre-allocation failure test;
2. no floating field or mesh allocation occurs before all channels pass Phase B;
3. `max_density_planning_bytes` bounds exact index construction;
4. estimated counts never fall below realized counts;
5. estimated total peak bytes are at least realized monitored peak bytes on required benchmarks, with overestimation reported but not treated as failure.

37.4 LD0-K - Canonical discrete smoothing operator

Implementation status: completed in `mdstats 0.19.43a0`.

Deliver `discrete_periodized_v1`, the $\sigma = 0$ identity path, direct and FFT dense convolution of the same stencil, explicit legacy compatibility, kernel diagnostics, and migration benchmarks.

Acceptance:

```
direct-vs-FFT relative L1 <= 5e-12
direct-vs-FFT relative L-infinity <= 2e-11
stencil sum absolute error <= 5e-15
integral error <= 5e-13 * max(1, total_measure)
```

All retained image contributions are counted exactly once before canonical aggregation. `legacy_spectral_v1` remains reproducible. No default switch occurs without approved release notes and compatibility evidence.

37.5 LD0-B - Effective artificial-broadening policy

Implementation status: completed in `mdstats 0.19.44a0`.

Deliver `effective_cic_stencil_rms_v1`, CIC-phase covariance, canonical-stencil covariance, versioned adaptive-resolution selection, and migration comparison against `gaussian_sigma_v1`.

Acceptance:

1. analytic CIC covariance matches brute-force assigned-node covariance with relative Frobenius error $\leq 5 \times 10^{-13}$;
2. stencil covariance matches its unaggregated image contributions with relative Frobenius error $\leq 5 \times 10^{-13}$;
3. the selected resolution satisfies $s_{\mathrm{art}} \leq \alpha s_q$ whenever the target is finite and resource-independent;
4. zero-spread and $\sigma = 0$ cases follow the declared policies;
5. no broadening-metric default changes silently.

37.6 LD1-A - Sparse CIC and canonical-convolution reference

Implementation status: completed in `mdstats 0.19.45a0`.

Deliver deterministic sparse CIC aggregation, sparse stencil scatter, exact normalization, HDR details, and dense conversion for small debugging cases.

Acceptance against dense direct convolution of `discrete_periodized_v1`:

```
relative L1 field error <= 2e-11
relative L-infinity field error <= 5e-11
absolute integral error <= 5e-13 * max(1, total_measure)
HDR threshold absolute difference <= 5e-12 * max(1, reference maximum)
achieved HDR mass-fraction difference <= 5e-13
```

Required cases include orthogonal and LTA-primitive cells, off-grid samples, face/edge/corner crossings, multiple images in support, overlapping sources, bimodal hopping, independent ensembles, and $\sigma = 0$.

37.7 LD1-B - Atomic block-sparse field

Implementation status: completed in `mdstats 0.19.46a0`.

Deliver production block packing, partial-block masks, public node access, atomic species/index selections, transactional sparse preflight, storage summaries, and canonical serialization.

A localized LTA benchmark must use no more than 10% of the dense logical scalar slots and must reduce allocated scalar slots by at least tenfold while preserving requested resolution and all LD1-A tolerances.

37.8 LD2-A - Sparse HDR and node-cloud rendering

Implementation status: completed in `mdstats 0.19.47a0`.

Deliver backend-neutral HDR details, exact node-coordinate clouds, deterministic selection, Cartesian bounds, trace provenance, and render-resource estimates without dense materialization. Identical inputs must produce byte-identical selected logical indices. Rendered Cartesian positions must agree with $\mathbf{f}H_d$ to $10^{-12}L_{\mathrm{ref}}$.

37.9 LD2-B - Periodic sparse mesh extraction

Implementation status: completed in `mdstats 0.19.48a0`.

Deliver candidate-cell components, exact cell masks or cell-aware contouring, partial blocks, lifted charts, winding detection, ghost lookup, canonical clipping, periodic-boundary edge pairing, deterministic duplicate removal, and fallbacks.

Acceptance:

```
interior mesh-edge incidence = 2
unpaired non-boundary edge count = 0
opposite-boundary seam mismatch <= 1e-10 * L_ref
duplicate canonical face count = 0
maximum mesh edge <= cell-diagonal upper bound * (1 + 1e-10)
```

Topology and canonicalized geometry must be unchanged under block-order permutation. Winding components must select the documented fallback deterministically.

37.10 LD3 - Framework vertex and edge block-sparse fields

Implementation status: completed in `mdstats 0.19.49a0`.

The common backend supports projected vertices, projected edges, and atom-resolved paths. Framework edge quadrature is resolution-aware, deterministic, and planned before allocation. Vertex occupancy and edge arc length retain separate scientific measures, units, field keys, provenance, and render groups.

Acceptance includes exact dense/sparse canonical equivalence, orientation reversal, periodic seams, exact total edge measure, structured provenance, explicit resolution-reference source, both edge sources, backend-neutral LD2 rendering, and the quantitative quadrature-convergence criteria specified under **Edge quadrature**.

37.11 LD4 - Transactional automatic backend selection

Implementation status: completed in `mdstats 0.19.50a0`.

Auto mode builds exact dense and local-sparse Phase-B candidates at one shared scientific resolution before scalar allocation. Each candidate records feasibility, logical and active nodes, stored values and blocks, kernel pairs, planning and retained bytes, estimated peak bytes, and a deterministic work proxy. Auto mode never lowers grid resolution, increases Gaussian bandwidth, loosens kernel tolerance, coarsens edge quadrature, or changes the scientific measure.

Normative field-local policy anchors are:

```
sparse active fraction >= 0.50 -> dense
sparse active fraction <= sparse_activation_fraction
and sparse estimated peak bytes <= 0.70 * dense -> sparse
otherwise -> lower estimated peak bytes, then lower estimated work, then dense on ties
```

The complete scene is approved by deterministic enumeration of feasible backend combinations. The score minimizes field-policy overrides first, then scene peak bytes, total estimated work, and sparse-field count. Any required global override is serialized explicitly. Selection records, both candidate estimates, reasons, policy anchors, and feasibility failures round-trip through canonical JSON and are retained in Phase-B plans and realized fields.

37.12 LD5 - Optimization and caching

Implementation status: completed in `mdstats 0.19.51a0`.

LD5 adds a production optimized sparse evaluator while retaining the LD1-A flat-node reference path. The implemented optimizations are preallocated periodic CIC contribution generation, chunked vectorized canonical pair generation, bounded dense `bincount` reduction for small logical grids, stable sparse reduction for large grids, chunked exact target-node planning, and a thread-safe bounded least-recently-used cache of immutable canonical stencil supports.

The public `DensityOptimizationOptions` record selects `optimized` or `reference`, enables or disables stencil caching, and controls pair-chunk size. Cache entries are keyed by exact logical shape, float64 display-cell bytes, Gaussian-bandwidth bits, and tail-tolerance bits. Cache hits revalidate the caller's current candidate and workspace limits. The cache is bounded to 16 entries and 256 MiB of retained NumPy arrays and is clearable through the public API.

Every optimized path satisfies the LD1-A field, integral, and HDR tolerances. Active logical nodes, block ordering, exact Phase-B hard counts, and LD4 backend selections remain unchanged. Compiled kernels, GPU execution, and parallel accumulation remain deferred.

37.13 LD6 - Multilevel AMR research gate

Implementation status: completed in `mdstats 0.19.52a0`.

LD6 adds a bounded, deterministic research profiler rather than a production multilevel field. It compares the realized backend and alternative 4^3, 8^3, and 16^3 single-level block plans against an optimistic dyadic coarse/fine surrogate. The surrogate retains HDR-selected bins exactly, replaces other positive bins by conservative piecewise-constant averages, and evaluates every periodic coarse-grid phase for factors two and four. All phases must satisfy the field, integral, and HDR tolerances, and adoption requires at least 2x worst-phase incremental storage reduction relative to the best single-level block plan.

The representative benchmark covers localized atomic density, separated framework vertices, overlapping oxygen clouds, multimodal Na hopping, projected framework edges, atom-resolved framework paths, and a broad mobile-ion field. The evidence outcome is `retain_single_level`. Only one path-like field shows more than 2x phase-robust optimistic gain; the projected-edge field that remains inefficient under the profiled block choices has no candidate within the scientific tolerances. The required two production-relevant insufficient cases are absent.

Therefore a production multilevel hierarchy is not authorized. Transfer, conservation, multilevel HDR integration, periodic coarse/fine ownership, and crack-free adaptive contouring remain deferred and require a new specification if future evidence reopens the question.

37.14 LD7 full-trajectory tractability standard

LD7 is the normative performance-correction layer for long adaptive-density trajectories. It does not authorize a new estimator or multilevel grid.

37.14.1 Separation of estimation roles

All selected frames remain density samples. Positional spread used only to resolve the adaptive grid may be estimated from a deterministic, weighted, stratified-random temporal subsample. The default is 128 frames with seed 0. Every temporal stratum contributes its complete frame weight through one selected representative. The exact sampled source indices and weights are audit metadata. Explicit `spread_sampling_strategy="all"` retains the prior all-frame diagnostic.

This sampling construction adapts stratified random sampling [10]. The ordered temporal strata and deterministic weighted selection are project-specific.

37.14.2 Bounded sparse realization

The optimized sparse backend shall not allocate arrays proportional to the complete kernel-pair count. It shall discover active target blocks in one bounded pair-stream pass and accumulate into those blocks in a second bounded pass. Peak pair workspace is controlled by `sparse_pair_chunk_size`.

Independent source identities shall be processed in ascending deterministic batches controlled by `sparse_group_batch_size` (default 8). Atomic identities, framework vertices, and framework edges/paths are valid groups. Batch fields share one grid and stencil and are merged by stable global node index with exact final measure correction.

37.14.3 Transactional planning

Phase-B planning shall use the same source-group batches as realization. It shall union exact per-batch target nodes and report cumulative pair count, peak batch pair count, batch count, final block packing, and bounded transient memory. Auto backend selection must use these execution-consistent counts.

37.14.4 Scientific invariants

LD7 may alter only execution and the bounded diagnostic sample used to choose resolution. It shall not silently change the chosen grid after resolution, Gaussian support, tail tolerance, sample weights, density units, total measure, HDR definition, or display geometry. The retained LD1-A reference path remains available.

37.14.5 Acceptance evidence

On the 1,300-frame Na-LTA acceptance trajectory, all-frame Na, Si, Al, and O density preparation completes under a 4 GiB workspace limit in approximately 125 seconds total on the validation runtime. The four fields process approximately 978 million cumulative kernel pairs while retaining exact integrated measures of 24, 24, 24, and 96 atoms.

The framework-dynamics scene schema at the LD7 milestone was `mdstats.framework-dynamics-scene.v14`; PARDENS4 advances the current scene schema to `mdstats.framework-dynamics-scene.v15`.

37.15 2026-07-21 - LD7 implementation update

- Marked LD7 implemented in `mdstats 0.19.53a0`.
- Added deterministic weighted temporal stratification for bounded periodic-spread estimation, with all selected frames retained in the final density.
- Added two-pass active-block discovery and block-local streaming convolution without complete pair-array materialization.
- Added exact deterministic source-group batching, sparse-field merging, and execution-consistent group-batched Phase-B planning.
- Added source/sample counts, selected frame indices, batch counts, cumulative and peak pair counts, reduction implementation, and workspace provenance.
- Demonstrated tractable 1,300-frame all-species Na-LTA density preparation under the 4 GiB validation limit.
- Advanced the framework-dynamics scene schema to `mdstats.framework-dynamics-scene.v14`.

37.16 LD8 exact finite-support execution refinement plan

Implementation status: LD8-P0 is implemented in `mdstats 0.19.54a0`; LD8-S0/S1 are implemented in `mdstats 0.19.55a0`; LD8-S2 is implemented in `mdstats 0.19.56a0`; LD8-S3 is implemented in `mdstats 0.19.57a0`; and LD8-S4 production integration and acceptance are implemented in `mdstats 0.19.58a0`.

LD8 accelerates the existing single-level local-sparse `discrete_periodized_v1` estimator without changing the estimator, resolved grid, Gaussian bandwidth, CIC assignment, density units, normalization, HDR semantics, or periodic geometry. It replaces repeated fine-node/stencil pair-stream planning with exact block-aware support planning and a hybrid bounded executor.

LD8 does **not** authorize multilevel AMR, variable-bandwidth KDE, relaxed Gaussian support, display-driven scientific coarsening, or a new density definition.

37.16.1 Fixed scientific invariants

The mathematical Gaussian has infinite support, but the numerical operator uses the finite radial support

$$r_\varepsilon = \sigma\sqrt{F^{-1}_{\chi^2_3}(1-\varepsilon)}.$$

The normal production policy remains

$$\varepsilon = 10^{-8}, \qquad r_{\varepsilon} \approx 6.334824\,\sigma.$$

The retained discrete stencil is normalized exactly. Its realized discrete covariance, combined with the analytic CIC phase covariance, remains the source of the effective artificial-broadening diagnostic. LD8 may not loosen `kernel_tail_tolerance`, shorten the support radius, replace the canonical normalized stencil, or revert the scientific criterion to Gaussian width alone.

Support is defined around occupied periodic CIC source nodes. Let $\mathcal{S}$ be the set of occupied source nodes and $\mathcal{K}_{\varepsilon}$ the exact retained signed stencil offsets. The exact target-node support is

$$\mathcal{A} = \{(\mathbf{n}_s + \delta) \bmod \mathbf{N} : \mathbf{n}_s \in \mathcal{S}, \delta \in \mathcal{K}_{\varepsilon}\}.$$

Inactive logical nodes are exact implicit zeros. An optimization is acceptable only if it produces this support and the same normalized field within the declared floating-point tolerance.

37.16.2 Evidence gap and production baseline

The implemented LD7 full-trajectory benchmark used a looser diagnostic run tolerance than the retained production value. The 10^{-8} stencil can contain several times more offsets than the prior stress-test stencil, depending on the resolved metric and grid. A speedup demonstrated only at a looser cutoff is insufficient evidence for LD8.

Before public implementation records are frozen, LD8 must establish a production baseline using:

- the complete 1,500-frame LTA trajectory;
- Na, Si, Al, and O density channels;
- `kernel_tail_tolerance=1.0e-8`;
- the canonical periodized stencil;
- the effective CIC-plus-stencil broadening criterion;
- all final trajectory frames retained in the estimator;
- cold-cache and warm-cache runs;
- separate timing and memory records for CIC deposition, support planning, convolution, packing, HDR selection, meshing, and serialization.

This evidence stage is **LD8-P0**. It must also measure source-node occupancy, source-block occupancy, target-block fill, support fragmentation, and representative block shapes 8^3, 16^3, and 32^3. The default 16^3 storage block remains in force unless P0 provides contrary evidence.

37.16.3 Required object separation and cache correctness

The initial atlas draft mixed reusable kernel geometry with field-specific source support. LD8 replaces it with four distinct immutable concepts.

37.16.3.1 Canonical finite stencil

```
@dataclass(frozen=True)
class PeriodicGaussianStencilSupport:
    schema_version: str
    logical_grid_shape: tuple[int, int, int]
    display_cell: NDArray[np.float64]
    gaussian_bandwidth: float
    kernel_tail_tolerance: float
    cutoff_radius: float
    signed_offsets: NDArray[np.int64]
    normalized_weights: NDArray[np.float64]
    discrete_covariance: NDArray[np.float64]
    metadata: FrozenJSONMapping
```

This record is reusable across fields only when its exact cache key matches the logical-grid shape, float64 cell bytes, Gaussian-bandwidth bits, tolerance bits, and operator version.

37.16.3.2 Reusable block-routing template

```
@dataclass(frozen=True)
class PeriodicKernelBlockRouting:
    schema_version: str
    logical_grid_shape: tuple[int, int, int]
    storage_block_shape: tuple[int, int, int]
    block_grid_shape: tuple[int, int, int]
    terminal_block_extents: tuple[tuple[int, ...], ...]
    relative_block_offsets: NDArray[np.int64]
    grouped_stencil_ranges: tuple[BlockOffsetStencilGroup, ...]
    terminal_validity_bitsets: NDArray[np.uint64]
    metadata: FrozenJSONMapping
```

This record contains only routing geometry shared by fields with the same exact stencil and storage layout. It contains no source blocks, source masks, or target support. Its cache key includes the exact stencil identity, storage-block shape, logical-grid shape, and terminal extents.

37.16.3.3 Field-specific sparse CIC source

```
@dataclass(frozen=True)
class PeriodicPackedCICSourceField3D:
    schema_version: str
    logical_grid_shape: tuple[int, int, int]
    storage_block_shape: tuple[int, int, int]
    source_block_indices: NDArray[np.int32]
    occupancy_bitsets: NDArray[np.uint64]
    block_value_offsets: NDArray[np.int64]
    packed_values: NDArray[np.float64]
    total_measure: float
    metadata: FrozenJSONMapping
```

All weighted samples for one requested output field are deposited once into this source field. This object is field-specific and is not globally cached.

37.16.3.4 Field-specific exact support atlas

```
@dataclass(frozen=True)
class DensitySupportAtlas:
    schema_version: str
    source_field_identity: str
    active_target_block_indices: NDArray[np.int32]
    target_support_bitsets: NDArray[np.uint64]
    source_to_target_block_ranges: NDArray[np.int64]
    source_to_target_block_indices: NDArray[np.int32]
    connected_component_labels: NDArray[np.int32] | None
    metadata: FrozenJSONMapping
```

The support atlas is derived from one source field and one routing template. Its identity must include the source-field content identity or it must remain attached to that source field and never enter a cross-field cache. This prevents incorrect reuse between spatially different fields that share the same cell, grid, and Gaussian.

37.16.4 Exact block support by bitset dilation

With the default storage block, each local support mask contains

$$16^3 = 4096$$

bits, or 512 bytes. Exact support planning shall use packed bitsets rather than `bool` arrays or global coordinate triples whenever practical.

For each occupied source-block mask, define its exact valid extent $\mathbf{E}$ and the componentwise signed-stencil extrema $\delta_{\min}$ and $\delta_{\max}$. LD8-S1 embeds the source mask in a bounded lifted brick of shape

$$\mathbf{Q} = \mathbf{E} + \delta_{\max} - \delta_{\min}.$$

The halo guarantees that adding any retained signed offset cannot cross a flattened brick row or leave the brick. The packed source mask is therefore shifted once per retained stencil offset and OR-reduced exactly. The resulting lifted support is unpacked once, mapped back to logical coordinates, folded periodically, and packed into canonical target-block bitsets. The target support is the periodic dilation

$$\mathcal{A} = \mathcal{S} \oplus \mathcal{K}_{\varepsilon}.$$

This implemented path is identified as `ld8_s1_exact_source_block_padded_bitset_dilation_v1`. The reusable routing groups remain available for later target-owned convolution scheduling, but S1 support construction does not build a fine local routing table. The plan must not materialize a complete source-node by stencil-offset map. Storage proportional to

$$B_x B_y B_z \, |\mathcal{K}_{\varepsilon}|$$

per routing class is forbidden.

The routing specification must define:

- local bit ordering;
- signed shift and carry across block faces, edges, and corners;
- periodic wrapping in logical-node coordinates;
- terminal partial-block validity masks;
- source-terminal and target-terminal extent classes;
- canonical block ordering and duplicate suppression.

For every source set, the implementation must prove or exhaustively verify

$$\text{nodes(atlas)} = \{(\mathbf{n}_s + \delta) \bmod \mathbf{N}\}.$$

At most eight local extent classes occur from full/terminal status along the three axes, but source/target class pairs and periodic wrap must be handled explicitly.

37.16.5 One global CIC source field

For one scientific output channel, all weighted samples are deposited once:

$$m_{\mathrm{CIC}}(\mathbf{n}) = \sum_s w_s C_h(\mathbf{n} - \mathbf{x}_s).$$

Atomic identities, framework vertices, and framework edge/path identities remain in provenance, but they do not create separately normalized partial fields unless the caller explicitly requests separate outputs. Linearity gives

$$\rho = \Delta V^{-1}(g * m_{\mathrm{CIC}}).$$

Execution may be chunked by source block or compute tile. Source-group field construction, repeated support discovery, repeated normalization, and sparse partial field merging are excluded from the target production path.

37.16.6 Storage blocks and compute tiles are distinct

LD8 separates three independent granularities:

```
storage_block_shape
convolution_tile_shape
render_tile_shape
```

The storage block controls sparse indexing and final packing. A convolution tile may merge adjacent storage blocks to improve cache locality or FFT efficiency. A render tile is governed by contour geometry and belongs to LD9. No API shall assume that these shapes are equal.

The default storage block remains $(16, 16, 16)$. Compute-tile selection is internal and must be recorded in provenance.

37.16.7 Canonical target-owned direct executor

The first exact executor is target-owned block-local direct convolution. Each approved target block is accumulated by one deterministic owner from all source blocks that can reach it.

Required execution order:

```
obtain canonical finite stencil
aggregate one global packed CIC source field
obtain reusable routing template
construct exact field-specific support atlas
approve target storage and transient workspace
for target block in canonical order:
    clear one bounded dense or packed accumulator
    for contributing source block in canonical order:
        accumulate exact local stencil interactions
    apply the common physical scale
    pack positive values and occupancy bits immediately
normalize once to the target measure
finalize immutable field metadata
```

The inner interaction kernel must be compiled or genuinely vectorized. Python loops may schedule blocks or tiles, but may not iterate over individual fine-node/stencil pairs. Global target-coordinate arrays are forbidden.

Target ownership permits immediate output packing and avoids concurrent writes to one target block. The retained LD1-A and LD7 paths remain numerical and migration oracles.

37.16.8 Hybrid tiled convolution executor

Block reorganization alone does not remove the arithmetic

$$N_{\text{source}}|\mathcal{K}_{\varepsilon}|.$$

LD8 therefore requires a hybrid production executor rather than treating FFT acceleration as optional late research.

37.16.8.1 Sparse-direct mode Use the canonical target-owned direct kernel for low-fill, fragmented source regions. The cost estimate must include occupied source nodes, reachable target blocks, exact stencil groups, and expected local fill.

37.16.8.2 Tiled overlap-add FFT mode For dense or moderately filled compute tiles:

1. assemble the tile's CIC source values into a bounded dense brick;
2. add the complete 10^{-8} stencil halo;
3. perform zero-padded linear convolution with the same normalized metric-aware discrete stencil;
4. overlap-add only the owned target interior into canonical target blocks;
5. release the dense brick and FFT workspace.

This is a local linear-convolution executor, not a global dense periodic FFT. The standard overlap-add organization follows discrete-time convolution practice [11]; the metric-aware three-dimensional stencil, periodic target ownership, and sparse tile selection are project-specific.

A naïve three-pass separable Gaussian is not valid for a general triclinic fractional grid because the Cartesian metric can introduce cross terms.

The executor selector must compare measured or calibrated estimates of direct and FFT costs. It must retain direct fallback for sparse or pathological tiles. FFT plans, thread count, library version, padding shape, and compute-tile shape enter provenance.

37.16.9 Packed final block-sparse field

Fixed dense values inside every active storage block can waste most retained memory. LD8 therefore authorizes a packed positive-value representation:

```
@dataclass(frozen=True)
class PeriodicPackedBlockScalarField3D:
    schema_version: str
    logical_grid_shape: tuple[int, int, int]
    storage_block_shape: tuple[int, int, int]
    active_block_indices: NDArray[np.int32]
    occupancy_bitsets: NDArray[np.uint64]
    block_value_offsets: NDArray[np.int64]
    packed_values: NDArray[np.float64]
    block_min_values: NDArray[np.float64]
    block_max_values: NDArray[np.float64]
    total_measure: float
    voxel_volume: float
    metadata: FrozenJSONMapping
```

Inactive and exact-zero nodes are implicit zeros. A reusable 16^3 scratch block may be inflated for local algorithms. Public accessors must preserve the existing backend-neutral scalar-field protocol.

Packing must occur target-by-target so that completed dense accumulators do not remain resident. `float64` remains required for scientific field values.

37.16.10 Downstream numerical reuse

The packed field and support atlas shall serve downstream numerical operations without reconstructing global coordinates or scanning the full logical cell.

Integration and normalization sum packed values only; inactive nodes contribute exact zero.

Block extrema compute minimum/maximum and positive count during packing. These records drive contour pruning and diagnostics.

HDR thresholds do not cache a full global descending permutation and cumulative array by default. Resolve the small requested set of mass fractions using exact weighted multi-selection/partitioning with deterministic tie handling. Cache only fraction, threshold, achieved mass, tie count, and selected-node count. A full sorted order is an optional lazy diagnostic.

Contour support for level ρ_q, include only blocks and local cells satisfying $\min \rho < \rho_q \leq \max \rho$. Scientific support and contour support remain separate views.

Connected components compute lazily only when required by convolution scheduling or rendering. Component labels are field-specific and must not be part of a reusable kernel cache.

37.16.11 Transactional planning and resource accounting

Phase-B planning must record at least:

```
source CIC node count
source storage-block count
source packed bytes
exact target block count
```

```
exact target positive-node upper bound
routing-template bytes
support-atlas bytes
packed target-field bytes
chosen executor per tile
compute-tile count and shapes
direct interaction estimate
FFT padded shapes and workspace estimates
largest per-tile transient workspace
retained final-field bytes
predicted scene peak bytes
```

Peak memory is

$$B_{\text{peak}} = B_{\text{retained}} + \max_t B_{\text{transient},t},$$

not the sum of mutually exclusive tile or shell transient peaks. Phase C must approve the complete requested scene before any scientific target field is allocated.

Caches must be byte-bounded, thread-safe, clearable, and keyed by exact immutable scientific inputs. Source-field and support-atlas objects are not placed in global cross-field caches.

37.16.12 Determinism and numerical reproducibility

LD8 distinguishes two execution contracts.

Canonical direct mode fixed target/source traversal, byte-identical repeated output on the validation platform, and use as the migration oracle.

Optimized direct/FFT mode exact support and normalization, declared numerical agreement with canonical direct, and reproducibility under the recorded library/thread configuration. Deliberately permuted execution order is tested by numerical tolerance rather than byte identity unless a reproducible summation algorithm is used.

Canonical serialization order remains deterministic for all immutable records.

37.16.13 Revised staged implementation

37.16.13.1 LD8-P0 - Production evidence and executor spike

- establish the full 10^{-8} all-species baseline;
- profile every pipeline stage and occupancy statistic;
- compare storage blocks 8^3, 16^3, and 32^3;
- prototype target-owned direct and tiled overlap-add FFT kernels;
- calibrate the executor cost model;
- set quantitative wall-time and memory targets from evidence.

No public API migration occurs in P0.

Recorded P0 evidence (`mdstats 0.19.54a0`). The full 1,500-frame, four-species current-LD7 run at the retained 10^{-8} cutoff completed successfully:

Channel	Grid	Estimated kernel pairs	LD7 scientific time	Peak RSS
Na	540^3	435,976,760	41.934 s	1.214 GiB
Si	1038^3	657,089,560	54.244 s	1.243 GiB
Al	1037^3	690,629,007	56.022 s	1.259 GiB
O	646^3	2,257,044,957	187.486 s	2.397 GiB

The four scientific fields require 339.686 s in aggregate. Packing and three HDR queries add approximately 12.97 s. The complete evidence script, including repeated planning and direct/FFT spikes, required 394.270 s. All field integrals recovered 24, 24, 24, and 96 atoms to the existing tolerance.

The bounded exact-kernel spikes measured approximately 26–55x direct-to-FFT speed ratios with relative L^1 disagreement near 10^{-15}. This is not a production FFT benchmark, but it is sufficient evidence that LD8-S3 must remain a core stage.

On the same validation host, the LD8-S4 production gate is provisionally:

- no more than 120 s aggregate scientific field time for the four channels;
- at least 3.0x aggregate speedup over the recorded 339.686 s LD7 baseline;
- no more than 1.5 GiB process peak RSS for any channel;
- no scientific or fallback benchmark regression greater than 10% when the selector chooses its retained direct path.

The wall-time thresholds must be re-recorded, not silently reused, when the reference host or thread configuration changes.

37.16.13.2 LD8-S0 - Split contracts and exact-support proof Status: implemented in `mdstats 0.19.55a0`.

- add the stencil, routing, packed-source, support-atlas, and packed-field records;
- define exact cache keys and ownership;
- specify bit ordering, terminal classes, periodic wrap, and serialization;
- add formal/exhaustive support-equivalence tests;
- preserve the current field protocol through adapters.

37.16.13.3 LD8-S1 - Bitset support atlas and exact planning Status: implemented in `mdstats 0.19.55a0`.

- build one global packed CIC source field;
- construct exact support by bounded source-block padded-bitset dilation;
- produce exact target blocks and masks without fine-pair arrays;
- add transactional memory and operation estimates;
- retain LD7 fallback.

Recorded S1 evidence (`mdstats 0.19.55a0`). The full 1,500-frame production benchmark at `kernel_tail_tolerance=1.0e-8` produced exact target-node counts matching the completed LD7 baseline:

Channel	Source nodes	Source blocks	Target blocks	Exact target nodes	Atlas time	Atlas bytes
Na	36,280	280	1,322	1,728,706	13.388 s	0.685 MiB
Si	54,680	280	1,381	1,833,591	13.301 s	0.714 MiB
Al	57,471	308	1,432	1,952,525	15.195 s	0.742 MiB
O	187,821	1,132	5,625	7,274,190	55.158 s	2.915 MiB

The exact planner replaced complete fine-pair enumeration with 129.6–195.3 times fewer source-block/stencil bit shifts by operation count. The complete benchmark took 138.680 s including repeated adaptive resolution and source preparation; atlas construction required 97.042 s. No complete fine-pair array was allocated, and no field-specific atlas entered a global cache. This evidence authorizes LD8-S2 but does not itself migrate the production density-value executor.

37.16.13.4 LD8-S2 - Canonical target-owned direct realization Status: implemented in `mdstats 0.19.56a0` as a bounded migration oracle; production dispatch remains LD7.

- implement the vectorized target-owned local kernel with one deterministic owner per target block;
- use conservative translated-source interval pruning and bounded offset/source chunks;
- preallocate one packed target vector from atlas bit counts and reuse one dense target-block accumulator;

- normalize once and apply the floating residual to the largest positive node;
- verify every completed block against the exact atlas support bitset;
- prove agreement with LD1-A on periodic and partial-terminal fixtures;
- establish deterministic canonical output and identity-bound execution plans.

Recorded S2 evidence (`mdstats 0.19.56a0`). A bounded production-stencil benchmark used a 96^3 grid, $\sigma = 2h$, and the retained 10^{-8} cutoff, yielding 8,409 exact stencil offsets. Cases with 64, 128, and 512 distributed source nodes covered 0.538–4.305 million exact contributions and agreed with LD1-A to relative L^1 errors between 1.47×10^{-18} and 2.13×10^{-16}. Packed retained storage was approximately half the LD1-A flat-index/value representation. The current NumPy target-owned oracle required 1.5–3.2 times the LD1-A runtime on these bounded cases, so S2 establishes correctness and bounded ownership but does not authorize production migration. This evidence strengthens the requirement for the compiled/tiled hybrid S3 executor.

37.16.13.5 LD8-S3 - Hybrid tiled direct/overlap-add FFT executor

Status: implemented in `mdstats 0.19.57a0` as an opt-in accelerator and integrated into normal production dispatch by LD8-S4 in `mdstats 0.19.58a0`.

- partition one globally aggregated packed CIC source into deterministic compute tiles independent of the storage-block shape;
- use bounded sparse direct scatter for fragmented/lightly occupied tiles;
- use zero-padded three-dimensional FFT linear convolution and periodic overlap-add for populated tiles;
- encode the triclinic metric only through the exact retained discrete stencil, without assuming separability;
- add explicit direct/FFT forcing, calibrated auto-selection, byte-bounded kernel-spectrum caching, exact target lookup, and complete executor provenance;
- retain the S2 target-owned direct path as the migration oracle;
- repair any nonpositive finite-support-boundary FFT node by exact direct recomputation and record the repair count;
- keep `production_backend=false` until the complete S4 gate passes.

Recorded S3 focused evidence (`mdstats 0.19.57a0`). A 96^3 production-cutoff benchmark with 8,409 retained stencil offsets covered fragmented, compact, periodic-boundary, and oxygen-heavy source layouts. The hybrid executor agreed with S2 to relative L^1 errors between 2.90×10^{-18} and 5.32×10^{-16} and was 2.81–40.44 times faster than S2 on the tested cases. Forced direct and forced FFT paths also passed periodic and partial-terminal fixtures within the declared 5×10^{-12} and 5×10^{-11} tolerances.

Full 1,500-frame 10^{-8} field evidence was completed for Na, Si, and Al. Their hybrid value-realization times were 2.088 s, 2.015 s, and 2.146 s, compared with 41.934 s, 54.244 s, and 56.022 s for the recorded LD7 field baselines. The selector used mixed direct/FFT tile sets, all fields recovered exactly 24 atoms, and finite-support-boundary repair counts were 10, 1, and 0. A complete production oxygen run was not completed in the shared validation environment because construction of the pre-existing S1 oxygen atlas exhibited unstable wall time; the oxygen-heavy focused fixture passed, but S4 must rerun all four production channels before default migration.

S3 also removes repeated per-source-block `numpy.unique` work from S1 planning by caching axis-level stencil target counts. This changes planning cost only and preserves exact support.

37.16.13.6 LD8-S4 - Production dispatch, downstream numerical reuse, and performance gate

Status: implemented in `mdstats 0.19.58a0`; hybrid local-sparse realization is the normal production path.

- integrated the S1 support atlas and S3 hybrid executor into normal atomic, framework-vertex, and framework-edge dispatch;
- restricted LD7 fallback to explicit resource/complexity failures and prohibited fallback for scientific or identity defects;
- added certified binary-FFT support dilation for production-size stencils while retaining exact bitset dilation as the small-case oracle;
- added one bounded positive-value ordering for exact multi-HDR selection;
- added lazy contour-support planning and optional periodic components from block extrema;
- retained packed scientific output and complete executor/fallback provenance;
- completed the all-frame, four-species production acceptance benchmark.

Recorded S4 production evidence (`mdstats 0.19.58a0`). The normal dispatcher prepared Na, Si, Al, and O fields on resolved grids of 540^3, 1038^3, 1037^3, and 646^3 at the retained 10^{-8} cutoff. Scientific preparation times were 11.189 s, 11.172 s, 12.658 s, and 45.496 s, respectively, for an aggregate of 80.515 s. This is 4.219 times faster than the recorded 339.686 s LD7 baseline. All fields recovered their exact selected measure, used the production hybrid backend without fallback, resolved 50/80/95 percent HDR levels, and remained below 1.218 GiB measured channel peak RSS. The S4 gate therefore authorizes normal production migration.

37.16.14 LD8 acceptance gates

LD8 is accepted only when all of the following hold.

Scientific identity

- source-node aggregation agrees with the reference path;
- support nodes are exactly the CIC/stencil Minkowski sum;
- active values agree with canonical direct within 5×10^{-12} relative L^1 for direct mode and a separately declared, justified FFT tolerance no weaker than 5×10^{-11};
- target measure and units agree to existing normalization tolerances;
- effective covariance and broadening metadata are unchanged;
- periodic translation and block-boundary invariance pass.

Planning and storage

- no array is proportional to the complete fine-pair count;
- no routing record is proportional to $B_x B_y B_z |\mathcal{K}_\varepsilon|$;
- terminal partial blocks are covered exhaustively;
- packed storage is no larger than dense-in-active-block storage and demonstrates a substantial reduction on localized acceptance fields;
- measured peak memory does not exceed the transactional prediction plus the declared allocator tolerance.

Performance

- the exact 10^{-8} all-species production benchmark completes within the P0-approved target;
- LD8 materially outperforms LD7 on the same scientific workload;
- no representative benchmark regresses beyond the approved fallback threshold;
- cold- and warm-cache results are reported separately;
- executor selection rationale is included in every field's provenance.

37.16.15 Planned module boundaries

Responsibility	Planned module
canonical finite stencil support	`mdstats.plotting.density_kernel`
reusable block-routing templates and terminal classes	`mdstats.plotting.density_block_routing`
packed CIC source fields and bitset support dilation	`mdstats.plotting.density_support_atlas`
packed scientific scalar fields	`mdstats.plotting.density_packed_field`
target-owned direct convolution	`mdstats.plotting.density_block_direct`
tiled overlap-add FFT convolution and selector	`mdstats.plotting.density_tiled_fft`
exact multi-HDR selection and contour support	`mdstats.plotting.density_field_queries`
transactional planning and executor cost model	`mdstats.plotting.density_scene_planning`
migration orchestration and fallback	`mdstats.plotting.density_backend_selection`

The existing `density_sparse_reference`, `density_sparse_optimization`, and `density_block_sparse` modules remain the LD1-A/LD7 comparison paths until migration is complete.

37.17 LD9 hard-budget display-mesh and browser-rendering plan

Implementation status: approved normative follow-on architecture; not implemented in `mdstats 0.19.53a0`.

LD9 addresses visualization cost independently of LD8 scientific-field cost. A fine scientific grid may be necessary to bound artificial density broadening. The browser mesh, however, must contain only enough geometry to reproduce

each selected isosurface within declared physical-space and topological tolerances. Scientific grid resolution is never used directly as a browser face-count requirement.

The defining LD9 invariant is:

$$N_{\text{faces}}^{\text{serialized}} \leq N_{\text{faces}}^{\max}$$

for every successful `interactive_browser` export, after all shells, species, components, periodic display replication, and trace assembly are counted. An export that cannot satisfy the face limit and fidelity constraints fails before HTML writing; it never emits an oversized artifact.

37.17.1 Scientific/display separation

HDR thresholds are computed from the complete scientific field. Mesh extraction, triangulation, simplification, coordinate precision, hover metadata, trace grouping, and browser serialization are display operations. They may not alter:

- the scientific field;
- Gaussian bandwidth or kernel support;
- CIC deposition;
- HDR fraction or threshold;
- enclosed-mass records;
- density normalization;
- selected species, shells, frames, or trajectories.

Every output mesh records:

```
scientific field identity
HDR fraction and scientific threshold
render profile
raw extraction method and render-tile layout
raw vertices and faces generated per tile
local presimplification method and retained counts
global simplification method
physical surface-error tolerance
achieved scalar residual and geometric error
normal error
component, topology, and periodic-seam checks
final vertices and faces before display replication
display replication multiplicity
final serialized vertices and faces
retained payload bytes
peak transient mesh bytes
browser validation results
```

A raw reference mesh may be retained by validation tools, but production rendering must not require it to remain resident.

37.17.2 Render profiles and hard browser contract

LD9 introduces explicit render profiles:

```
DensityRenderOptions(
    render_profile="interactive_browser",  # interactive_browser | raw_reference
    extraction=MeshExtractionOptions(...),
    simplification=MeshSimplificationOptions(...),
    browser_budget=BrowserMeshBudget(...),
)
```

`interactive_browser` production interactive Plotly/WebGL output. Hard final face, vertex, trace, and payload limits are mandatory. Oversized output is prohibited.

`raw_reference` development and validation output. Simplification may be disabled and browser limits may be relaxed explicitly. This profile is never selected implicitly and is not intended for ordinary browser use.

The normative initial browser budget is:

```
BrowserMeshBudget(
    max_final_density_faces=300_000,
    max_final_density_vertices=200_000,
    max_final_html_bytes=40 * 1024**2,
    max_plotly_traces=64,
    apply_after_display_replication=True,
    hard_limit=True,
)
```

The limits above are initial production targets for the all-species LTA stress scene. LD9-V0 may tighten them after browser measurements. Raising them requires an explicit user option and must still pass the browser smoke test. `hard_limit=False` is invalid for `interactive_browser`.

37.17.3 Raw, transient, and final resource limits

Extraction, simplification, and serialization use distinct limits:

```
MeshExtractionOptions(
    render_tile_shape=None,              # resolved independently of storage blocks
    max_crossing_cells_per_tile=...,
    max_raw_faces_per_tile=...,
    max_raw_vertices_per_tile=...,
    max_transient_mesh_bytes=...,
)

MeshSimplificationOptions(
    enabled=True,
    max_surface_error=0.02,              # Angstrom; calibrated by LD9-V0
    max_normal_error_degrees=8.0,
    target_final_faces=None,             # assigned by scene allocator
    preserve_topology=True,
    preserve_periodic_seams=True,
    min_component_faces=...,
    local_presimplification=True,
)
```

The three resource classes are:

1. **Raw tile limits** - bound one extraction tile before local reduction.
2. **Transient mesh limits** - bound peak extraction, indexing, clipping, and simplification workspace.
3. **Final browser limits** - bound complete serialized geometry and HTML after all display replication.

A large estimated global raw face count is not itself a rejection condition. Tiled extraction is allowed to process a large contour incrementally. Rejection occurs when one tile exceeds its bounded raw workspace, total planned work exceeds an explicit work limit, or the final browser contract cannot be met.

Peak memory is accounted as:

$$B_{\text{peak}} = B_{\text{retained final}} + \max_t B_{\text{transient},t},$$

not as a sum of mutually exclusive per-tile transient peaks.

37.17.4 Contour-local tiled extraction

The current cell-by-cell invocation of a general marching-cubes implementation is replaced by bounded tile- or component-level extraction:

1. compute per-block minimum and maximum field values during scientific-field packing;
2. select only blocks satisfying
$$\rho_{\min,B} < \rho_q \leq \rho_{\max,B};$$
3. assemble bounded crossing tiles plus a one-node halo;
4. call the topologically consistent marching-cubes implementation once per tile and contour level;
5. assign deterministic logical-grid-edge keys to shared vertices;
6. classify triangles as wholly inside, wholly outside, or boundary-intersecting before invoking general periodic clipping;
7. emit indexed tile geometry and release scalar tile arrays immediately.

Marching-cubes geometry follows Lorensen and Cline [3] and the topologically consistent case handling of Lewiner et al. [4]. Periodic tile ownership, logical-edge keys, and canonical seam pairing are project-specific.

The extractor reports raw counts per tile. It does not accumulate every raw tile mesh in memory before simplification.

37.17.5 Streaming local presimplification

Each tile follows the bounded pipeline:

```
extract tile
-> form indexed logical-edge geometry
-> protect tile boundaries and periodic seam vertices
-> constrained local presimplification
-> append retained indexed geometry
-> release raw tile geometry
```

Local presimplification removes redundant low-curvature and near-coplanar geometry before global assembly. It must preserve all protected tile-boundary, contour-component, and periodic-seam constraints. It is not permitted to consume the complete final face budget independently; its purpose is to reduce transient and retained raw mesh memory.

The retained tile meshes are merged by deterministic logical-edge keys. A second, global seam-aware simplification then allocates and enforces the final scene budget.

37.17.6 Error-controlled periodic mesh simplification

After tiled extraction and local reduction, apply topology-aware simplification using a quadric error metric in the spirit of Garland and Heckbert [12]. Periodic seam counterparts, component boundaries, nonmanifold constraints, and protected small components must be preserved.

The final simplifier is controlled simultaneously by:

- maximum physical-space surface error;
- scalar residual relative to the scientific field;
- normal error;
- component and topology preservation;
- periodic seam consistency;
- the scene-wide hard face and vertex budgets.

The scientific field remains the geometric oracle. Production validation does not require retention of a complete raw mesh. It samples or adaptively evaluates:

$$|\rho(\mathbf{x}) - \rho_q|,$$

bidirectional surface displacement, and field-gradient normal agreement. Raw meshes remain optional test oracles in LD9-V0.

A face target is not advisory for `interactive_browser`. If no mesh satisfies both the fidelity/topology requirements and the final hard budget, the renderer returns a structured failure before serialization.

37.17.7 Scene-wide face-budget allocation

The budget allocator operates on the complete requested scene, not independently on each trace. For shells s and connected components c,

$$\sum_{s,c} m_s N_{s,c}^{\text{faces}} \leq N_{\text{faces}}^{\max},$$

where m_s is the approved display-replication multiplicity.

Allocation proceeds transactionally:

1. apply all display-replication multipliers;
2. reserve a minimum topology-preserving budget for every retained component;
3. estimate component area, curvature, contour complexity, and screen-space significance;
4. assign shell weights using opacity and visual role;
5. allocate remaining faces under the hard scene cap;
6. simplify components within their assigned budgets and fidelity limits;
7. redistribute unused budgets deterministically;
8. verify the final post-replication totals before Plotly trace construction.

Low-opacity outer shells may receive fewer faces or a looser validated display-error tolerance than inner shells, but they may not receive altered scientific thresholds. No component or shell is dropped silently.

37.17.8 Browser payload and trace organization

Production browser output must:

- cast final display coordinates to `float32` after validation;
- use 32-bit face indices when valid;
- remove repeated per-vertex constant `customdata`;
- disable density-mesh hover by default or store constants once at trace level;
- group atomic trajectories by species with `NaN` separators rather than one trace per atom;
- account for graph, trajectory, mesh, legend, and helper traces against `max_plotly_traces`;
- write final traces only after hard face, vertex, and byte preflight passes;
- serialize incrementally where supported and release transient meshes;
- report retained Plotly array bytes separately from temporary JSON/HTML encoding overhead.

Scientific fields remain `float64`; only final validated display geometry may use compact dtypes.

37.17.9 Structured success and failure semantics

An `interactive_browser` request has exactly two valid outcomes.

Success requires:

- every requested scientific channel and shell is present;
- scientific fields, thresholds, and enclosed masses are unchanged;
- topology and periodic seams pass;
- geometric and normal errors pass;
- final post-replication face, vertex, trace, and HTML-byte limits pass;
- browser smoke validation passes for release benchmarks.

Structured failure requires:

- no oversized HTML is written;
- the scientific scene remains available;
- the limiting constraint is identified;
- requested and minimum measured face counts are reported when available;
- achieved error at the nearest feasible budget is reported;
- no Gaussian, grid, shell, species, trajectory, or display-cell policy is changed.

The failure record is:

```
BrowserMeshBudgetFailure(
    violated_limit=...,
    requested_budget=...,
    measured_or_estimated_value=...,
    nearest_feasible_faces=...,
    nearest_feasible_surface_error=...,
    nearest_feasible_normal_error=...,
    shell_component_diagnostics=...,
)
```

It is raised as a structured `GraphComplexityError` subtype or payload. "Fidelity wins" means failure rather than silent degradation or oversized export.

37.17.10 Browser usability validation

Face count is necessary but not sufficient. LD9 release validation includes an automated Chromium/WebGL smoke test on the declared validation workstation:

- HTML loads without JavaScript error or WebGL context loss;
- the first complete interactive frame appears within the calibrated limit;
- scripted camera rotation completes;
- visibility toggles for representative density and trajectory traces complete;
- no individual Plotly trace exceeds validated vertex/index limits;
- memory use remains within the declared browser benchmark envelope.

Initial targets for the all-species LTA scene are:

```
first complete interactive frame <= 15 s
no WebGL context loss
scripted camera-orbit median >= 20 frames/s
representative trace toggle <= 2 s
```

LD9-V0 calibrates these thresholds and records the browser, GPU, driver, Plotly, and Chromium versions. Release comparisons use the same declared benchmark environment.

37.17.11 LD9 staged implementation

37.17.11.1 LD9-V0 - Rendering baseline, browser calibration, and fidelity metrics

- preserve raw reference meshes only for development validation;
- record stage timings, per-tile and global raw counts, components, payload, and peak memory;
- define scalar-residual, symmetric surface-distance, normal-error, topology, and seam metrics;
- calibrate browser face, vertex, trace, byte, load-time, and frame-rate limits;
- freeze the first `interactive_browser` profile.

37.17.11.2 LD9-V1 - Bounded tiled contour extraction **Status:** implemented in `mdstats 0.19.59a0`; this is the normal nonwinding local-sparse contour-extraction path.

- identify exact crossing cells by expanding only nodes above the immutable `float32` render level;
- retain the scientific HDR threshold while applying a deterministic 16-ULP upper-endpoint guard only to numerically point-like `float32` display levels;
- partition crossing cells into deterministic `32^3` render tiles with partial terminal tiles;
- invoke Lewiner marching cubes once per tile rather than once per logical cell;
- assign deterministic logical-edge vertex identities and preserve canonical periodic seam copies;
- classify triangles into inside, outside, and boundary-intersecting fast paths;
- enforce raw per-tile, total-work, planning, and transient-memory limits before or during extraction;
- release tile-local scalar and raw geometry before the next tile;
- retain the cell-wise extractor only as an explicit validation oracle.

V1 does not simplify the indexed output. The saved four-species 50% HDR stress shells contain 283,531 crossing cells but require only 694 marching-cubes calls, a 408.5-fold call-count reduction. They produced 565,474 unsimplified indexed faces in 55.42 s total. This established the bounded extraction baseline subsequently consumed by LD9-V2.

37.17.11.3 LD9-V2 - Local and global periodic simplification Status: implemented in `mdstats 0.19.60a0` and extended by the periodic-quotient path in `mdstats 0.19.61a0`.

- apply streaming tile-local QEM presimplification only to closed components wholly inside one render tile;
- retain every tile-boundary component exactly during the local pass;
- simplify closed interior components independently using quadric-error decimation;
- reconstruct paired canonical pieces of nonwinding seam components in the periodic quotient;
- lift each physical quotient component into one continuous fractional/Cartesian chart;
- simplify the lifted closed surface, preserve its physical topology, and clip it back into the canonical cell;
- protect winding, open, nonmanifold, small, or topology-sensitive components;
- project accepted candidate vertices toward the immutable density level with bounded Newton updates when projection preserves valid triangles;
- validate against the scientific sparse scalar field through corrected sampled surface distance, implicit displacement, gradient-normal degradation, scalar residual, physical topology, and final seam pairing;
- fail structurally when a hard target conflicts with topology or scientific fidelity.

The original four 50% shells fell from 565,482 V1 faces to 226,636 V2 faces. The V3 periodic-quotient amendment removes the previous whole-component seam immutability that made diffuse oxygen shells unnecessarily large.

37.17.11.4 LD9-V3 - Hard scene budgeting and compact Plotly export Status: implemented in `mdstats 0.19.61a0`; this is the normal complete-scene browser-budget layer.

- allocate one deterministic post-replication face budget across all requested fields and HDR shells;
- reserve a minimum per shell and 15% of the scene budget for topology/fidelity-constrained overshoot;
- weight nominal allocation by selected HDR volume proxy and shell visual importance;
- run large shells through fresh extract–simplify–release worker processes;
- apply periodic-quotient simplification to nonwinding seam clouds;
- cast final Plotly coordinates to `float32` and face indices to `int32`;
- omit repeated per-vertex hover metadata and disable mesh hover by default;
- group every atomic path of one species into one line trace without removing frames;
- enforce face, vertex, trace, and final UTF-8 HTML byte limits before writing.

The complete 1,500-frame four-species scene with all twelve 50%, 80%, and 95% shells, trajectories, mean framework, and atomic network contains 286,008 density faces, 147,477 density vertices, 28 Plotly traces, and 26,233,233 HTML bytes. All hard output and scientific-fidelity gates pass. Mesh preparation and scene assembly took 258.597 s on the validation runtime; the earlier raw renderer required roughly 729 s for 3.18 million faces. A supplemental headless Chromium run reached first frame in 14.539 s, 29.71 frames/s scripted orbit, 0.102 s trace toggle, and no context loss. The managed environment exposed no physical WebGL renderer, so production-default acceptance and further wall-time optimization remain LD9-V4 responsibilities.

37.17.11.5 LD9-V4 - Bounded shell execution and browser acceptance Status: execution and functional-browser acceptance implemented in `mdstats 0.19.62a0`; physical-WebGL production-default authorization remains pending external hardware evidence.

- schedule independent large-shell extract–simplify–release jobs through a bounded fresh-process pool;
- constrain every worker to one native numerical thread by default to prevent nested OpenMP/BLAS oversubscription;
- recover deterministic final scene order by stable shell key after asynchronous completion;
- record wall time, summed shell time, maximum shell time, worker count, and parallel efficiency;
- automate Chromium/WebGL smoke tests and record first frame, orbit rate, trace-toggle latency, JavaScript heap, renderer identity, and context loss;
- classify renderer evidence as physical, software, or unavailable;
- distinguish functional browser acceptance from physical-WebGL production-default authorization;
- propagate worker timeout, scientific-fidelity, topology, and browser-budget failures without partial serialization.

A bounded real-mesh three-shell benchmark completed in 3.701 s with three workers versus 9.997 s serial, a 2.701-fold wall-time speedup with 96.9% parallel efficiency and identical geometry counts. The complete self-contained V3 scene passes the functional browser gate at 13.392 s first frame, 27.698 frames/s scripted orbit, 0.119 s trace toggle, approximately 199 MiB JavaScript heap, and no context loss. The managed validation environment exposed no WebGL vendor or renderer string, so physical-GPU authorization is not claimed.

The complete twelve-shell V4 mesh-preparation wall-time gate was not reproducibly rerun in the shared runtime because full scientific-scene reconstruction showed unstable wall time. The V3 complete-scene scientific, fidelity, geometry, and payload evidence remains normative until a stable full-scene rerun and physical-WebGL validation are available.

37.17.11.6 LD13 - Block-grouped packed-field contour reads Status: implemented in `mdstats 0.19.70a0`.

The packed scientific field previously decoded a complete storage-block occupancy bitset once for every queried node. A normal `32^3` contour tile gathers a `33^3` node brick, so near-full Gaussian support blocks caused tens of thousands of repeated Python bitset decodes per tile. This scalar-read path, not Lewiner marching cubes or the final face count, dominated high-resolution sparse-shell rendering.

LD13 preserves the packed-field access contract but groups queries by active storage block. It resolves all periodic block identities vectorially, decodes each touched active block once, searches all requested local indices for that block in one operation, and scatters the values back to query order. The number of occupancy decodes is now the number of distinct touched active blocks rather than the number of queried active nodes.

A fully occupied `33^3` diagnostic tile backed by `16^3` storage blocks fell from 25.7809 s under the historical access path to 0.02135 s under the block-grouped path on the validation runtime, with bit-identical gathered values. This isolated 1207-fold improvement explains the observed order-of-magnitude shell timeout and restores tiled marching-cubes extraction to its intended cost regime.

Shell progress records now include tile count, candidate-cell count, raw faces, final faces, and wall time so future regressions can distinguish scalar-brick access from contour generation and simplification.

37.17.12 LD9 acceptance gates for the all-species stress scene

The implemented V3 hard-output gates are:

- no change to scientific fields, HDR thresholds, or enclosed-mass metadata;
- physical periodic component topology preserved and final canonical seams paired;
- scalar-field distance, residual, implicit-displacement, and normal limits pass under the resolved per-shell display policy;
- no more than **300,000 final serialized density-mesh faces**, after all display replication;
- no more than **200,000 final serialized density-mesh vertices**, after all display replication;
- no more than **64 Plotly traces** for the complete scene;
- no more than **40 MiB final self-contained HTML**.

The face, vertex, trace, and byte limits are hard browser-output limits. If geometric or topology constraints conflict with them, the interactive export fails before writing. The scientific field is never silently degraded.

The earlier 120-second mesh-preparation objective was not met by the first complete V3 implementation: the measured result is 258.597 s. LD9-V4 adds bounded parallel shell scheduling and demonstrates a 2.701-fold speedup on a real three-shell benchmark, but the complete twelve-shell wall-time objective was not reproducibly rerun in the shared runtime. The complete V3 scientific and browser-payload evidence therefore remains normative. Functional browser acceptance passes; production-default authorization still requires physical-WebGL renderer evidence.

37.17.13 LD9 module boundaries

Responsibility	Planned module
render profiles, hard browser budgets, and structured failures	`mdstats.plotting.density_render_budget`
contour-crossing block and tile planning	`mdstats.plotting.density_contour_tiles`
tiled periodic marching cubes and logical-edge ownership	`mdstats.plotting.density_tiled_mesh`
streaming local presimplification and global periodic simplification	`mdstats.plotting.density_mesh_simplify`
scalar-field fidelity, topology, and seam validation	`mdstats.plotting.density_mesh_validation`
scene-wide post-replication budget allocation	`mdstats.plotting.density_scene_budget`

Responsibility	Planned module
compact Plotly trace and grouped-trajectory assembly	`mdstats.plotting.framework_dynamics`
bounded fresh-process shell scheduling	`mdstats.plotting.density_mesh_execution`
browser acceptance policy and renderer classification	`mdstats.plotting.density_browser_acceptance`
Chromium/WebGL release smoke testing	`benchmarks.density_browser_validation`

The simplification strategy is adapted from published polygonal-mesh decimation and quadric-error methods [12, 13]. Periodic seam coupling, density-level fidelity checks, streaming two-stage simplification, and shell-aware hard browser budgets are project-specific.

38 Backend and operator selection policy

The common option records are additive:

```
DensityResolutionOptions(
    grid_interval=0.20,
    gaussian_to_grid_ratio=2.0,
    adaptive_smearing=True,
    broadening_metric="gaussian_sigma_v1",
)

DensityKernelOptions(
    smoothing_operator="discrete_periodized_v1",
    kernel_tail_tolerance=1.0e-8,
)

DensityStorageOptions(
    grid_backend="auto",            # dense | local_sparse | auto
    local_block_shape=(16, 16, 16),
    sparse_activation_fraction=0.20,
)
```

Compatibility is explicit:

Backend	`legacy_spectral_v1`	`discrete_periodized_v1`
`dense`	supported explicitly for compatibility	supported explicitly
`local_sparse`	rejected	supported explicitly for atomic, framework-vertex, and framework-edge fields
`auto`	rejected	supported and default for atomic, framework-vertex, and framework-edge fields

`effective_cic_stencil_rms_v1` is supported explicitly with `discrete_periodized_v1`. Pairing the effective metric with `legacy_spectral_v1` is rejected because the effective covariance is defined by the canonical stencil. Automatic selection is operational only with `discrete_periodized_v1`; pairing `auto` with `legacy_spectral_v1` is rejected. `sparse_activation_fraction` is the localized-field policy anchor used by LD4 and is serialized with every automatic decision.

LD11 is the versioned migration that makes the canonical operator and automatic backend selection the defaults. The broadening metric remains `gaussian_sigma_v1`. Further default changes require another versioned release note and migration record.

39 Normative resource model

39.1 Separation of resource domains

The package distinguishes three resource domains that may not be substituted for one another.

host compute budget package-owned additional memory, native/process threads, and complete-scene wall time available to density preparation and rendering;

algorithmic work estimates input-dependent exact or conservative counts such as samples, nodes, blocks, stencil values, kernel pairs, contour cells, faces, and worker workspaces;

browser-output profile final Plotly faces, vertices, traces, and HTML bytes delivered to the client.

The host budget is derived from the current runtime allocation. Algorithmic counts are computed from the requested scene and compared with that budget. Browser profiles are explicit client constraints and neither increase nor reduce host-compute admission.

39.2 Authoritative runtime budget

A complete scene resolves one immutable `RuntimeResourceBudget` through `FrameworkDynamicsResources`:

```
FrameworkDynamicsResources(
    max_memory_bytes=None,
    max_threads=None,
    max_wall_time_seconds=None,
    memory_fraction=0.80,
    thread_fraction=0.90,
)
```

Let T_{aff} be process CPU affinity, T_{cgroup} a finite cgroup quota, and T_{sched} the most restrictive applicable scheduler CPU declaration. Missing terms are omitted:

$$T_{\mathrm{available}} = \max\bigl(1, \min(T_{\mathrm{aff}}, T_{\mathrm{cgroup}}, T_{\mathrm{sched}})\bigr), \qquad T_{\mathrm{default}} = \max(1, \lfloor 0.9 T_{\mathrm{available}} \rfloor).$$

Memory discovery forms additional-headroom candidates from host available memory, finite cgroup memory minus current cgroup usage, scheduler allocation minus process RSS, and finite `RLIMIT_AS` minus process virtual memory. Missing terms are omitted:

$$M_{\mathrm{available}} = \max\bigl(1, \min(M_{\mathrm{host}}, M_{\mathrm{cgroup}}, M_{\mathrm{sched}}, M_{\mathrm{rlimit}})\bigr), \qquad M_{\mathrm{default}} = \max(1, \lfloor 0.8 M_{\mathrm{available}} \rfloor).$$

The default complete-scene objective is

$$W_{\mathrm{default}} = 1200 \text{ s}.$$

The 20% memory and CPU reserve is a policy headroom for the interpreter, allocator, input trajectory, operating system, concurrent runtime activity, and third-party libraries. It is not calibrated from any trajectory or structure. Explicit user memory and thread values may consume more than the default fraction but are clamped to the detected runtime ceiling. Explicit wall time may be larger or smaller.

The same controls are available through:

```
MDSTATS_MAX_MEMORY_BYTES
MDSTATS_MAX_THREADS
MDSTATS_MAX_WALL_TIME_SECONDS
```

Precedence is explicit API argument, then `MDSTATS_*` environment value, then the runtime-derived default. Every resolved record stores provenance and clamp flags.

39.3 Scene scoping and low-level limits

`density_resource_budget_scope` stores the scene budget in a `ContextVar`. Every nested planner, dense or sparse kernel, cache, contour extractor, and worker scheduler inherits exactly that budget. A nested explicit value can only tighten it:

$$B_{\text{nested}} = \min(B_{\text{requested}}, B_{\text{scene}}).$$

This prevents a second 80% reduction, inconsistent mid-scene host probes, and old serialized low-level records from bypassing a smaller current runtime.

Historical `DEFAULT_MAX_*` compute constants remain importable for compatibility but are not active defaults. A static AST regression test rejects future use of those constants as compute admission. Omitted low-level count, byte, pair, block, cache, workspace, worker, and wall-time limits are derived from the active runtime budget. Explicit legacy values are tightening-only.

The common planner continues to expose compatibility and diagnostic fields such as:

```
max_density_voxels
max_density_samples
max_density_sample_bytes
max_density_planning_bytes
max_density_stencil_values
max_density_nonzero_nodes
max_density_stored_block_values
max_density_blocks
max_density_kernel_pairs
max_density_component_values
max_density_mesh_cells
max_density_mesh_faces
max_density_raw_faces_per_tile
max_density_raw_vertices_per_tile
max_density_transient_mesh_bytes
max_density_render_points
max_density_total_peak_bytes
max_density_threads
max_density_wall_time_seconds
```

These are resolved values, not package-wide fixed capacities. Browser fields such as `max_density_final_faces`, `max_density_final_vertices`, `max_plotly_traces`, and `max_density_final_html_bytes` belong to the separate browser-output profile.

39.4 Input-independent wall-time model

A small synthetic calibration measures fractional transforms, indexed accumulation, Gaussian exponential evaluation, FFT work, scalar crossing-cell scans, and marching cubes when available. Calibration is bounded by the scene thread budget through `threadpoolctl`; no cell shape, atom count, species, trajectory length, or previously successful example enters the model. Conservative throughput fractions and a safety multiplier are used for admission.

For representative counts N_s (samples), N_k (stencil values), N_p (kernel pairs), and N_d (dense nodes), preparation is estimated by

$$\hat{t}_{\text{prep}} = s\left[Ft_F + \frac{N_s}{r_s} + \frac{N_k}{r_k} + \frac{N_p}{r_p} + \frac{N_d}{r_d}\right].$$

For S shells, N_c contour cells, N_f raw faces, P workers, efficiency η, and process-start cost t_0,

$$\hat{t}_{\text{mesh}} = s\left[\frac{St_S + N_c/r_c + N_f/r_f}{\max(1, P\eta)} + \frac{St_0}{P}\right].$$

The complete scene is rejected before allocation when the conservative estimate exceeds the available wall-time objective. Actual stage elapsed time is also checked. Isolated workers receive a timeout no larger than the remaining scene time. Main-process native kernels are preflighted and checked at safe stage boundaries rather than forcibly terminated in a potentially inconsistent state.

39.5 Aggregate memory admission

Individual count checks are early guards only. The authoritative preparation peak is

$$M_{\text{prep,peak}} = \max\left[P + \sum_i R_i, \max_i\left(P + \sum_{j<i} R_j + T_i\right)\right],$$

where P is retained Phase-B planning memory, R_i is retained field memory, and T_i is the transient construction peak for field i in deterministic order.

Rendering separately checks

$$M_{\text{serial,peak}} = M_{\text{parent}} + M_{\text{output}} + M_{\text{serial}},$$

and

$$M_{\text{pool,peak}} = M_{\text{parent}} + M_{\text{output}} + PM_{\text{worker}}.$$

A set of individually valid fields or shells is rejected when its aggregate peak does not fit the scene memory budget.

39.6 Worker containment

The isolated shell-worker count is jointly bounded by shell count, scene threads, and worker-pool memory:

$$P = \min\left[S, \left\lfloor \frac{T_{\max}}{t_w} \right\rfloor, \left\lfloor \frac{M_{\max} - M_{\text{parent}} - M_{\text{output}}}{M_{\text{worker}}} \right\rfloor\right].$$

The default is one native numerical thread per isolated worker. Child processes receive explicit `OMP_NUM_THREADS`, `OPENBLAS_NUM_THREADS`, `MKL_NUM_THREADS`, `BLIS_NUM_THREADS`, `VECLIB_MAXIMUM_THREADS`, `NUMEXPR_NUM_THREADS`, and `MDSTATS_*` resource values. A worker cannot rediscover and consume the full parent allocation.

39.7 Reporting and failure policy

Each field and scene report:

```
runtime snapshot, resolved memory/threads/wall time, and override provenance
logical shape and logical node count
axis intervals and reciprocal-resolution diagnostic
source samples, bytes, and transient group mappings
occupied CIC node count
stencil offsets, image contributions, and bytes
continuous tail bound and stencil normalization diagnostics
nonzero evaluated nodes
active blocks, valid nodes, and allocated block slots
planning structures and planning bytes
kernel accumulation pairs
largest temporary component
scientific HDR thresholds and achieved fractions
actual render levels
mesh cells, vertices, faces, and replicated display geometry
raw field normalization correction
estimated and realized preparation/rendering seconds
```

`estimated and realized peak bytes`
`backend/operator/broadening choices and rationale`

Resource failure is explicit and pre-allocation whenever the required count is known. The package never raises an internal cap until a benchmark passes, changes scientific resolution solely to fit resources, omits requested frames/species/shells, substitutes point rendering for meshes, or treats a browser face budget as host memory.

Phase-A estimates are conservative upper bounds. Phase-B exact index counts may lower them but may not exceed their approved budget. Realized allocations may not exceed the Phase-B transaction.

40 Validation architecture

40.1 Registration and topology

- material mode removes homogeneous strain;
- laboratory trajectories retain homogeneous strain;
- variable-cell laboratory periodic density is rejected;
- framework registration removes common translation from every overlay;
- topology identity and residual winding are invariant across frames;
- integer image choices do not move periodic means or densities;
- trajectories reject ensembles while valid periodic densities accept them.

40.2 Mean framework and atomic net

- node/edge identity is stable;
- projected and atomic-path displays use authoritative paths;
- mean atomic positions match periodic registered sample means;
- periodic-mean convergence and ambiguity diagnostics are deterministic;
- ambiguous means do not silently drive adaptive refinement;
- occupancy thresholds retain/remove bonds correctly;
- gauge changes do not create runaway vertices or bonds.

40.3 Density operators and fields

- CIC deposition conserves source measure within $5 \times 10^{-13} \max(1, M)$;
- `legacy_spectral_v1` remains reproducible under the normative array policy;
- direct and FFT canonical convolution pass LD0-K tolerances;
- dense and sparse canonical fields pass LD1-A tolerances;
- atomic and vertex integrals equal selected counts within the integral tolerance;
- edge integral equals weighted total arc length within the integral tolerance;
- $\sigma = 0$ returns the CIC field;
- every retained periodic image contribution is counted once;
- continuous tail, stencil normalization, and covariance diagnostics are reproducible;
- triclinic kernels are Cartesian-isotropic within the documented discretization benchmark tolerance;
- $h_{\max}$ and $h_{\text{reciprocal}}$ diagnostics are correct;
- both broadening metrics and unresolved targets are honest and versioned;
- HDR thresholds, tie achievements, and render levels are distinct and reproducible.

40.4 Rendering validation

- one Cartesian unit has equal display scale on all axes;
- folded trajectories break at periodic crossings;
- node clouds use i/N_i within 10^{-14} fractional tolerance;
- canonical clipped meshes satisfy the LD2-B incidence and periodic seam criteria;
- sparse meshes contain no duplicate canonical faces or overlong edges;
- winding components follow the deterministic fallback;
- `canonical` density display emits one image;

- `match_graph` emits exactly the approved graph-image shifts and counts all replicated geometry against render budgets;
- legend groups and trace provenance are stable;
- `interactive_browser` counts faces and vertices after all display replication;
- every successful interactive export satisfies its hard final face, vertex, trace, and HTML-byte budgets;
- no oversized HTML is written after a browser-budget failure;
- production fidelity checks evaluate the scientific scalar field without requiring a retained complete raw mesh;
- grouped trajectory traces preserve every selected trajectory point and species;
- automated Chromium/WebGL smoke tests detect context loss, excessive load latency, failed camera motion, and failed trace toggles;
- HTML export does not alter scientific objects.

40.5 Resource and determinism

- Phase A, Phase B, and Phase C occur in order for every scene;
- one fresh runtime snapshot resolves an immutable complete-scene budget;
- default memory is 80% and default threads are 90% of the detected process/job allocation, and the default complete-scene objective is 1,200 seconds;
- nested helpers inherit the exact scene budget and may only tighten it;
- preparation and rendering use aggregate peak-memory and summed wall-time checks;
- browser-output profiles never authorize host-compute work;
- every declared limit has a focused pre-allocation failure test;
- planning arrays, block slots, temporary arrays, replication, retained final mesh bytes, and maximum mutually exclusive transient mesh bytes are counted separately;
- raw per-tile, transient, and final post-replication browser limits are not conflated;
- no scientific degradation is silent;
- stable integer indices, metadata, and serialized records are byte-identical;
- floating fields and meshes satisfy the normative numerical/geometric policy;
- LD4 policy anchors choose dense for broad fields and sparse for qualifying localized fields.

41 Required benchmark systems

1. stationary on-grid and off-grid single-atom fields in an orthorhombic cell;
2. the same cases in the 60-degree LTA primitive cell;
3. face, edge, and corner periodic crossings;
4. support large enough to include multiple periodic images;
5. separated localized framework-atom clouds;
6. overlapping oxygen clouds;
7. multimodal Na hopping occupancy;
8. broad delocalized mobile-ion occupancy;
9. projected framework-edge density;
10. atom-resolved framework-path density;
11. edge quadrature much coarser than an adaptively refined density grid;
12. variable-cell material versus laboratory trajectories;
13. rejection of variable-cell laboratory periodic density;
14. common rigid framework drift;
15. zero and near-zero positional spread;
16. converged, nonconverged, and ambiguous periodic means;
17. orthogonal and skew grids with identical $h_{\max}$ but different $h_{\text{reciprocal}}$;
18. legacy versus canonical operator migration at $\sigma/h = 2, 1, 0.5$;
19. localized boundary-crossing mesh components;
20. a percolating/winding shell component;
21. terminal partial blocks;
22. HDR ties and a render-level adjustment case;
23. canonical versus expanded density-display replication;
24. canonical JSON and read-only-array round trips;

25. Phase-A rejection, Phase-B planning rejection, and scene peak-memory rejection;
26. CIC effective-covariance fixtures at on-node and half-node phases;
27. the 12-shell all-species LTA stress scene under the hard 300,000-face browser profile;
28. a scene where display replication would exceed the face limit unless counted before export;
29. a fidelity-versus-budget conflict that must fail without writing HTML;
30. automated Chromium/WebGL load, orbit, toggle, trace-count, and context-loss validation.

42 Deliberate limitations

Current and near-term architecture does not provide:

- topology-state mixtures in one density field;
- automatic segmentation of reactive trajectories;
- rotational or affine framework alignment;
- time-quadrature weights;
- mixed-boundary convolution;
- variable-cell laboratory periodic density;
- variable-bandwidth KDE;
- covariance ellipsoid glyphs;
- rolling-window or difference densities;
- direct VTK/cube export helpers;
- multilevel AMR;
- browser-side scientific triangulation;
- production LD8 hybrid execution before its evidence and acceptance gates pass;
- production LD9 browser export before its geometric-fidelity, hard-budget, and Chromium/WebGL acceptance gates pass.

These features require separate approved specifications rather than silent approximations.

43 Citation and provenance policy

Code and documentation must distinguish borrowed algorithms, standard mathematical background, and project-specific integrations.

Borrowed methods used by this architecture are:

- cloud-in-cell particle-mesh assignment: Hockney and Eastwood;
- highest-density regions: Hyndman;
- marching cubes: Lorensen and Cline;
- topologically consistent marching-cubes implementation: Lewiner et al.;
- periodic center-of-mass construction: Fréchet and Karcher;
- block-structured patch organization inspiration: Berger and Colella;
- tiled overlap-add linear convolution: Oppenheim, Schafer, and Buck;
- polygonal mesh decimation and quadric-error simplification: Schroeder et al., and Garland and Heckbert.

The following are project-specific integrations unless a more direct source is adopted later:

- topology-compatible periodic gauge normalization;
- registration shared across all scene channels;
- dimensional separation of framework vertex and edge measures;
- sparse CIC node aggregation for these fields;
- metric integer-stencil construction and periodic ownership;
- exact finite-support bitset dilation, periodic terminal-block routing, and target ownership;
- hybrid sparse-direct/metric-aware tiled-FFT selection for periodic density fields;
- periodic seam coupling and density-level fidelity validation during mesh simplification;
- component-local periodic mesh assembly;
- transactional backend selection and scene resource accounting.

44 Implementation source map for `0.19.64a0`

Responsibility	Main implementation module
framework topology to graph view	`mdstats.plotting.framework_topology_graph`
atomic connectivity graph adaptation	`mdstats.plotting.atomic_connectivity_graph`
graph identity/filter/complexity	`mdstats.plotting.graph_view`
periodic materialization	`mdstats.plotting.periodic_graph`
graph styling	`mdstats.plotting.graph_styles`
generic interactive graph rendering	`mdstats.plotting.graph_3d`
registration, mean framework, trajectories, composite scene	`mdstats.plotting.framework_dynamics`
runtime discovery, scene budgets, calibration, and numeric limit derivation	`mdstats.plotting.runtime_resources`
backend-neutral density contracts and dense adapters	`mdstats.plotting.density_contracts`
registration, periodic-mean, spread, and reciprocal-grid diagnostics	`mdstats.plotting.density_diagnostics`
density planning records, limits, approval, and realization checks	`mdstats.plotting.density_planning`
transactional dense-versus-sparse candidate estimation and selection	`mdstats.plotting.density_backend_selection`
canonical periodized stencil construction and dense convolution	`mdstats.plotting.density_kernel`
effective CIC-plus-stencil covariance and RMS diagnostics	`mdstats.plotting.density_broadening`
deterministic sparse CIC and canonical reference convolution	`mdstats.plotting.density_sparse_reference`
optimized sparse CIC/scatter, target planning, and bounded stencil cache	`mdstats.plotting.density_sparse_optimization`
source-independent finite-stencil block routing, terminal validity, and routing cache	`mdstats.plotting.density_block_routing`
packed global CIC sources, exact padded-bitset support atlases, and equivalence audits	`mdstats.plotting.density_support_atlas`
transactional LD8 support-atlas planning and lifted-brick resource bounds	`mdstats.plotting.density_scene_planning`
packed positive scientific-field contract and fixed-block adapter	`mdstats.plotting.density_packed_field`
canonical target-owned direct realization and identity-bound plans	`mdstats.plotting.density_block_direct`
hybrid tiled direct/overlap-add FFT realization and crossover planning	`mdstats.plotting.density_tiled_fft`
multilevel evidence profiling and architecture decision	`mdstats.plotting.density_multilevel_research`
production periodic block-sparse field packing and storage	`mdstats.plotting.density_block_sparse`
periodic sparse candidate cells, lifted components, clipping, seam validation, and winding fallback	`mdstats.plotting.density_sparse_mesh`
bounded render-tile planning and transactional raw/transient extraction limits	`mdstats.plotting.density_contour_tiles`
tiled Lewiner marching cubes, logical-edge indexing, and tile-local release	`mdstats.plotting.density_tiled_mesh`
tile-local and global periodic QEM simplification with implicit-field validation	`mdstats.plotting.density_mesh_simplify`
hard post-replication interactive-browser accounting and structured budget failure	`mdstats.plotting.density_render_budget`
deterministic scene-wide shell allocation and reserved browser-face apportionment	`mdstats.plotting.density_scene_budget`
isolated extract–simplify–release shell worker	`mdstats.plotting._density_mesh_worker`

Responsibility	Main implementation module
bounded fresh-process shell scheduling and execution reports	`mdstats.plotting.density_mesh_execution`
browser acceptance policy, renderer classification, and reports	`mdstats.plotting.density_browser_acceptance`
deterministic mesh topology and sampled geometric-fidelity validation	`mdstats.plotting.density_mesh_validation`
atomic density numerics and mesh helpers	`mdstats.plotting.atomic_density`
framework vertex/edge density	`mdstats.plotting.framework_density`

45 Recommended next implementation

LD0-R1 is implemented in `mdstats 0.19.40a0`. It delivers the four shared option records, unified weighted samples, structured provenance, recursively frozen metadata, backend-neutral field/node protocols, storage summaries, zero-copy dense adaptation, reserved identifier validation, and focused compatibility tests.

LD0-R2 is implemented in `mdstats 0.19.41a0`. It delivers exact laboratory-cell equivalence validation for periodic densities, deterministic multi-start Fréchet/Karcher mean diagnostics, validity-filtered spread references, `numpy.quantile(method="linear")`, explicit zero-spread handling, the certified reciprocal-resolution diagnostic, atomic-mean-graph diagnostics, and logical-node voxel-cloud coordinates. The legacy dense CIC plus `legacy_spectral_v1` numerical path is unchanged.

LD0-R3 is implemented in `mdstats 0.19.42a0`. It delivers metadata-only Phase-A bounds, exact bounded Phase-B CIC index plans, scene-wide Phase-C approval, package-owned peak-memory accounting, immutable planning records, realization checks, and pre-allocation failures for every declared hard resource limit. No floating density field is constructed before the complete requested density scene is approved.

LD0-K is implemented in `mdstats 0.19.43a0`. It delivers the canonical `discrete_periodized_v1` stencil, deterministic metric support enumeration, the zero-bandwidth identity path, direct and FFT dense convolution of the same stored weights, kernel covariance and normalization diagnostics, scene planning for dense stencil storage, and explicit legacy compatibility. `legacy_spectral_v1` remains the default and is numerically unchanged.

LD0-B is implemented in `mdstats 0.19.44a0`. It delivers analytic weighted CIC phase covariance, covariance-only canonical-stencil moments, `effective_cic_stencil_rms_v1`, deterministic bounded effective-width refinement, explicit/budget-limited/zero-spread policies, atomic and framework provenance, and versioned migration while retaining `gaussian_sigma_v1` as the default.

LD1-A is implemented in `mdstats 0.19.45a0`. It delivers sparse-only canonical stencil support, deterministic periodic CIC node aggregation, stencil-major sparse scatter, exact final measure normalization, auditable HDR tie details, guarded dense debugging conversion, public flat-node access, and a retained simple oracle for later optimized sparse paths.

LD1-B is implemented in `mdstats 0.19.46a0`. It delivers deterministic periodic block packing, terminal-block validity masks, atomic index/species sparse preparation, public node access, canonical serialization, exact sparse Phase-B planning and realization accounting, and the required localized storage reduction.

LD2-A is implemented in `mdstats 0.19.47a0`. It delivers backend-neutral HDR details, deterministic two-pass logical-node cloud selection, exact Cartesian node coordinates and bounds, resource summaries, trace-indexed provenance, atomic local-sparse voxel-cloud rendering without dense materialization, and deterministic expanded-cell cloud replication.

LD2-B is implemented in `mdstats 0.19.48a0`. It delivers threshold-local logical-cell ownership, deterministic periodic face components, lifted charts and torus-winding detection, cell-aware Lewiner contouring, whole-triangle canonical clipping, vertex and face canonicalization, periodic seam validation, exact render-resource metadata, expanded-cell mesh replication, and deterministic dense-or-cloud fallback for winding components.

LD3 is implemented in `mdstats 0.19.49a0`. It extends exact block-sparse planning, preparation, provenance, HDR, logical-node clouds, and periodic meshes to framework-vertex occupancy and projected or atom-resolved framework-edge arc length. It adds deterministic resolution-aware midpoint quadrature, exact segment-weight normalization, sparse endpoint-orientation canonicalization, and separate occupancy/arc-length units.

LD4 is implemented in `mdstats 0.19.50a0`. It adds exact dense and sparse candidate planning for every atomic and framework field, deterministic broad/localized policy anchors, whole-scene resource-aware combination selection, canonical selection records, and explicit global-override provenance. Auto mode preserves the resolved scientific grid, Gaussian, broadening, kernel, and quadrature decisions.

LD5 is implemented in `mdstats 0.19.51a0`. It adds vectorized and bounded sparse evaluation, exact optimized target planning, immutable canonical-support caching, explicit reference-mode forcing, and benchmark evidence while preserving LD4 selection semantics and the LD1-A oracle.

LD6 is implemented in `mdstats 0.19.52a0`. It adds phase-robust dyadic coarse/fine research profiling, alternative single-level block-shape comparisons, conservative mass-preserving surrogate reconstruction, immutable evidence records, and the explicit `retain_single_level` architecture decision.

LD7 is implemented in `mdstats 0.19.53a0`. It adds deterministic temporal-stratified spread estimation, bounded two-pass block streaming, exact source-group batching and merging, and execution-consistent transactional planning.

LD8-P0 is implemented in `mdstats 0.19.54a0`. It adds the full-frame production-`1.0e-8` planning benchmark, effective-CIC resolution evidence, 8/16/32 block occupancy and storage profiles, bounded direct/FFT executor spikes, optional current-LD7 execution, and auditable failure evidence. The evidence stage does not migrate the production backend.

LD8-S0 through LD8-S4 are implemented through `mdstats 0.19.58a0`. S0/S1 add immutable source-independent block routing, exact terminal validity classes, normative packed local bitsets, one global packed CIC source per field, transactional support planning, exact finite-support atlases, canonical identities, and packed scientific fields. S2 adds the bounded exact target-owned direct migration oracle. S3 adds deterministic compute tiles, mixed bounded direct and metric-aware overlap-add FFT execution, exact packed-target accumulation, supported-node repair, and selector provenance. S4 makes the hybrid executor the normal local-sparse production path, adds certified binary-FFT support dilation for production-size stencils, narrowly scoped LD7 fallback, shared exact multi-HDR ordering, lazy contour support, and the passing four-species production gate.

LD9-V0 is implemented in `mdstats 0.19.54a0`. It adds immutable hard browser-budget contracts, structured pre-serialization failure, deterministic mesh topology summaries, sampled geometric-fidelity metrics, Chromium/WebGL smoke tooling, and stress-scene budget calibration. The saved 12-shell stress artifact contains 3,184,902 density faces, 1,599,109 density vertices, 173 Plotly traces, and 177.70 MiB of self-contained HTML. Relative to the initial `interactive_browser` profile, it requires at least 10.616x face reduction, 7.996x vertex reduction, 4.443x HTML reduction, and trace grouping below 64 traces. Managed Chromium blocked local navigation for the raw artifact, and the artifact exceeded the bounded direct-injection safety limit, so V0 records a structured environment/payload failure rather than an unsupported browser-performance claim.

LD9-V1 is implemented in `mdstats 0.19.59a0`. It adds exact high-node crossing-cell pruning, deterministic bounded render-tile plans, one Lewiner marching-cubes call per nonempty tile, logical-edge indexed vertices, canonical periodic seam copies, clipping fast paths, and tile-local raw-workspace release. Four saved full-resolution 50% HDR shells reduced 283,531 potential cell-wise marching-cubes calls to 694 tile calls.

LD9-V2 is implemented in `mdstats 0.19.60a0`. It adds tile-local presimplification, component-wise QEM reduction, topology checks, bounded implicit-level projection, and scientific-field fidelity validation. `mdstats 0.19.61a0` extends this layer with periodic-quotient reconstruction and lifted simplification for nonwinding seam components.

LD9-V3 is implemented in `mdstats 0.19.61a0`. It adds deterministic post-replication allocation, periodic-quotient seam simplification, process-isolated shell preparation, compact Plotly geometry, grouped trajectories, and hard pre-write browser limits. The complete twelve-shell stress scene passes at 286,008 faces, 147,477 vertices, 28 traces, and 26,233,233 HTML bytes.

LD9-V4 is implemented in `mdstats 0.19.62a0` at the execution and functional-browser layer. It adds bounded parallel fresh-process shell preparation, native-thread containment, deterministic execution reports, browser acceptance policies, renderer classification, and separate functional versus physical-WebGL production authorization. A real three-shell benchmark records a 2.701-fold wall-time speedup with identical geometry counts. The complete V3 scene passes the functional browser gate, but physical-WebGL production-default authorization remains pending because the managed environment exposed no renderer identity.

LD10 is implemented in `mdstats 0.19.64a0`. It replaces benchmark-fitted host-compute caps with one runtime-derived scene budget, defaults to 80% of detected memory and 90% of detected CPUs with a 1,200-second scene

objective, makes legacy low-level limits tightening-only, adds input-independent throughput calibration and aggregate peak admission, and propagates bounded resources into isolated mesh workers. Browser-output profiles remain independent.

No production multilevel implementation is authorized. Multilevel work may be reopened only by new representative evidence satisfying the LD6 adoption policy. LD8-P0 and LD8-S0 through LD8-S4, LD9-V0 through LD9-V4, and LD10 are implemented. Scientific density semantics and explicit browser-output profiles remain unchanged; host-compute admission is runtime-derived. Physical-WebGL production-default certification remains an external acceptance task.

46 Cross-cutting structured progress port

Long-running framework, density, and rendering operations expose a shared observability boundary owned by `mdstats/progress.py`. Numerical modules emit immutable `ProgressEvent` records through a structural `ProgressPort`; they do not print, configure global logging, or depend on one command-line wrapper.

The package boundary is

numerical module → `ProgressEvent` → `ProgressPort` → application-selected presentation.

The normative public keyword is:

```
progress: ProgressPortLike | None = None
```

The former

```
progress_callback: Callable[[str], None] | None
```

remains a deprecated compatibility alias. Both may not be supplied together.

Every event carries:

```
source
stage
message
status
current / total / unit, when a finite count exists
small scalar metadata
schema_version = mdstats.progress-event.v1
```

`source` and `stage` are stable machine-readable identifiers. Human-readable message text may evolve. Scientific arrays, trajectories, meshes, and mutable planning records are forbidden in event metadata.

A public module resolves the port once and passes the same resolved object into nested modules. This preserves one sink and, for `TextProgressPort`, one elapsed-time origin. Nested modules create their own `ProgressEmitter` with a module-specific source.

Current sources are:

```
plotting.framework_dynamics.prepare
plotting.atomic_density
plotting.framework_density
plotting.framework_dynamics.render
examples.lta_density
```

Current count-bearing stages include framework registration by frame, atomic density realization by field, framework-density realization by channel, and isosurface extraction by shell. Fine-grained Gaussian pairs, grid nodes, and triangle operations remain unreported because event overhead and terminal volume would distort execution.

Built-in consumers are:

- `NullProgressPort` for silent default operation;
- `TextProgressPort` for stdout or stderr;
- `LoggingProgressPort` for a caller-owned standard-library logger;

- `CallbackProgressPort` for GUI, notebook, test, or orchestration callbacks.

Computational modules never configure logger handlers or levels. Worker subprocesses currently report through the parent coordinator after future completion rather than serializing UI ports into workers.

Future expensive modules should open the same keyword-only port, use stable snake-case stages, emit stage boundaries and coarse `X/Y` updates where a real total exists, and preserve identical scientific behavior when no port is supplied. The normative API and adoption checklist are defined in `docs/specs/progress_spec.{md,pdf}`.

47 Revision record

47.1 2026-07-22 - Package-wide progress-port abstraction

- Added `mdstats/progress.py` with the immutable `ProgressEvent` schema, `ProgressPort` protocol, module-side `ProgressEmitter`, and null, text, logging, callback, and legacy adapters.
- Standardized the keyword-only `progress=` port for long-running framework-dynamics, atomic-density, framework-density, and interactive-mesh operations.
- Preserved `progress_callback=` as a deprecated string-callback compatibility path.
- Migrated the LTA examples from a private reporter class to the package port.
- Kept modules silent by default and retained environment-triggered stderr reporting.

47.2 2026-07-21 - LD10 runtime-derived resource policy

- Replaced scene-fitted memory, count, pair, block, cache, workspace, worker, and wall-time defaults with one runtime-derived complete-scene budget.
- Added fresh CPU-affinity, cgroup, scheduler, host-memory, process-memory, and finite address-space discovery.
- Defaulted to 80% of detected memory and 90% of detected CPUs and a 1,200-second complete-scene objective, with API, environment, and example-CLI overrides.
- Added context-local exact budget inheritance, tightening-only legacy controls, input-independent synthetic throughput calibration, and aggregate preparation/rendering peak admission.
- Limited isolated shell workers jointly by CPU, memory, shell count, and remaining wall time, and propagated child/native-library resource bounds.
- Kept browser final-face, vertex, trace, and HTML profiles separate from host-compute admission.
- Added focused dynamic and static regression tests preventing restoration of historical fixed compute defaults.

47.3 2026-07-21 - LD9-V4 bounded shell execution and browser acceptance

- Added bounded parallel fresh-process execution for independent large density shells.
- Added one-thread native-library containment for each worker and per-shell timeout propagation.
- Added immutable execution options and reports with wall time, summed shell time, maximum shell time, worker count, and parallel efficiency.
- Added browser acceptance policies, physical/software/unavailable renderer classification, and separate functional versus production-default reports.
- Recorded a real three-shell wall-time reduction from 9.997 s to 3.701 s with three workers, 2.701-fold speedup, 96.9% parallel efficiency, and identical geometry counts.
- Revalidated the complete self-contained V3 scene functionally at 13.392 s first frame, 27.698 frames/s orbit, 0.119 s toggle, approximately 199 MiB JavaScript heap, and no context loss.
- Retained physical-WebGL production-default authorization as pending because the managed validation environment exposed no renderer identity.
- Retained the complete V3 scientific, fidelity, face, vertex, trace, and HTML evidence as normative because the full twelve-shell V4 wall-time gate was not reproducibly rerun in the shared runtime.

47.4 2026-07-21 - LD9-V3 hard-budget browser density scenes

- Added deterministic post-replication shell allocation with minimum reserves, visual-importance weighting, and a 15% topology/fidelity reserve.
- Added periodic-quotient reconstruction, continuous-chart lifting, QEM reduction, recanonicalization, and final seam-pair validation for nonwinding seam clouds.

- Added fresh-process extract–simplify–release shell execution.
- Added compact Plotly dtypes, omitted redundant hover payload, and grouped trajectory lines by species.
- Added hard pre-write limits of 300,000 density faces, 200,000 density vertices, 64 traces, and 40 MiB self-contained HTML.
- Passed the complete twelve-shell stress gate at 286,008 faces, 147,477 vertices, 28 traces, and 26,233,233 bytes.
- Recorded 258.597 s mesh preparation and scene assembly.
- Recorded supplemental headless Chromium evidence: 14.539 s first frame, 29.71 frames/s orbit, 0.102 s toggle, and no context loss; physical-WebGL production acceptance remains LD9-V4 work.

47.5 2026-07-21 - LD9-V2 periodic fidelity-constrained simplification

- Added bounded tile-local presimplification for closed components wholly inside one render tile while copying all tile-boundary and seam geometry exactly.
- Added global connected-component target allocation and QEM reduction adapted from Garland and Heckbert.
- Initially protected seam-touching components; LD9-V3 supersedes whole-component seam protection with periodic-quotient simplification for nonwinding physical clouds.
- Added bounded projection toward the immutable density contour and fallback to unprojected topology-valid geometry when projection degenerates triangles.
- Added scalar-field trilinear sampling, corrected sampled surface-distance metrics, implicit-displacement limits, gradient-normal degradation, scalar residual, topology, and seam validation.
- Added hard-target structured failure, calibration-mode achieved counts, canonical JSON round trips, and V1-to-V2 schema migration.
- Reduced the four saved 50% HDR stress shells from 565,482 to 226,636 faces in 50.440 s with maximum species peak RSS 0.905 GiB and all fidelity gates passing.
- Established the V2 baseline later integrated by LD9-V3 hard post-replication scene budgeting and compact Plotly export.

47.6 2026-07-21 - LD9-V1 bounded tiled contour extraction

- Replaced the normal nonwinding per-cell contour extractor with bounded deterministic render tiles.
- Added exact high-node crossing-cell pruning, partial terminal tiles, one Lewiner marching-cubes call per tile, logical-edge indexed vertices, canonical seam copies, and clipping fast paths.
- Added raw per-tile, total-work, planning, and transient-memory preflight plus tile-local scalar/raw-array release.
- Retained strict periodic topology validation, winding fallbacks, the legacy cell-wise oracle, and unchanged scientific fields and HDR thresholds.
- Recorded 694 tile calls for 283,531 crossing cells across the four 50% stress shells; the unsimplified 565,474-face result confirms that LD9-V2 remains required.
- Advanced the next implementation stage to LD9-V2 periodic fidelity-constrained simplification.

47.7 2026-07-21 - LD8-S4 production integration

- Migrated normal local-sparse atomic and framework density dispatch to the S3 hybrid executor and packed fields.
- Added narrowly scoped LD7 fallback for declared resource failures only.
- Added certified binary-FFT support dilation for production stencils and retained bitset dilation as the exact small-case oracle.
- Added shared exact multi-HDR ordering, lazy contour-support planning, and packed-field query methods.
- Completed the full 1,500-frame Na/Si/Al/O gate in 80.515 s aggregate scientific time, 4.219x faster than LD7, with no fallback and peak channel RSS below 1.218 GiB.
- Authorized LD8 production migration and advanced the next implementation stage to LD9-V1.

47.8 2026-07-21 - LD8-S3 implementation

- Added immutable hybrid-executor options, hard realization limits, per-tile plans, and identity-bound whole-field plans.
- Added deterministic compute-tile partitioning of the global packed CIC source.

- Added bounded sparse-direct tile realization and zero-padded three-dimensional FFT linear convolution with periodic overlap-add.
- Added calibrated direct/FFT crossover selection, explicit executor forcing, byte-bounded kernel-spectrum caching, and exact packed-target lookup.
- Added exact direct repair of nonpositive FFT boundary nodes, one final normalization, and complete executor provenance.
- Added fragmented, compact, boundary-crossing, oxygen-heavy, terminal-block, periodic, cache, resource, and S2-equivalence tests.
- Recorded 2.81–40.44x focused speedups over S2 and 20.08–26.91x value-realization speedups over the recorded LD7 Na/Si/Al field baselines.
- Retained LD7 as production dispatch and advanced the next numerical stage to LD8-S4; LD9-V1 remains the next rendering stage.

47.9 2026-07-21 - LD8-S2 implementation

- Added immutable direct-realization limits and identity-bound execution plans.
- Added deterministic target-owned block convolution over the packed CIC source, exact finite stencil, routing template, and support atlas.
- Added conservative source-target stencil pruning, bounded vectorized offset/source chunks, and hard preflight limits.
- Added preallocated immediate packed output, exact support-bitset verification, one final normalization, and deterministic residual correction.
- Recorded bounded production-stencil equivalence to LD1-A and retained LD7 as production because the canonical S2 oracle is not the accelerator.
- Advanced the next numerical stage to LD8-S3; LD9-V1 remains the next rendering stage.

47.10 2026-07-21 - LD8-S0 and LD8-S1 implementation

- Added immutable source-independent periodic kernel block routing with exact stencil identities, terminal extent classes, validity bitsets, byte-bounded cache ownership, and canonical JSON.
- Added one field-specific packed global CIC source with occupancy bitsets and packed positive masses.
- Added transactional support-atlas planning with target/edge bounds, complete-pair reference counts, lifted-brick workspace bounds, and preallocation failure.
- Added exact source-block padded-bitset support dilation without complete fine-pair arrays or source-specific global cache reuse.
- Added field-specific target support bitsets, source-to-target CSR metadata, optional periodic block components, explicit modular-Minkowski-sum verification, and canonical identities.
- Added the packed positive scientific-field contract and fixed-block compatibility adapter; production target-owned realization remains pending for LD8-S2.
- Recorded all-frame `1.0e-8` Na/Si/Al/O support evidence with exact target-node agreement against LD7.
- Kept the production scientific executor and renderer unchanged; the next stages are LD8-S2 and LD9-V1.

47.11 2026-07-21 - LD8-P0 and LD9-V0 implementation

- Implemented full-frame `1.0e-8` production-cutoff planning and optional current-LD7 execution benchmarks.
- Recorded effective-CIC refined grids, exact stencil sizes, source occupancy, block profiles, packed-storage estimates, and bounded direct/FFT agreement.
- Implemented hard post-replication browser face, vertex, trace, and HTML-byte contracts with structured pre-write failure.
- Implemented deterministic indexed-mesh topology summaries and sampled distance, normal, and scalar-residual fidelity reports.
- Added Chromium/WebGL smoke-validation and raw stress-scene calibration tools.
- Corrected LD7 sparse batch normalization by applying floating residuals to the largest positive node.
- Kept the scientific backend and browser renderer unchanged; LD8-S0/S1 and LD9-V1 remain pending.

47.12 2026-07-21 - Hard browser-budget completion of LD9

- Made the final density face count a mandatory post-replication hard limit for the `interactive_browser` profile.

- Split raw per-tile, transient-memory, and final browser-payload limits.
- Added `BrowserMeshBudget`, explicit render profiles, and structured `BrowserMeshBudgetFailure` semantics that prohibit oversized HTML output.
- Added transactionally allocated scene-wide shell/component face budgets.
- Added bounded tile extraction, streaming local presimplification, deterministic logical-edge merging, and global seam-aware simplification.
- Required final validation directly against the scientific scalar field so production does not retain complete raw reference meshes.
- Added hard face, vertex, trace, and HTML-byte acceptance gates and counts after all display replication.
- Made compact Plotly arrays and species-grouped trajectories normative.
- Added automated Chromium/WebGL load, orbit, toggle, memory, and context-loss gates.
- Added LD9-V4 as the production-default authorization gate.

47.13 2026-07-21 - Rigorous LD8 revision and LD9 visualization plan

- Retained `kernel_tail_tolerance=1.0e-8` and all canonical estimator invariants.
- Added LD8-P0 to benchmark the actual production cutoff and calibrate direct/FFT execution.
- Corrected cache ownership by separating reusable stencil/routing geometry from field-specific source support.
- Made packed bitset support dilation, exact terminal-block routing, one global packed CIC source field, and packed scientific output normative.
- Forbade routing maps proportional to the complete local-node/stencil Cartesian product.
- Elevated hybrid target-owned direct and tiled overlap-add FFT execution into the core LD8 plan.
- Replaced default global HDR sorting with exact weighted multi-selection.
- Clarified peak-memory accounting and canonical-versus-optimized reproducibility contracts.
- Added the initial LD9 plan for tiled contour extraction, periodic seam-aware quadric-error simplification, compact Plotly serialization, and geometric-fidelity gates; the hard browser-output contract was completed in the subsequent revision.
- Kept multilevel AMR, variable bandwidth, and support loosening unauthorized.

47.14 2026-07-21 - Initial LD8 finite-support refinement draft (superseded)

- Introduced the initial periodic block-atlas concept and one-global-source-field direction.
- Superseded by the rigorous split-cache, bitset, packed-field, hybrid-executor plan above.

47.15 2026-07-20 - LD6 research-gate completion

- Marked LD6 completed in `mdstats 0.19.52a0`.
- Added `density_multilevel_research.py` with bounded phase-robust profiling, alternative block-shape plans, optimistic conservative coarse/fine surrogates, canonical evidence records, and explicit decision logic.
- Added representative atomic, framework-vertex, framework-edge, path, and broad-field evidence benchmarks.
- Recorded the normative `retain_single_level` decision and closed the staged adaptive-density roadmap without authorizing production multilevel AMR.
- Preserved all production density fields, backend decisions, meshes, and scene schemas unchanged.

47.16 2026-07-20 - LD0-R1 implementation update

- Marked LD0-R1 implemented in `mdstats 0.19.40a0`.
- Added `density_contracts.py` as the renderer-independent contract boundary.
- Added the four shared option records, unified weighted samples, structured source provenance, frozen JSON metadata, field/node protocols, storage summaries, and zero-copy dense adapters.
- Preserved the `legacy_spectral_v1` dense numerical path and rejected identifiers owned by later gates.
- Advanced the recommended next gate to LD0-R2.

This revision completes the second pre-implementation audit. In addition to the previous corrections, it:

- unifies the weighted-sample model and separates transient group IDs from persistent provenance;
- replaces the undefined stencil-normalization claim with auditable stencil normalization diagnostics;
- moves kernel-tail controls out of storage options;

- renames the skew-grid measure to $h_{\text{reciprocal}}$ and limits its meaning;
- introduces the effective CIC-plus-stencil artificial-width metric as a versioned migration;
- defines a public backend-neutral node-access capability;
- makes planning a bounded Phase-A/Phase-B/Phase-C transaction;
- adds sample, planning, stencil, and scene peak-memory limits;
- provides an explicit backend/operator compatibility matrix;
- replaces qualitative gate language with executable numerical and geometric tolerances;
- removes unimplementable post hoc triangle ownership from black-box marching cubes;
- distinguishes torus closure from Euclidean watertightness after canonical clipping;
- defines canonical versus graph-matched density display replication;
- requires deep immutability and canonical schema-versioned serialization;
- fixes cell-equivalence, periodic-mean, quantile, and tail-tolerance policies;
- corrects the Lorensen-Cline DOI;
- splits the former LD0-R into LD0-R1, LD0-R2, and LD0-R3 and adds LD0-B.

47.17 2026-07-20 - LD0-R2 implementation update

- Marked LD0-R2 implemented in `mdstats 0.19.41a0`.
- Added `density_diagnostics.py` for laboratory-cell equivalence, deterministic multi-start periodic means, validity-filtered positional spreads, and certified reciprocal sampling-vector diagnostics.
- Rejected variable-cell laboratory periodic density while retaining laboratory trajectories and mean geometry.
- Added explicit zero-spread and insufficient-valid-reference behavior.
- Corrected diagnostic density-cloud coordinates from $(i + 1/2)/N_i$ to i/N_i.
- Added atomic/framework field metadata and atomic-mean-graph diagnostics.
- Preserved dense CIC, `legacy_spectral_v1`, normalization, HDR, and mesh numerics.
- Advanced the recommended next gate to LD0-R3.

47.18 2026-07-20 - LD0-R3 implementation update

- Marked LD0-R3 implemented in `mdstats 0.19.42a0`.
- Added `density_planning.py` with immutable Phase-A, Phase-B, and scene-plan records.
- Added exact bounded CIC target-node planning without dense scalar allocation.
- Added scene-wide sample, planning, storage, mesh-bound, and peak-byte approval.
- Attached the approved plan and realization summary to framework-dynamics scenes.
- Preserved dense CIC, `legacy_spectral_v1`, normalization, HDR, and mesh numerics.
- Advanced the recommended next gate to LD0-K.

47.19 2026-07-20 - LD0-K implementation update

- Marked LD0-K implemented in `mdstats 0.19.43a0`.
- Added `density_kernel.py` with the immutable `PeriodicGaussianStencil` record.
- Added deterministic integer-image support enumeration in the exact Cartesian metric and canonical periodic aggregation.
- Added direct and FFT circular convolution of one stored discrete stencil and the explicit $\sigma = 0$ identity path.
- Added cutoff, normalization, image-count, active-offset, covariance, roundoff, and post-convolution diagnostics.
- Enabled `discrete_periodized_v1` for dense atomic, framework-vertex, and framework-edge fields while preserving `legacy_spectral_v1` as the default.
- Added dense-stencil counts and mixed-operator provenance to transactional scene planning.
- Verified exact legacy compatibility and quantified canonical-versus-legacy migration differences.
- Advanced the recommended next gate to LD0-B.

47.20 2026-07-20 - LD0-B implementation update

- Marked LD0-B implemented in `mdstats 0.19.44a0`.
- Added `density_broadening.py` with weighted periodic CIC phase covariance, canonical-stencil covariance composition, and immutable diagnostics.
- Added covariance-only canonical-stencil moments without dense logical-grid allocation.

- Enabled explicit `effective_cic_stencil_rms_v1` only with `discrete_periodized_v1`; the legacy metric/operator defaults remain unchanged.
- Added deterministic bounded effective-width refinement and honest explicit, budget-limited, zero-spread, and zero-bandwidth policies.
- Added atomic, framework-vertex, and edge-quadrature broadening metadata and Phase-B planning provenance.
- Advanced the recommended next gate to LD1-A.

47.21 2026-07-20 - LD1-A implementation update

- Marked LD1-A implemented in `mdstats 0.19.45a0`.
- Added `PeriodicGaussianStencilSupport` and sparse-only canonical support construction without logical dense-stencil allocation.
- Added `density_sparse_reference.py` with deterministic periodic CIC aggregation, stencil-major sparse canonical scatter, final normalization, HDR tie diagnostics, flat-node public access, and bounded dense debugging conversion.
- Added explicit CIC-contribution, stencil-candidate, kernel-pair, workspace, and dense-debug resource limits.
- Retained the dense legacy and canonical production paths unchanged and kept `grid_backend="local_sparse"` unavailable through production plotting options.
- Advanced the recommended next gate to LD1-B.

47.22 2026-07-20 - LD1-B implementation update

- Marked LD1-B implemented in `mdstats 0.19.46a0`.
- Added `density_block_sparse.py` with deterministic block packing, partial-terminal masks, public periodic node access, guarded dense conversion, and canonical JSON serialization.
- Enabled explicit atomic `local_sparse` preparation with `discrete_periodized_v1`, while retaining dense as the default and rejecting sparse rendering until LD2.
- Added exact sparse target-node and block-count planning before scalar block allocation, then integrated those counts with scene approval and realization checks.
- Preserved framework sparse channels for LD3 and automatic backend selection for LD4.
- Verified exact LD1-A reference equivalence, the localized storage gate, and exact dense-default compatibility with `mdstats 0.19.45a0`.
- Advanced the recommended next gate to LD2-A.

47.23 2026-07-20 - LD2-A implementation update

- Marked LD2-A implemented in `mdstats 0.19.47a0`.
- Added `density_node_cloud.py` with backend-neutral HDR details, deterministic two-pass logical-node selection, exact Cartesian coordinates, bounds, resource summaries, and canonical serialization.
- Enabled atomic `local_sparse` fields in `render_mode="voxel_cloud"` without dense materialization while retaining sparse mesh rejection until LD2-B.
- Added trace-indexed scientific provenance and exact cloud-resource metadata to framework-dynamics render results.
- Added deterministic `match_graph` replication for node clouds from expanded periodic primary-cell shifts without recomputing density.
- Preserved the historical canonical dense cloud selection and intensity policy and exact dense scientific compatibility with `mdstats 0.19.46a0`.
- Advanced the recommended next gate to LD2-B.

47.24 2026-07-20 - LD2-B implementation update

- Marked LD2-B implemented in `mdstats 0.19.48a0`.
- Added `density_sparse_mesh.py` with positive-level candidate-cell discovery, deterministic face-connected components, lifted charts, and nonzero torus-winding diagnostics.
- Added cell-aware per-owned-cell Lewiner contouring with shared-edge interpolation recomputed from common endpoint values.

- Added whole-triangle image replication and Sutherland-Hodgman clipping against the canonical fractional cell; independent vertex wrapping remains forbidden.
- Added deterministic vertex/face canonicalization, duplicate and degenerate removal, periodic seam pairing, incidence checks, and logical-cell edge-length bounds.
- Added dense canonical and node-cloud fallbacks for winding components, exact mesh resource/topology metadata, canonical JSON mesh records, and expanded-cell `match_graph` mesh replication.
- Preserved dense scientific fields and historical dense canonical mesh output; framework sparse fields remain reserved for LD3.
- Advanced the recommended next gate to LD3.

47.25 2026-07-20 - LD3 implementation update

- Marked LD3 implemented in `mdstats 0.19.49a0`.
- Enabled `grid_backend="local_sparse"` for framework-vertex occupancy and projected or atom-resolved framework-edge arc-length density with `discrete_periodized_v1`.
- Added deterministic resolution-aware midpoint edge quadrature with explicit `auto`/`explicit` modes, configurable refinement depth, exact segment-weight correction, and orientation-invariant endpoint canonicalization.
- Added structured framework vertex/edge provenance, separate occupancy and arc-length units, exact sparse Phase-B plans, and realization accounting.
- Reused the LD2 backend-neutral cloud and periodic mesh renderers for both framework channels without dense materialization.
- Advanced the framework-dynamics scene schema to `mdstats.framework-dynamics-scene.v11` and the recommended next gate to LD4.

47.26 2026-07-20 - LD4 implementation update

- Marked LD4 implemented in `mdstats 0.19.50a0`.
- Added `density_backend_selection.py` with schema-versioned dense/sparse candidate estimates, selection records, canonical JSON round trips, and deterministic policy anchors.
- Enabled `grid_backend="auto"` for atomic, framework-vertex, and framework-edge fields with `discrete_periodized_v1`.
- Added exact pre-allocation dense and sparse Phase-B candidates, whole-scene combination approval, explicit global-resource overrides, and realized-field provenance.
- Preserved the requested scientific resolution and all forced dense/sparse behavior; dense remains the default and legacy-spectral auto selection remains rejected.
- Advanced the framework-dynamics scene schema to `mdstats.framework-dynamics-scene.v12` and the recommended next gate to LD5.

47.27 2026-07-20 - LD5 implementation update

- Marked LD5 implemented in `mdstats 0.19.51a0`.
- Added `density_sparse_optimization.py` with vectorized preallocated CIC, chunked canonical scatter, exact optimized target planning, bounded dense/sparse reduction, and a thread-safe immutable stencil-support LRU cache.
- Added `DensityOptimizationOptions`, cache inspection/clear APIs, explicit optimized/reference forcing, and operational cache provenance.
- Preserved the LD1-A reference path, exact active-node and block identities, LD4 selection semantics, and explicit dense compatibility.
- Advanced the framework-dynamics scene schema to `mdstats.framework-dynamics-scene.v13` and the recommended next gate to LD6.

47.28 2026-07-21 - LD11 implementation update

- Marked LD11 implemented in `mdstats 0.19.65a0`.
- Changed `DensityKernelOptions()` to default to `discrete_periodized_v1`.
- Changed `DensityStorageOptions()` to default to `grid_backend="auto"`.
- Made default atomic and framework density planning resolve the physical grid and Gaussian bandwidth before evaluating dense and local-sparse candidates.

- Prohibited dense voxel limits from silently broadening automatic fields when sparse storage can realize the requested resolution.
- Retained explicit dense, explicit local-sparse, and legacy-spectral dense behavior as reproducibility overrides.
- Added default-policy, sparse-selection, dense-selection, and adaptive-resolution regression tests.

48 References

1. R. W. Hockney and J. W. Eastwood, *Computer Simulation Using Particles*, Adam Hilger, 1988. Reprint DOI: 10.1201/9780367806934.
2. R. J. Hyndman, "Computing and Graphing Highest Density Regions," *The American Statistician* **50** (1996), 120-126. DOI: 10.1080/00031305.1996.10474359.
3. W. E. Lorensen and H. E. Cline, "Marching Cubes: A High Resolution 3D Surface Construction Algorithm," *Computer Graphics* **21** (1987), 163-169. DOI: 10.1145/37402.37422.
4. T. Lewiner, H. Lopes, A. W. Vieira, and G. Tavares, "Efficient Implementation of Marching Cubes' Cases with Topological Guarantees," *Journal of Graphics Tools* **8** (2003), 1-15. DOI: 10.1080/10867651.2003.10487582.
5. M. Fréchet, "Les elements aleatoires de nature quelconque dans un espace distancie," *Annales de l'Institut Henri Poincare* **10** (1948), 215-310.
6. H. Karcher, "Riemannian center of mass and mollifier smoothing," *Communications on Pure and Applied Mathematics* **30** (1977), 509-541. DOI: 10.1002/cpa.3160300502.
7. M. J. Berger and P. Colella, "Local Adaptive Mesh Refinement for Shock Hydrodynamics," *Journal of Computational Physics* **82** (1989), 64-84. DOI: 10.1016/0021-9991(89)90035-1.
8. I. S. Abramson, "On Bandwidth Variation in Kernel Estimates - A Square Root Law," *The Annals of Statistics* **10** (1982), 1217-1223. DOI: 10.1214/aos/1176345986. Variable-bandwidth KDE is deferred.
9. Plotly Technologies Inc., `plotly.graph_objects.Mesh3d` and `Scatter3d` documentation. Plotly is an optional interactive rendering dependency only.
10. W. G. Cochran, *Sampling Techniques*, 3rd ed., Wiley, 1977. Stratified random sampling is adapted in LD7 for weighted ordered trajectory frames.
11. A. V. Oppenheim, R. W. Schafer, and J. R. Buck, *Discrete-Time Signal Processing*, 2nd ed., Prentice Hall, 1999. The overlap-add organization for bounded linear convolution is adapted in LD8; the periodic triclinic sparse execution is project-specific.
12. M. Garland and P. S. Heckbert, "Surface Simplification Using Quadric Error Metrics," *Proceedings of SIGGRAPH 1997*, 209-216. DOI: 10.1145/258734.258849.
13. W. J. Schroeder, J. A. Zarge, and W. E. Lorensen, "Decimation of Triangle Meshes," *Proceedings of SIGGRAPH 1992*, 65-70. DOI: 10.1145/133994.134010.
14. Linux Kernel Documentation, *Control Group v2*, sections `cpu.max`, `memory.current`, and `memory.max`.
15. Linux Kernel Documentation, *Memory Resource Controller* (cgroup v1), `memory.limit_in_bytes` and `memory.usage_in_bytes`.
16. Python Software Foundation, Python standard-library documentation for `os.cpu_count`, `os.sched_getaffinity`, `resource.getrlimit`, and `RLIMIT_AS`.
17. SchedMD, Slurm documentation for job-step environment variables including `SLURM_CPUS_PER_TASK`, `SLURM_CPUS_ON_NODE`, `SLURM_MEM_PER_CPU`, and `SLURM_MEM_PER_NODE`.
18. Thomas Moreau et al., `threadpoolctl` documentation, used to bound native BLAS/OpenMP thread pools during calibration and scene execution.
19. Python Software Foundation, PEP 544, "Protocols: Structural subtyping (static duck typing)."
20. Python Software Foundation, Python standard-library `logging` documentation.

48.0.1 LD12 - hybrid-aware global density admission (`mdstats 0.19.67a0`)

- Production local-sparse Phase B now uses the LD8 packed source, exact support atlas, and mixed direct/FFT tile planner.
- `kernel_pair_count` means actual direct-tile pairs for hybrid plans and all-direct pairs only for explicit LD7 plans.
- Phase C accounts for FFT work through the calibrated hybrid wall-time estimate and records nominal exact contributions separately.
- Automatic backend selection no longer rejects a feasible mixed direct/FFT scene because its hypothetical all-direct contribution count exceeds the direct-pair cap.

48.0.2 LD14 - Python interpreter hot-path boundary (`mdstats 0.19.71a0`)

Status: implemented and audited in `mdstats 0.19.71a0`.

The density and framework architecture treats Python as an orchestration layer, not a dense numerical execution engine. Numerical work that scales with grid nodes, Gaussian contributions, trajectory-frame/atom products, mesh vertices, mesh faces, or sparse relation rows must execute in NumPy, SciPy, scikit-image, fast-simplification, or a future compiled mdstats extension.

Permitted Python loops operate only over bounded outer units such as fields, tiles, chunks, components, or the three Cartesian axes. Irregular graph searches may remain in Python while representative problem sizes are small, but dominant adjacency, queue, path, and exact-predicate kernels must move to a compiled extension when profiling demonstrates material cost.

The registered interpreter-free kernels currently include packed bitset construction/decoding/popcount, sparse CSR transposition, and exact tiled-contour vertex recovery. An AST regression test prevents Python `for` or `while` loops from being reintroduced into these kernels.

The full policy, benchmark requirements, and remaining candidates are defined in the dedicated performance hot-path specification.

48.0.3 LD15 - Stage-2 interpreter hot-path elimination (`mdstats 0.19.72a0`)

Status: implemented and focused-tested.

The remaining dense numerical bottlenecks identified in LD14 are now array-backed. Cell-list candidate-bin expansion uses bounded vector joins and packed pair deduplication; exact metric-stencil pruning evaluates each of the fixed 27 active-set patterns over batches of candidate boxes. Bond-angle generation uses bounded ragged pair templates and compiled reductions. Support-atlas occupancy and source-target CSR construction use fixed-width uint64 arrays, sorting, and sparse connected-components. Tiled contour outputs are reconciled through one global sort/unique pass, retaining Python clipping only for true canonical-boundary intersections. Fragmented direct sparse realization constructs array schedules per target block and evaluates them in bounded contribution chunks.

These changes do not alter density resolution, Gaussian support, periodic wrapping, topology identity, or resource admission. Vectorized temporaries remain subject to the scene `RuntimeResourceBudget`.

Primitive-ring, natural-tiling, and symmetry searches remain beyond the dense-kernel boundary. Their irregular traversal kernels should move to a deterministic Cython/C++/Rust extension only after representative profiling demonstrates that Python traversal dominates. The extension boundary shall accept immutable integer/CSR arrays and preserve exact periodic shifts and canonical ordering.

48.0.4 LD16 - Interactive mesh/topology revision Stage 1 (`mdstats 0.19.74a0`)

Status: regression baseline implemented; behavior revision not yet implemented.

Stage 1 converts the reported interactive failure modes into deterministic regression fixtures. The package now locks browser-scene failures at 301,838 and 314,640 faces against the historical 300,000-face cap, a sparse-shell failure at 582,375 faces against the historical 250,000-face terminal check, and a real seven-class partitioned projected framework catalog.

The sparse terminal checks share one private count boundary solely so the large case can be tested without retaining a large triangle fixture. The LTA examples share an explicit uniform-catalog guard with unchanged error semantics. No scientific density, mesh, topology, or rendering behavior changes in this stage.

The staged architecture remains: separate raw work from visual targets, add a closed browser-fit loop, add controlled mesh fallbacks, stabilize high-temperature connectivity, then prepare and render one framework/atomic-mean category layer per topology class. The dedicated Stage-1 specification is normative for the regression fixtures.

48.0.5 LD17 - Interactive mesh/topology revision Stage 2 (`mdstats 0.19.75a0`)

Status: explicit face-count contract implemented; closed-loop refitting remains deferred.

Stage 2 removes the overloaded use of `max_mesh_faces`. One immutable `DensityMeshFaceContract` now distinguishes:

1. `raw_extraction_face_limit`, capped by the runtime-derived resource budget;
2. `visual_target_faces`, assigned by the scene allocator and treated as soft;
3. `standalone_final_face_limit`, used only when no scene fitting controller owns the shell.

Interactive scenes use `mode="scene_controller"`, require a visual target, and set the standalone final limit to `None`. The sparse and dense mesh paths may therefore return a valid shell above its initial target. The resulting `DensityMeshFaceReport` records exact fitting debt and becomes the input boundary for LD18/Stage 3. Raw marching-cubes work, periodic seam validity, and standalone terminal limits remain hard.

`DensityRenderOptions.standalone_final_mesh_faces` is the normative standalone option. `max_mesh_faces` remains a compatibility alias in constructors, attributes, and serialized payloads. The historical 250,000-face value is no longer a scene allocation cap.

The final browser budget remains hard in this stage. Scenes initially measuring 301,838 or 314,640 faces may still fail at the final browser check; Stage 3 must close the loop by reallocating and refitting before HTML export. Partitioned framework topology remains unchanged in Stage 2.

49 Revision 0.19.76a0: closed-loop mesh fitting and partitioned topology layers

The renderer now has an explicit control boundary between scientific density preparation and interactive geometry. Dense and sparse contour backends both produce backend-neutral `DensityShellGeometry` records. A closed-loop scene controller measures exact post-replication usage, reallocates visual targets, tries periodic QEM simplification, compensates for simplifier overshoot, and may recontour the same scalar field at lower display resolution. Only an irreducible, exactly measured violation raises `BrowserMeshBudgetFailure`.

Runtime raw-face limits, shell visual targets, and final browser budgets are separate contracts. The default interactive browser profile is `balanced` (600,000 density faces); named compact and quality profiles and a custom budget are available.

Framework dynamics now treats `TopologyCatalog` as a first-class scene input. Global trajectories and density fields are prepared once. Each exact topology class receives its own averaged framework and occupancy-filtered atomic mean graph. Category traces share `framework-topology:<id>` and are toggled as a single Plotly legend group. The most populated category is visible initially; other categories begin as legend-only layers.

The LTA example uses hysteretic T–O connectivity and caches the complete catalog, including frame assignments, segments, transitions, and topology probabilities. This prevents thermal cutoff chatter from being confused with a persistent framework class while preserving genuine high-temperature partitioning.

50 Revision 0.19.78a0: validated sparse-mesh repair and compact topology traces

Two integration defects remained after the closed-loop scene revision. First, a tile-local presimplification could produce an open globally welded surface. The sparse mesher validated that surface only after the simplification boundary and raised before the scene fitter could try a different representation. Sparse extraction now validates the assembled surface first, retries the same tiled contour with local presimplification disabled, and then uses bounded coarse recontouring of the same scalar field and contour level. A failed global simplification restores the last validated geometry. Node-cloud fallback remains the final declared non-mesh path.

Second, partitioned framework rendering reused the general graph renderer for every topology class. Style and species buckets multiplied across categories, producing 340 Plotly traces for a seven-class scene. Partitioned scenes now use a compact adapter: one framework-edge trace, one framework-node trace, one atomic-edge trace, and one atomic-node trace per category. Per-point marker colors preserve species distinction, and all traces remain under the category legend group. The balanced browser profile remains at 96 traces; the implementation reduces trace construction rather than weakening the budget.

51 Stable mesh-pipeline ownership map

Permanent documentation follows source-module responsibility rather than release stage:

Responsibility	Normative specification
shell face-limit meanings	`density_mesh_contracts_spec.md`
exact final browser accounting	`density_render_budget_spec.md`
initial scene allocation	`density_scene_budget_spec.md`
periodic QEM reduction	`density_mesh_simplify_spec.md`
closed-loop fitting and profiles	`density_scene_fit_spec.md`
isolated worker scheduling	`density_mesh_execution_spec.md`
browser interaction acceptance	`density_browser_acceptance_spec.md`
partitioned framework scene and grouped legends	`framework_dynamics_spec.md`
exact topology classes and frame groups	`analysis/topology_catalog_spec.md`

Chronological revision specifications are temporary implementation aids and are removed after their contracts are absorbed into these owners.

52 PAR-DENS long-trajectory density refinement and parallel-execution plan

Status: PAR-DENS0 is `completed` in `mdstats 0.20.120a0`; PAR-DENS1 and PAR-DENS2 are `completed` in `mdstats 0.20.141a0`; PAR-DENS3 is `completed` in `mdstats 0.20.142a0`; PAR-DENS4 is `completed` in `mdstats 0.20.143a0`; PAR-DENS5 is `completed` in `mdstats 0.20.144a0`; PAR-DENS6 is `completed` in `mdstats 0.20.145a0`. The PAR-DENS0–PAR-DENS6 program is production-authorized on the qualified CPU path; CUDA performance remains hardware-conditional.

The trigger for this plan is the long-trajectory LTA density workflow. The current plotting stack already owns a scheduler/cgroup-aware runtime budget through LD10 and already defaults to 90% of available CPUs and 80% of available host memory, but much of density preparation remains serial Python orchestration or single-worker numerical execution. At the same time, LD7 adaptive spread estimation uses a bounded stratified-random temporal sample without distinguishing within-basin vibration from inter-site motion. Correct scientific width estimation must therefore precede aggressive execution optimization.

The implementation order is normative:

1. PAR-DENS0 - basin-aware and convergence-qualified vibrational spread estimation;
2. PAR-DENS1 - execution-faithful direct/FFT cost calibration;
3. PAR-DENS2 - one global resource-aware density scheduler;
4. PAR-DENS3 - parallel density planning and realization kernels;
5. PAR-DENS4 - parallel trajectory preprocessing and reuse of expensive geometry;
6. PAR-DENS5 - optional GPU density execution under an explicit VRAM budget;
7. PAR-DENS6 - end-to-end qualification, auto-tuning, and production authorization.

A later gate may depend on all earlier gates. Performance work may not bypass PAR-DENS0 by coarsening the scientific grid, broadening a Gaussian, changing a density operator, or redefining a transition as vibration.

52.1 Planning benchmark and regression evidence

The reference stress case is the supplied 300 K Na-LTA production trajectory:

- 10,001 frames;
- 168 atoms per frame;
- 24 Na ions;
- current LD7 spread default `sample_size=128` with temporal-stratified random sampling and deterministic seed 0.

The full-trajectory current-definition Na reference spread is approximately `0.0746688 Angstrom`; the basin-aware reference is approximately `0.0746753 Angstrom`. Their near equality is important: the very fine Na adaptive resolution is not primarily an artifact of the single observed basin change. The current 10th-percentile species reference is robust for this particular trajectory, but the per-ion estimator is not semantically correct for mobile ions.

One Na ion provides the required transition-contamination regression case. Its two persistent basin centers are separated by approximately `0.618 Angstrom`, with one long-lived residence on each side of a state change near the midpoint of the trajectory. The current single-global-mean definition gives an apparent spread of approximately `0.19586 Angstrom`, whereas the pooled within-basin vibrational spread is approximately `0.07977 Angstrom`. The new estimator must recover the latter class of quantity and must not include the between-basin displacement in the vibrational variance.

Temporal-stratified sample-size regression against the 10,001-frame reference gave approximately:

Effective frames	Mean reference SD (Angstrom)	Approximate 95% sampling interval (Angstrom)	Mean bias versus full trajectory
32	0.07159	0.06744–0.07653	-4.12%
64	0.07240	0.06883–0.07560	-3.04%
128	0.07343	0.07084–0.07624	-1.66%
256	0.07402	0.07210–0.07577	-0.87%
512	0.07419	0.07293–0.07550	-0.65%
1,024	0.07445	0.07352–0.07525	-0.30%
2,048	0.07459	0.07409–0.07505	-0.10%
4,096	0.07466	0.07443–0.07490	-0.014%
10,001	0.0746688	full reference	reference

The production seed-0 128-frame estimate happened to be much closer to the full reference than the ensemble-average 128-frame bias, so one deterministic seed must not be mistaken for a convergence proof. A target effective coverage near 512 frames is the initial production candidate because it materially reduces sampling error without requiring one large quadratic periodic-medoid solve.

52.2 Scientific invariants for all PAR-DENS gates

The following contracts are fixed throughout the sequence.

1. The scientific density measure, CIC deposition semantics, Gaussian/operator identity, physical grid-resolution rule, HDR definition, periodic wrapping, framework registration, and mesh contour level are unchanged unless a separate scientific specification explicitly revises them.
2. Adaptive spread means **within-basin vibrational spread**. Between-basin displacement, passage frames, transition-region frames, ambiguous membership, unsupported/unknown samples, and conflict samples do not contribute to the vibrational variance.
3. If one atom occupies multiple qualified basins, every qualified basin may contribute its own within-basin covariance, weighted by represented residence time. The variance of the basin centers about one global mean is never added.
4. Performance optimization must not silently coarsen an adaptive grid or inflate a Gaussian bandwidth. A faster backend must realize the same approved scientific field within the declared floating-point tolerance.
5. Host CPU and host-memory bounds are hard resource constraints. Wall-time models remain advisory for backend ranking and progress diagnostics and are not a feasibility veto.
6. Deterministic ordering, cache identity, provenance, and error semantics remain stable across worker-count changes.
7. GPU acceleration is optional. CPU execution remains a complete reference path.

For atom `i` and qualified basin/residence `b`, the target spread is conceptually

$$\sigma_i^2 = \frac{\sum_b W_{ib}\,\langle \|\mathbf{r} - \mu_{ib}\|^2 \rangle_b}{3 \sum_b W_{ib}},$$

where `W_ib` is represented residence weight and `mu_ib` is that basin's periodic mean. This is a within-basin pooled second moment, not a global mixture variance.

52.3 Basin and transition ownership

PAR-DENS0 must reuse existing mdstats state semantics instead of inventing a second incompatible transition classifier.

The preferred evidence order is:

1. If final Stage-11E membership/segmentation is already available, consume the authoritative `CORE/BASIN` membership and `FinalResidenceInterval` lineage from `mdstats.analysis.density.temporal_assignment` and `mdstats.analysis.density.final_segmentation`. Passage samples represented by `FinalPassageInterval` and samples labeled transition/unknown/conflict are excluded.
2. If an explicit geometry-based site assignment is available independently of the high-resolution density, use the corresponding assigned basin classes from `mdstats.analysis.site_assignment` and exclude `TRANSITION_REGION`, ambiguous, and unassigned samples.
3. If neither source is available and the final Stage-11 density-attractor analysis would be circular because it depends on the adaptive density being planned, run a **provisional density-independent coarse residence prepass**. That prepass exists only to separate persistent basins from passage motion for spread estimation. Its labels must not be reused as final kinetic evidence.

A `FinalResidenceInterval.sample_indices` array alone is not sufficient because a residence can include retained excursions. The spread estimator must use the membership class associated with each sample and include only samples whose final classification represents the basin/core vibrational population.

52.4 PAR-DENS0 - basin-aware, convergence-qualified spread estimation

Status: `completed` in `mdstats 0.20.120a0`.

PAR-DENS0 is the scientific-correctness gate and must be completed before any performance gate changes automatic density planning.

Required work:

- introduce a basin-aware spread diagnostic record carrying per-item and per-basin means, second moments, represented weights, accepted/rejected sample counts, transition/passages excluded, reference quantile, and convergence evidence;
- preserve temporal stratification, but stratify within or across qualified residence intervals according to represented time rather than sampling transition frames as if they were vibrations;
- replace one large `O(N_sample^2)` periodic-medoid problem with bounded compact-basin estimators and/or replicated small stratified solves; a deterministic fallback to the existing periodic Frechet/Karcher diagnostic remains required when compactness cannot be certified;
- use replicated stratified coverage as the initial production strategy. Four independent 128-stratum random replicates estimate sampling uncertainty, while deterministic represented-time midpoint anchors at approximately 256 and 512 effective samples establish the production point estimate and convergence without one quadratic medoid solve;
- expose replicate dispersion and convergence diagnostics. Automatic escalation may increase effective coverage when the reference changes materially between levels or uncertainty remains too large;
- retain `numpy.quantile(method="linear")` and the existing validity-filtered species reference unless a separate scientific change is approved.

Minimum acceptance:

- synthetic one-basin trajectories agree with the current estimator within numerical tolerance;
- synthetic two-basin trajectories recover the known within-basin width rather than the mixture width;
- the Na-LTA benchmark keeps the species reference near `0.074675 Angstrom` while the identified two-basin Na ion moves from the approximately `0.196 Angstrom` global mixture spread to approximately `0.080 Angstrom` within-basin spread;
- effective coverage near 512 frames is within 1% of the full-trajectory reference on the Na-LTA benchmark, with the uncertainty estimate reported rather than hidden;

- increasing sample coverage cannot reintroduce the current quadratic runtime growth as the normal compact-basin path.

52.4.1 PAR-DENS0 implementation record (0.20.120a0)

The completed implementation refines the planning candidate in one important way. Four independent 128-stratum random solves are retained for **sampling-uncertainty estimation**, but they are not averaged into the production point estimate. Regression on the Na-LTA trajectory showed that a four-replicate random pooled estimate can still move slightly beyond the 1% point-estimate target for an unlucky seed. The compact periodic-mean fast path removes the reason to accept that seed sensitivity. Production therefore uses a deterministic represented-time midpoint convergence anchor:

1. four independent 128-stratum random replicates estimate sampling dispersion;
2. deterministic 256- and 512-stratum midpoint anchors establish the point estimate and the first convergence comparison;
3. if the relative 256-to-512 change exceeds the configured tolerance, coverage escalates in bounded groups up to the configured maximum replicate-equivalent coverage;
4. random-replicate Student-t uncertainty is centered on the final deterministic anchor, so deterministic convergence is not confused with finite-trajectory statistical certainty.

The implementation adds `BasinSpreadDiagnostic`, `SpreadConvergenceDiagnostic`, and version-3 `PeriodicSpreadDiagnostic` Authoritative labels may be supplied directly, or translated from geometry-based site assignment or from Stage-11E6 final hysteretic segmentation. The Stage-11E6 adapter requires both residence lineage and final `CORE/BASIN` membership; retained excursions, passages, conflicts, and unknown/unresolved samples remain excluded even when they occur inside the span of a residence. When authoritative labels are unavailable, `basin_mode=auto` invokes the conservative density-independent provisional residence prepass specified above.

Compact basins now use a circular/Karcher initialization and an `O(N)`-per-iteration periodic mean path. The historical weighted-medoid multi-start path remains only as a certification fallback for noncompact/ambiguous basins. Consequently, using all 10,001 frames no longer forces the normal `O(N_sample^2)` medoid cost. Low-level direct callers preserve the historical one-replicate/global defaults unless they opt into basin-aware replicated behavior, while production atomic-density resolution defaults to `basin_mode=auto`, four 128-stratum uncertainty replicates, a maximum of eight replicate-equivalent coverage levels, and a 1% convergence threshold.

The supplied 10,001-frame 300 K Na-LTA regression qualified as follows after framework translation registration:

Quantity	Qualified result
full-trajectory global Na reference	0.0746688146 Angstrom
full-trajectory basin-aware Na reference	0.0746688146 Angstrom
512-effective-frame production anchor	0.0746859880 Angstrom
production-anchor error vs full reference	+0.023%
256-to-512 anchor relative change	0.521%
random-replicate 95% relative half-width	3.95%
two-basin Na global mixture spread	0.195859 Angstrom
two-basin Na full within-basin spread	0.079601 Angstrom
two-basin Na production within-basin spread	0.078787 Angstrom
transition-boundary samples excluded for that Na	156

The full species reference is unchanged here because only one of 24 Na ions is transition-contaminated and the approved species reference remains the validity-filtered 10th percentile. That numerical coincidence is not used as the transition-removal mechanism: the per-ion two-basin regression demonstrates that between-basin motion is actually removed. The remaining ~3.95% random-replicate confidence half-width is reported rather than hidden and is distinct from the <1% deterministic convergence criterion.

PAR-DENS0 is therefore closed. PAR-DENS1 may change planner calibration and backend ranking, but may not alter these spread semantics.

52.5 PAR-DENS1 - execution-faithful direct/FFT cost calibration

Status: `completed` in `mdstats 0.20.141a0`; depends on PAR-DENS0.

The current hybrid planner may substantially underprice direct sparse realization because a contiguous arithmetic calibration is not representative of irregular scatter/reduction work. PAR-DENS1 must calibrate the operations that the production executor actually performs.

Required work:

- benchmark destination-index generation, irregular direct accumulation, segmented or block-local reduction, support-atlas operations, and FFT overlap-add at realistic tile sizes;
- calibrate FFT throughput with the same `scipy.fft` worker semantics that production will use;
- record calibration thread count, array dtype, tile shape, source occupancy, contribution count, and temporary-memory footprint;
- separate advisory wall-time ranking from hard memory/thread feasibility;
- retain nominal all-direct contribution counts only as diagnostics; automatic backend selection must rank the actual planned direct/FFT mixture.

Acceptance requires the planner to choose the faster measured route on representative sparse, intermediate, and dense tiles and to avoid the order-of-magnitude underestimation observed in the long-trajectory Na case.

52.5.1 PAR-DENS1 implementation record (0.20.141a0)

The runtime time model advances to `mdstats.density-time-model.v3`. Calibration is synthetic and input-independent but now executes the same cost classes used by the production path: CIC-style coordinate/index preparation, irregular destination-index generation, both bounded `bincount` and `np.add.at` reduction, packed support-region bit operations, and worker-aware `scipy.fft` round trips at the production 32^3 tile scale. Direct reduction is priced from the slower measured irregular-reduction path so the planner cannot underprice direct sparse work merely because `bincount` is fast on a particular host. Array dtype, thread count, tile/kernel/padded shapes, source occupancy, contribution counts, transient calibration memory, and FFT backend are persisted as execution evidence. Wall time remains advisory only.

Representative cross-over regression tests derive direct and FFT costs from the calibrated model and verify direct selection on sparse/intermediate work and FFT selection once the measured cross-over is exceeded.

52.6 PAR-DENS2 - global resource-aware density scheduler

Status: `completed` in `mdstats 0.20.141a0`; depends on PAR-DENS1.

PAR-DENS2 introduces one scene-level execution scheduler analogous in philosophy to the MLFF campaign concurrency controller, but specialized for density work.

The scheduler inherits the LD10 host snapshot and enforces

$$N_{\mathrm{CPU,max}} = \max\left(1, \lfloor 0.90 N_{\mathrm{available}} \rfloor\right)$$

and

$$M_{\mathrm{host,max}} = 0.80 M_{\mathrm{available}},$$

where `available` is affinity-, cgroup-, and scheduler-aware rather than raw machine capacity.

Every schedulable task must expose estimated retained bytes, transient bytes, minimum and preferred worker counts, numerical backend, and parent/child ownership. Admission must satisfy the aggregate peak-memory budget before launch. Native library threads count against the same global CPU pool as Python worker threads/processes; concurrent fields may not each inherit the full scene thread count.

The scheduler must dynamically return CPUs from completed short tasks to remaining heavy tasks. It must support:

- native-thread numerical work for NumPy/SciPy kernels;
- bounded Python threads where the underlying kernel releases the GIL;

- bounded worker processes for genuinely Python-heavy graph/object work;
- deterministic task completion collation independent of completion order.

No worker-count choice may alter scientific output, cache identity, or provenance.

52.6.1 PAR-DENS2 implementation record (0.20.141a0)

`DensitySceneScheduler` now owns one LD10-resolved host budget and admits declared density tasks only when aggregate retained plus active transient memory fits that budget and aggregate minimum worker demand fits the CPU pool. Every task records retained/transient bytes, minimum/preferred workers, execution mode, backend, construction order, and optional parent ownership. Active leases are deterministically water-filled toward preferred worker counts; when short tasks finish, their CPU tokens are returned and may be consumed by surviving heavy tasks. Deterministic collation is by construction order rather than completion order. Bounded thread and spawn-process helpers consume the lease worker count, and nested density APIs inherit the task CPU allocation while retaining the scene memory ceiling; aggregate-memory admission, not a smaller nested planning budget, owns task peaks. Parent failure blocks descendants and preserves the primary deterministic failure.

PAR-DENS2 deliberately does not yet execute independent density fields concurrently. Complete scene preparation establishes and validates per-field contracts, then runs the existing realization order under one scene task lease. PAR-DENS3 owns concurrent field realization and worker-aware kernel partitioning.

Density-scene planning advances to `mdstats.density-scene-plan.v2`: the authoritative scientific approval ID excludes worker count, calibrated timing, storage-backend selection, and other execution-only diagnostics, while a separate `execution_plan_id` retains the complete resource/backend realization for audit. Historical v1 plan JSON continues to deserialize with its original resource-sensitive approval digest. On the qualified 300 K Na-LTA trajectory, a bounded 21-frame/64^3 Na-density smoke produced bit-identical scalar fields at one and four threads (`max |Delta rho| = 0`) and the same v2 scientific approval ID.

52.7 PAR-DENS3 - parallel density planning and realization

Status: `completed` in `mdstats 0.20.142a0`; depends on PAR-DENS2.

This is the primary CPU-performance gate.

Required work:

- schedule independent atomic/framework density fields concurrently when aggregate memory admission permits;
- parallelize support-atlas/source-block construction over independent bounded block groups;
- partition direct sparse realization by destination ownership so workers do not contend for the same output block;
- replace repeated hot `np.add.at` scatter where practical with grouped/segmented reductions such as destination sorting plus `bincount` or equivalent compiled reductions;
- standardize production FFT execution on worker-aware `scipy.fft` calls and allocate FFT workers from the global scheduler rather than from a per-field unconstrained maximum;
- allow a single dominant field to consume CPUs released by short fields rather than statically assigning one quarter of the machine to each species;
- preserve exact finite-support, packed-field, normalization, and supported-node repair contracts from LD8.

The acceptance suite must compare scalar fields, total integrated measure, HDR levels, periodic support, and resulting meshes against the serial/reference path under multiple worker counts.

52.7.1 PAR-DENS3 implementation record (0.20.142a0)

Independent atomic and framework field realization now executes as separate `DensityScheduledTask` objects under the single PAR-DENS2 scene scheduler. Exact Phase-B resource contracts remain authoritative for admission. When memory permits, fields overlap; scheduler leases are deterministically water-filled toward preferred worker counts and a field still running after siblings finish can consume the returned CPU tokens. Completion order remains non-authoritative: realized fields are collated by construction order before Phase-B/realization authentication. Established plotting errors are unwrapped at the facade so parallel execution does not change public error semantics.

Sparse Phase-B planning and realization now use bounded parallel kernels. Support-atlas source blocks are processed in independent groups sized jointly by the active CPU budget and available transient memory. Before a field lease

exists, Phase-B construction uses the global LD10 host budget; inside a scheduled task the same code automatically uses the live field lease. Target-owned direct realization is partitioned by destination block so each worker owns a private accumulator and writes only a disjoint packed output range. Hybrid direct scatter replaces repeated global `np.add.at` operations with stable destination grouping plus compiled segmented reduction where practical. Dense, tiled, and support-dilation FFT paths use worker-aware `scipy.fft`; live FFT worker counts come from the scheduler lease and therefore cannot oversubscribe the scene CPU pool.

Density-scene planning advances to `mdstats.density-scene-plan.v3`. The scientific approval projection now excludes sparse storage geometry, direct/FFT executor and tile selection, hybrid execution-plan identities, FFT worker counts, cache hits, calibrated timing, and memory/work decomposition. These remain serialized in the separate execution plan. Historical v1 resource-sensitive approval semantics and v2 PAR-DENS2 scientific-digest semantics remain separately versioned and round-trip without reinterpretation.

The serial/parallel acceptance matrix covers dense and local-sparse framework fields at one and four workers. It requires exact scalar values, integrals, packed periodic support, field content identities, 50/80/95% HDR thresholds, and resulting sparse mesh geometry/topology to agree. The density-focused regression surface closes with 351 passing tests and one optional `mdstats[interactive]` mesh-simplification skip.

The supplied 10,001-frame 300 K Na-LTA source was requalified with a bounded 101-frame stride-100, 64^3 local-sparse Na/Si/O scene. At four workers the scheduler admitted all three fields concurrently. One- and four-worker results had `max |Delta rho| = 0` for every field, identical packed-field content identities, identical v3 scientific approval IDs, equal integrated measures, and equal 50/80/95% HDR thresholds. Scene preparation measured 9.209 s at one worker and 8.304 s at four workers in the qualification environment (1.109x). This bounded timing is evidence that concurrency is active, not a universal speedup guarantee; very small scenes may remain scheduler-overhead dominated.

PAR-DENS3 is therefore closed. PAR-DENS4 may optimize trajectory preprocessing and geometry reuse but may not alter the density scientific contracts established here.

52.8 PAR-DENS4 - parallel trajectory preprocessing and geometry reuse

Status: `completed` in `mdstats 0.20.143a0`; depends on PAR-DENS2 and follows PAR-DENS3.

This gate removes secondary long-trajectory bottlenecks outside density convolution.

Required work:

- split hysteretic connectivity into a frame-parallel geometric candidate stage and a deterministic ordered hysteresis fold;
- reuse compatible per-frame neighbor geometry between framework-only and full atomic connectivity instead of repeating minimum-image/candidate work;
- parallelize framework registration/lifting over independent frames, followed by deterministic consistency validation;
- parallelize independent periodic atomic-mean calculations and other per-item reductions;
- parallelize topology-category scene preparation when categories are independent and aggregate memory admission permits;
- hoist trajectory-wide quantities out of topology-category loops so parallelism does not amplify redundant work.

LAMMPS text parsing, small graph bookkeeping, and lightweight Plotly object assembly are not priority GPU or high-thread targets unless profiling later proves otherwise.

52.8.1 PAR-DENS4 implementation record (0.20.143a0)

Hysteretic atomic connectivity is now explicitly two-stage. The geometric outer/inner candidate sets for independent frames may be generated in bounded worker threads with stateless exact neighbor searches, while the bond-retention/formation state machine is folded once in authoritative collection-frame order. The parallel candidate path uses contiguous frame chunks to amortize executor/diagnostic setup. State canonicalization, frame-state IDs, transition records, and edge-image shifts are unchanged by worker count.

`AtomicConnectivityGeometryCache` provides an execution-only cache keyed by exact collection identity, frame, center/candidate atom sets, cutoff, pair-counting policy, block size, and neighbor-search options. Compatible framework-only and broader atomic-connectivity passes therefore reuse the same periodic minimum-image/candidate

work instead of rebuilding it. Cache state and hit/miss counters are diagnostic only and are excluded from scientific connectivity identity.

Framework scene preprocessing now owns a reusable frame-geometry cache for projected framework graph views and lifted fractional coordinates. Independent frames are prepared under a PAR-DENS2 scheduler lease and then graph-key consistency and residual winding are validated in deterministic frame order. Independent periodic atomic means are likewise prepared in bounded threads. Floating occupancy accumulation remains in canonical frame order so parallel preprocessing cannot alter threshold decisions by changing summation order.

Partitioned topology scenes hoist the frame-to-connectivity-state map and framework geometry cache outside category loops. Independent category preparation is admitted as separate scheduler tasks under the same global CPU/RAM authority and nested work inherits its active lease. This prevents topology concurrency from multiplying worker budgets or recomputing trajectory-wide geometry.

The framework-dynamics scene schema advances to `mdstats.framework-dynamics-scene.v15` and records the preprocessing policy, scheduler summary, cache statistics, and trajectory-wide-state reuse evidence. These execution records do not redefine density, topology, registration, or graph scientific semantics.

The supplied 10,001-frame 300 K Na-LTA trajectory was requalified on a stride-100 101-frame sample. Serial and four-worker hysteretic preparation produced identical frame-state IDs, 94 canonical state digests, and transition records. The small 168-atom/frame dense neighbor problem is thread-overhead dominated in the qualification environment, so four-worker candidate generation is not claimed as a speedup. The geometry-reuse path is materially beneficial: after framework-only connectivity populated the shared cache, broader framework+Na-O connectivity reused 202 exact neighbor requests and measured 1.024 s versus 1.730 s for a cold full pass, about 1.689x, with identical connectivity. Parallelism therefore remains bounded and resource-aware rather than mandatory at unfavorable task granularity.

PAR-DENS4 is closed. PAR-DENS5 may add an optional GPU density backend, but the CPU reference path and all PAR-DENS0–PAR-DENS4 scientific invariants remain authoritative.

52.9 PAR-DENS5 - optional GPU density backend

Status: `completed` in `mdstats 0.20.144a0`; PAR-DENS0–PAR-DENS4 establish the qualified CPU reference path.

GPU work is an acceleration backend, not a scientific mode. Candidate kernels are CIC deposition, finite-support Gaussian accumulation, support-mask dilation, grouped sparse reduction, and sufficiently large 3-D FFT tiles.

GPU admission must auto-detect device availability and free/usable memory and enforce

$$M_{\mathrm{GPU,max}} = 0.80 M_{\mathrm{GPU,available}}.$$

The first implementation permits at most one major density job per GPU and batches its tiles internally rather than launching competing species jobs that cause VRAM churn. Host staging buffers remain subject to the 80% host-memory budget.

FP64 scientific accumulation remains the reference contract. FP32 or mixed-precision density accumulation is not authorized by this gate and would require separate numerical-error qualification. CPU fallback remains automatic and complete.

Automatic GPU selection includes host-device transfer and setup cost; a kernel is not selected merely because a GPU exists.

52.9.1 PAR-DENS5 implementation record (0.20.144a0)

The optional accelerator is implemented in `mdstats.plotting.density_gpu`. CUDA is runtime-discovered through PyTorch only when PyTorch is already present in the user environment; mdstats does not add Torch or CuPy as a hard plotting dependency. The execution policy defaults to `MDSTATS_DENSITY_GPU=auto`, accepts `off` and `force` for reference/diagnostic runs, and always enforces the memory contract even under `force`. Device admission snapshots currently free VRAM and exposes only `floor(0.80 * free)` bytes to density work. Host transfer/staging bytes are checked against the active PAR-DENS2 memory lease before GPU admission.

One process-global lock per CUDA device implements the first-gate one-major-job policy. A scheduled density field acquires the device lazily only after one of its kernels passes cost and memory admission, retains that ownership

across its internal tiles, and releases/synchronizes at field completion. Concurrent species/framework fields that cannot obtain the device continue on CPU instead of waiting and creating GPU memory churn. Low-level standalone calls use the same lock at kernel scope.

The automatic selector estimates GPU wall time as setup plus host/device transfer plus a calibrated/conservative fraction of the qualified CPU estimate. Work below the GPU amortization floor, transfer-bound work, insufficient-VRAM work, unavailable CUDA, and a busy field-owned GPU are rejected before allocation. A runtime CUDA failure also returns to the original CPU kernel. GPU decisions are recorded in a bounded journal with complete aggregate reason/kernel counts, device/free/usable-memory evidence, data transfer, required VRAM, and predicted CPU/GPU wall times.

The initial CUDA realization remains FP64 throughout and covers the operations whose memory traffic can reasonably amortize device setup without changing scientific semantics:

- dense CIC deposition uses FP64 source weights, sorted flattened destination nodes, stable/ordered grouping where available, FP64 cumulative segmented reduction, and unique-node assignment rather than an uncontrolled final-grid atomic scatter;
- canonical periodic and legacy spectral Gaussian smoothing use FP64 CUDA FFTs while preserving the existing discrete/spectral operator definitions;
- binary support dilation may use FP64 CUDA linear FFT convolution and still passes the existing integer-roundoff certificate before thresholding to exact support;
- hybrid sparse/tiled FFT realization may execute sufficiently expensive FP64 tile convolutions on CUDA under the field-owned device lease.

Target-owned grouped direct sparse accumulation deliberately remains on CPU in this gate. Its irregular packed traffic would otherwise require frequent host/device movement or a new scatter/reduction authority. PAR-DENS5 does not claim acceleration where the transfer and deterministic-reduction design has not been qualified.

GPU execution is excluded from field content identity and density-scene scientific approval. It is execution evidence only. This preserves the PAR-DENS2/PAR-DENS3 rule that worker/backend choice cannot change cache or scientific provenance. Kernel metadata may state `torch_cuda_fp64` versus `scipy_fft_cpu`, but operator identity, grid, support, integral, HDR definition, and density-content identities are unchanged.

The supplied 10,001-frame 300 K Na-LTA trajectory was requalified on the same bounded stride-100, 101-frame, 64^3 local-sparse Na/Si/O density workload. The packaging host has no CUDA runtime, so `auto` correctly reports `cuda_unavailable` and exercises the complete CPU fallback. Explicit `off` and automatic execution produce `max |Delta rho| = 0` for every packed field, identical integrated populations, identical field content identities, and identical 50/80/95% HDR thresholds. Conditional real-CUDA unit tests compare the FP64 circular-FFT and CIC implementations with their qualified CPU references and run automatically on CUDA-capable hosts; they are skipped on this packaging host. Consequently no GPU performance factor is claimed here. PAR-DENS6 owns hardware-scale CPU/GPU throughput, VRAM-peak, and auto-tuning qualification.

PAR-DENS5 is therefore closed. PAR-DENS6 is the final PAR-DENS gate.

52.10 PAR-DENS6 - end-to-end qualification and auto-tuning

Status: `completed` in `mdstats 0.20.145a0`; CPU production path authorized. GPU performance remains conditional on a CUDA-capable qualification host.

PAR-DENS6 turns the previous gates into a production policy rather than a collection of local optimizations.

Required qualification uses at least three trajectory scales, including approximately 100, 1,000, and 10,000+ frames, and records:

- basin-aware spread estimate and its convergence/uncertainty diagnostics;
- preprocessing, planning, realization, contouring, and total wall time;
- CPU utilization, scheduler-resolved CPU ceiling, worker allocation history, and oversubscription checks;
- measured and predicted peak host memory versus the 80% budget;
- direct/FFT tile choices and predicted-versus-observed timing;
- GPU utilization and VRAM peak when GPU execution is enabled;
- field-integral, pointwise/fidelity, HDR, mesh, and deterministic-repeat comparisons against the approved reference path.

Auto-tuning may choose field-level concurrency, tile-group size, direct versus FFT, FFT worker count, and CPU versus GPU execution. It may **not** tune scientific resolution or operator identity for speed.

Production authorization requires that the 90% CPU and 80% host-memory safeguards are obeyed under scheduler/cgroup restrictions, that optional GPU execution obeys the 80% VRAM safeguard, and that the long-trajectory benchmark shows material speedup without scientific regression. No fixed speedup factor is architecture-mandated before measurement; the qualification record must report the achieved factor honestly.

52.10.1 PAR-DENS6 implementation record (0.20.145a0)

PAR-DENS6 closes the long-trajectory density program with a hardware-local, execution-only auto-tuner and an authenticated Na-LTA production qualification. The tuner may cap field concurrency, work-group depth, FFT workers, and optional CPU/GPU execution, but the grid, Gaussian bandwidth, support/operator definition, normalization, HDR levels, and scientific content identities remain immutable. The previously qualified PAR-DENS3 group multiplier of four is retained unless a future workload-aware authority supersedes it; the current isolated microprobe is recorded diagnostically because it does not model cooperative lease redistribution well enough to override that baseline.

The final gate also closes two resource/identity gaps found only at 10,001-frame scale. First, the provisional basin prepass no longer submits all item trajectories to one giant triclinic minimum-image batch; it processes bounded item blocks so MIC workspace scales with a bounded number of step vectors while preserving the same per-vector geometry and basin semantics. Second, sparse CIC transient workspace is carried into Phase-B scheduler contracts and dead geometry temporaries are released before stable reduction. The PAR-DENS2 scheduler admits the largest-peak independent field first so retained outputs from smaller siblings cannot strand a later high-workspace field.

Most importantly, the direct-versus-FFT tile partition is now frozen by the scene-level Phase-B planning authority **before** cooperative field workers are admitted. A live worker lease may change the number of FFT/native workers used to execute an already-approved tile, but may not re-plan the tile as direct or FFT. This distinction is necessary because the two algebraically equivalent paths have different floating-point reduction orders. The prior lease-local re-plan produced a single-ulp O-field difference on the 10,001-frame Na-LTA case; the frozen Phase-B execution plan restores exact worker-count invariance without weakening tolerance. Execution-plan identity and calibration timing remain execution evidence and do not enter scalar-field scientific identity.

Final CPU qualification uses the supplied 300 K Na-LTA trajectory, whose authenticated SHA-256 is recorded as:

`81c86cc40f5a11031f80817213eb558c02348494d1c6cad9b4775a5bc3c9f9cd`

The trajectory contains 10,001 frames x 168 atoms, with a fixed Na/Si/O `64^3` local-sparse operator and `0.5 Angstrom` Gaussian bandwidth. Independent fresh-process benchmark legs were used so allocator retention from one scale cannot warm or fragment the next. The 101-, 1,001-, and 10,001-frame auto paths all obeyed the scheduler CPU/RAM authority. At 10,001 frames, two auto repeats took `17.7906 s` and `18.0370 s`; two one-worker reference repeats took `19.3897 s` and `20.5076 s`. Median total-wall speedup is therefore `1.1136x` (11.36%), while median scheduled-realization speedup is `1.1203x`. The maximum measured full-density-stage RSS growth among the long repeats is about `640.3 MB`, below every corresponding dynamic 80%-RAM budget (about 1.33–1.34 GB), and declared scheduler peaks also remain within budget.

Na, Si, and O are pointwise **bit-identical** between the auto and one-worker paths: `max |Delta rho| = 0` for all three species, content identities match, integrals and 50/80/95% HDR thresholds match, the direct/FFT executor partition matches, and independent deterministic repeats match. The bounded full-framework audit records preprocessing/planning/realization timing and Phase-B predicted-versus-observed field costs; the current model underpredicts absolute wall time but correctly identifies O as the dominant field, so its timing remains a relative backend-ranking model rather than a hard realization bound. The authenticated evidence is stored in `release/par_dens6_na_lta_qualification.json`.

This packaging host has no CUDA runtime. PAR-DENS5 no-CUDA fallback remains exact, and CUDA numerical tests remain conditional. PAR-DENS6 therefore authorizes the CPU production path and the auto-tuning policy; GPU performance authorization must be obtained on a CUDA-capable host without changing any scientific contract.

52.11 Planned ownership boundaries

The permanent implementation should extend existing owners rather than create a parallel density subsystem:

Responsibility	Owning or expected module family
basin-aware spread diagnostics and stratified convergence	`mdstats.analysis.density.diagnostics`
explicit geometry-based site membership	`mdstats.analysis.site_assignment`
Stage-11 temporal membership/residence evidence	`mdstats.analysis.density.temporal_assignment`, `mdstats.analysis.density.final_segmentation`
scene resource snapshot and hard CPU/RAM admission	`mdstats.plotting.runtime_resources`
atomic density planning/realization	`mdstats.plotting.atomic_density` and existing density execution helpers
framework density planning/realization	`mdstats.plotting.framework_density` and existing density execution helpers
framework registration and scene preparation	`mdstats.plotting.framework_dynamics`
optional GPU execution policy	new density execution helper(s), subordinate to `runtime_resources` rather than a second resource manager

A gate-specific specification may refine low-level APIs, but this section owns the ordering, scientific invariants, and resource ceilings for the complete PAR-DENS program.